# Distributed Monotone Policy Encryption with Stronger Security for DNFs and Threshold Policies from Lattices

Rishab Goyal
UW-Madison*

Saikumar Yadugiri
UW-Madison†

**Abstract**

Distributed monotone-policy encryption (DPE) lets each user sample and publish its own key, after which anyone can encrypt to a list of published keys under a monotone access policy that determines which coalitions can decrypt. Silent threshold encryption is the $t$-out-of-$N$ special case. What makes the primitive non-trivial is compactness, where the ciphertext stays sublinear in the policy description. Every post-quantum DPE scheme so far settles for selective security, fixing the challenge policy and the corrupted positions before setup, and complexity leveraging cannot close the gap without giving up compactness. The one DPE scheme known in the stronger static model, where the policy and the placement of malicious keys are chosen adaptively, relies on witness encryption (Devadas-Jain-Waters-Wu, Asiacrypt'25).

We give the first statically secure DPE schemes from falsifiable lattice assumptions. For DNF policies, ciphertexts are of size $\mathsf{poly}(\lambda, \log N)$, independent of the number and widths of the clauses, and public keys, secret keys, and partial decryptions are of size $\mathsf{poly}(\lambda)$. For $t$-out-of-$N$ threshold policies, ciphertext of size $\tau^6 \cdot \mathsf{poly}(\lambda)$ for $\tau = \min(t^2, N-t)$, improving to $\tau^2 \cdot \mathsf{poly}(\lambda)$ given a common reference string. We prove security under decomposed LWE, and the improved threshold parameters under succinct LWE, in the random oracle model.

Our constructions generalize the equivocal encryption framework of Goyal-Yadugiri to policies. We define equivocal DPE, which simulates public keys and partial decryptions and withholds the equivocation trapdoor while releasing the public coins that accompany a ciphertext, and compiles to static DPE with no loss in parameters.

## 1 Introduction

Threshold encryption distributes decryption authority across a set of users: a sender encrypts to a committee, and only an authorized coalition recovers the message. Traditional realizations fix the committee key through a trusted dealer or an interactive distributed key-generation protocol, so the setup requires coordination, and any change to the committee or to its access structure requires running the setup again. Silent threshold encryption (STE) eliminates this requirement [DHMR07, DHMR08, WQZD10, RSY21, GKPW24, ADM+24]. Each user samples a public-secret key pair on its own and publishes the public key; a sender later selects a committee and a threshold at encryption time, without interacting with the recipients and without any party holding a master secret key. Asynchronous enrollment together with distributed decryption is what makes STE attractive for committee-based protocols such as threshold-encrypted mempools [BCF+25], and aligns with the ongoing effort to standardize multi-party threshold schemes [BP23].

Distributed monotone-policy encryption (DPE) replaces the threshold predicate with a class of monotone access policies [DJWW26, WW26]. An encryptor takes a policy $P$, a sequence of independently generated public keys $(\mathsf{pk}_1, \ldots, \mathsf{pk}_N)$, and a message $\mu$. Each user produces a partial decryption using its secret key, and a collection of shares recovers $\mu$ exactly when the contributing users satisfy $P$. The syntax captures distributed broadcast encryption through the 1-out-of-$N$ policy, STE through a $t$-out-of-$N$ policy, and richer authorization rules described by monotone formulas, including DNFs. Three properties are required jointly:

*Email: rishab@cs.wisc.edu.
†Email: saikumar@cs.wisc.edu.

1. *Silent setup*: users generate keys independently, without a dealer, no key escrow, and no interactive key generation.
2. *Policy flexibility*: the encryptor selects both the recipient keys and the access policy dynamically, and users generate keys without a policy in mind.
3. *Compactness*: the ciphertext is sublinear in the policy description, ideally of size $\mathsf{poly}(\lambda, \log N)$.

Compactness makes DPE and its various specializations non-trivial and practically useful. Without it, one can secret-share the message and separately encrypt a share to every relevant user, at which point the ciphertext reproduces the size of the policy and none of the compression sought from STE or DPE remains.

**The security gap and complexity-leveraging barrier.** The natural security experiment lets an adversary initialize honest users, supply malformed public keys of its own, corrupt honestly generated users, request partial decryptions of chosen ciphertexts, and select the challenge policy and target recipients after performing all such actions. The desired goal is to have the challenge message remain hidden as long as the corrupted and maliciously generated keys do not satisfy the challenge policy. In the fully adaptive experiment, corruptions and partial-decryption queries continue after the adversary sees the challenge ciphertext, so the ciphertext itself may inform which honest users the adversary decides to corrupt later on.

Current DPE constructions [DJWW26, CW26a, AGY26] are currently far from attaining (full) adaptive security. The most common approach taken by the majority of prior works [RSY21, GKPW24, ADM+24, WW26, CW26a, AGY26] is to establish security in a restrictive, *selective* security experiment. In the selective experiment, the adversary must fix the challenge policy $\mathcal{P}$ together with the corrupted positions $\mathcal{C}$ before seeing the system parameters. As an example, for a DNF-based access policy this means committing in advance both to the formula and to an unsatisfying assignment that determines which wires the adversary controls. Although the selective model does not capture real-world threats in a meaningful way, it serves as a useful starting point to attain full adaptive security, as demonstrated by the vast literature in a variety of advanced encryption systems [BF01, BB04, Wat05, GPSW06, Wat09, LOS+10, GPV08, GW09, ABB10, CHKP10, Wat15, ABSV15, DG17b, DG17a, KMW23, GKR25, CHW25, WW26, GY26b, AMR26, GY26a].

While the folklore 'complexity-leveraging' paradigm (i.e., selectively guessing the adversary's choices) typically suffices for upgrading a selectively secure encryption scheme to a full adaptively secure scheme (at the cost of super-polynomial hardness assumptions), this approach fails miserably for closing the selective vs. adaptive gap for DPE schemes. The reason is that a reduction that plays against a selectively secure challenger has to guess both the eventual policy and the eventual corruption pattern, and for a policy over $N$ users, there can be exponentially many admissible corruption patterns. Absorbing this loss forces the security parameter to grow with $N$, which in turn makes the ciphertext grow polynomially with $N$. The resulting scheme satisfies an adaptive definition as a matter of asymptotics while losing the parameter guarantee that made DPE worth studying; the same tension is what separates selective from adaptive security in broadcast encryption [FN94, BGW05].

**The static model.** Given the large gap between selective and adaptive security notions for DPE, the community has considered an intermediate security notion called the *static* security model [GW09, DJWW26].[1] Here the adversary sees the system parameters, initializes honestly generated public keys, supplies malicious public keys, requests partial decryptions under honest keys, and only then chooses the challenge policy together with the assignment of honest and malicious keys to its wires. What it may not do is corrupt an honestly generated key: adversarial control is exercised through maliciously sampled keys, and once the challenge ciphertext is out, the adversary may no longer query partial decryptions of it under honest keys. The static model permits the adaptive choice of policy, placement of corrupt users, and their keys, but removes user corruption queries for honestly instantiated users.

For distributed broadcast encryption, static security is already known to sit next to adaptive security.[2] Gentry and Waters gave a two-key compiler in the random oracle model, which was later extended to the

[1] We remark that the terminology is not uniform across the DPE literature: some prior works refer to the selective model described earlier as 'super-static' [RSY21], while some refer to selective as 'static' itself [WW26]. For consistency with prior literature (such as distributed broadcast encryption) and to avoid overloading terminology, we use static model to refer to intermediate model and selective/adaptive to be the two ends of the spectrum.

[2] In broadcast encryption, the static model is usually referred to as the semi-static model [GW09].

plain-model [GW09, KMW23, HWW25], to construct an adaptively secure scheme from any statically secure scheme. These transformations exploit the set-membership structure of a 1-out-of-$N$ policy. Although similar transformations are not yet known for more general DPE access policies (since a satisfying coalition would combine keys across several parts of the policy), conventional wisdom suggests that attaining static security would drastically cut down the gap to full adaptive security.

**Prior approaches beyond broadcast encryption.** Early works designed DPE for threshold policies satisfying weak compactness [DHMR07, DHMR08] or assuming obfuscation [RSY21]. More recent works build DPE for threshold policies from idealized algebraic models [GKPW24], for monotone formulas under pairing-based $q$-type assumptions [WW26], for DNFs and sublinear threshold policies from decomposed/succinct LWE [AGY26, CW26a], and for expressive policies from succinct witness encryption [DJWW26]. In terms of security, nearly all prior works prove security in the selective model (or mild strengthenings generically implied by it) [RSY21, GKPW24, ADM+24, WW26, AGY26, CW26a]. One notable exception is [DJWW26] which designs a statically secure DPE scheme, but assumes witness encryption [GGSW13]. Since witness encryption is a non-falsifiable assumption, the following is a major open question in the study of DPE:

*Can DPE for monotone policies achieve static security from a falsifiable assumption?*

Outside of distributed/flexible broadcast encryption (DPE for 1-out-of-$N$) [KMW23, CHW25, WW26, GY26b, AMR26, GY26a], we do not have even a partial solution to the above.

## 1.1 Our Results

We answer this question affirmatively for DNF policies and threshold policies from falsifiable lattice assumptions. We provide two DPE constructions, one for DNFs and other for threshold policies, that are statically secure under decomposed/succinct LWE assumption in the random oracle model. Decomposed/succinct LWE is a falsifiable lattice assumption designed to supply the compression that $q$-type assumptions provide in pairing-based cryptography [AMR25, Wee24]. We prove the following two theorems.

**Informal Theorem 1.** *Assuming hardness of the decomposed LWE assumption, there exists a statically secure DPE scheme for DNF policies in the random oracle model. The ciphertext size is* $\mathsf{poly}(\lambda, \log N)$*, and each user's public key, secret key, and partial decryption size is* $\mathsf{poly}(\lambda)$*.*

**Informal Theorem 2.** *Assuming hardness of the decomposed LWE assumption, there exists a statically secure STE scheme for $t$-out-of-$N$ threshold policies in the random oracle model with ciphertexts of size $O^{\dagger}_{\lambda,N}(\tau^6)$, public keys and partial decryptions of size $O^{\dagger}_{\lambda,N}(\tau^2)$, where $\tau = \min(t^2, N-t)$ and $O^{\dagger}_{\lambda,N}(\cdot)$ hides $\mathsf{poly}(\lambda, \log N)$ terms. Assuming additionally a common reference string, the succinct LWE assumption improves the ciphertext compactness to $O^{\dagger}_{\lambda,N}(\tau^2)$.*

These results improve the security of expressive lattice-based silent encryption without sub-exponential hardness or complexity leveraging. Post-challenge corruption of an honest challenge key stays outside the model, so full adaptive security for policies beyond set membership remains open. We summarize our results and compare it with the state of the art in Table 1.

**Concurrent Work.** Independently, Champion and Wu [CW26b] constructed a distributed monotone policy with static security for DNFs. We provide a detailed comparison between our work and theirs below.

**Optimal parameters in random oracle model.** Both our work and [CW26b] constructs a statically secure DPE scheme for DNFs where all parameters are optimal and do not rely any a-priori bound in the random oracle model. On a technical level, both works do not use secret sharing for constructing DPE for DNFs unlike the prior work of [AGY26, CW26a]. On the other hand, we rely on EqMCOM proposed by [GY26a] in the context of flexible broadcast encryption and show that the same tool can be used for DPE as well. [CW26b] uses the matrix commitment scheme of [Wee25] directly.

| Reference | Assumption | Policy | $\lvert\mathsf{crs}\rvert$ | $\lvert\mathsf{pk}\rvert$ | $\lvert\mathsf{sk}\rvert$ | $\lvert\mathsf{ct}\rvert$ | TP | PQ | No RO | SS |
|---|---|---|---|---|---|---|---|---|---|---|
| [DJWW26] | WE + LWE | DNFs | – | 1 | 1 | 1 | ✓ | – | ✓ | ✓ |
| [AGY26] | decomposed LWE | DNFs | 1 | $R$ | 1 | 1 | ✓ | ✓ | ✗ | ✗ |
| [CW26a] | decomposed LWE | DNFs | 1 | $R$ | 1 | 1 | ✓ | ✓ | ✗ | ✗ |
| [RSY21] | $i\mathcal{O}$ + OWF | Threshold | – | 1 | 1 | 1 | – | – | ✓ | ✗ |
| [ADM+24] | $i\mathcal{O}$ +LWE | Threshold | – | 1 | 1 | 1 | ✗ | ✗ | ✓ | ✗ |
| [GKPW24] | Generic Bilinear Group | Threshold | $N$ | $N$ | 1 | 1 | ✗ | ✗ | ✓ | ✗ |
| [WW26] | N-Extended BDHE | Threshold | $N \log N$ | $N$ | 1 | 1 | ✗ | ✗ | ✓ | ✗ |
| [AGY26] | decomposed LWE | Threshold | 1 | $\tau$ | 1 | $\tau^6$ | ✓ | ✓ | ✗ | ✗ |
| Corollary 5.4 | decomposed LWE | DNFs | 1 | 1 | 1 | 1 | ✓ | ✓ | ✗ | ✓ |
| Corollary 6.8 | decomposed LWE | Threshold | 1 | $\tau^2$ | 1 | $\tau^6$ | ✓ | ✓ | ✗ | ✓ |

Table 1: Comparison of our results with prior works on distributed monotone policy encryption. The results in the table are for policies expressive than trivial 1-out-of-$N$ thresholds that covers the notion of (distribute/flexible) broadcast encryption. $\lvert\mathsf{crs}\rvert, \lvert\mathsf{pk}\rvert, \lvert\mathsf{sk}\rvert$, and $\lvert\mathsf{ct}\rvert$ denote the sizes of the common random string / master public key, user public keys, user secret keys, and ciphertexts respectively for $N$ users. The values reported in these columns hide $\mathsf{poly}(\lambda, \log N)$ terms. For instance, if $\lvert\mathsf{crs}\rvert$ is reported as $X$, it signifies $\lvert\mathsf{crs}\rvert = X \cdot \mathsf{poly}(\lambda, \log N)$. **TP**, **PQ**, **SS**, and **No RO** denote transparent setup, post-quantum, static security, and absence of random oracles respectively. BDHE stands for the Bilinear Diffie-Hellman Exponent assumption [Wat11, WW26]. Here, $R$ denotes the number of times a user can be reused in the policy for encryption. For schemes whose parameters grow with $R$, this should be declared a-priori during $\mathsf{crs}$ generation. The value $\tau$ in the last row denotes $\tau = \min(t^2, N-t)$ for $t$-out-of-$N$ threshold policies. As we remark later, further assuming a common reference string, ciphertext size in Corollary 6.8 can be further reduced to $\tau^2$.

**Silent threshold encryption.** The main distinguishing feature between our work and theirs is that we construct a silent threshold encryption from lattices in the random oracle model with sub-linear size ciphertexts for non-trivial thresholds that are not possible from DNFs. [CW26b] does not consist any result for threshold policies beyond what is possible from DNFs.

**Standard model instantiations and other results.** [CW26b] constructs a DPE for DNFs with sub-linear size ciphertexts (and compact public-secret keys, partial decryptions) in the standard model. Moreover, they also construct an adaptively secure flexible broadcast encryption scheme in the standard model.

# 2 Technical Overview

Our starting point is the equivocal encryption framework of Goyal-Yadugiri [GY26b, GY26a]. The framework splits an adaptive-looking proof into two tasks: build a fake mode that postpones every challenge-dependent choice until the attacker reveals it, and show that the ciphertext prepared in that fake mode hides the message. Existing formulations carry this out for broadcast encryption, where the only delayed choice is a set of recipients. DPE delays considerably more, since the challenge consists of a policy, an assignment of honest and adversarial public keys to its inputs, and a transcript of partial decryptions that fake mode must keep consistent with every earlier oracle answer. We first formulate an equivocal DPE abstraction that exposes this richer interface (Section 4.1), and then construct it for DNF policies (Section 5) and threshold policies (Section 6).

## 2.1 Equivocal DPE

**Syntax.** An equivocal DPE (EqDPE) scheme contains the real algorithms $\mathsf{GSetup}, \mathsf{KGen}, \mathsf{PreProcess}, \mathsf{Enc}, \mathsf{PartDec}, \mathsf{Recon}$ of a DPE scheme, together with a fake setup $\mathsf{FakeSetup}(1^\lambda, 1^\kappa, \mu) \to (\widetilde{\mathsf{crs}}, \widetilde{\mathsf{ct}}, \mathsf{aux})$ that samples a fake CRS and a message-dependent fake ciphertext along with a trapdoor $\mathsf{aux}$, a stateful public-key simulator $\mathsf{SimKGen}$, a corruption simulator $\mathsf{SimCorr}$, a partial-decryption simulator $\mathsf{SimPartDec}$, and an equivocation algorithm $\mathsf{Equivocate}(\mathsf{aux}, \mathsf{st}, P, (H_1, \ldots, H_N)) \to (\mathsf{ct}, \mathsf{coin}, \mathsf{st}')$ that converts $\widetilde{\mathsf{ct}}$ into a ciphertext consistent with the policy $P$ and the final sequence of honest or adversarial handles. For the static specialization used throughout this work there are no honest-key corruptions, so $\mathsf{SimCorr}$ drops out of the interface. We retain

$\mathsf{SimPartDec}$, since a static attacker may ask for partial decryptions of ciphertexts of its choice both before and after the challenge, and may only be denied an honest share of the challenge ciphertext itself.

The three simulators are what separate EqDPE from the equivocal frameworks for distributed and flexible broadcast encryption, and the reason is functional rather than cosmetic. A broadcast-encryption key has one relevant behavior against a challenge ciphertext, namely, it decrypts or it does not. A DPE key instead emits a structured partial decryption that may later be combined with other shares, so even a key that is individually unauthorized has an observable interface that fake mode has to reproduce. A single malicious public key may label several inputs of the eventual policy, and the policy is unavailable while the challenger is still answering key-generation and partial-decryption queries. Generalizing the semantics of equivocal encryption to carry these consistency requirements is a contribution in its own right, and the sharpest part of it concerns the auxiliary information, which we turn to next.

**Two security properties.** The first, *mode indistinguishability*, compares an execution of the real algorithms against an execution of the fake ones. The attacker sees the CRS, adaptively asks for honest public keys and partial decryptions, and then submits $P$ together with the placement of honest and malicious keys. Real mode preprocesses those keys and encrypts whereas fake mode calls $\mathsf{Equivocate}$, and only after the policy and handles arrive. Both modes hand back the ciphertext along with the public pre-processing coins $\mathsf{coin}$, after which the attacker resumes its permitted queries. The two interactive views must be indistinguishable.

The second property is *selective security for fake setup*, which says that an attacker cannot distinguish between the output of $\mathsf{FakeSetup}(1^\lambda, 1^\kappa, \mu)$ for both $\mu = 0/1$. That is, even given the trapdoor $\mathsf{aux}$, distribution of $(\widetilde{\mathsf{crs}}, \widetilde{\mathsf{ct}}, \mathsf{aux})$ should look computationally indistinguishable. This is the same property that [GY26b, GY26a] uses in the context of distributed/flexible broadcast encryption.

The formulation isolates three distinct sources of delayed information, namely the policy, the honest/malicious assignment to its inputs, and the partial-decryption transcript. A future EqDPE construction may handle each by a different mechanism while reusing the EqDPE-to-DPE compiler unchanged.

**Static DPE from EqDPE.** The compiler uses the real EqDPE algorithms verbatim. The fake algorithms appear only in the proof. Fix a challenge message $\mu$. Mode indistinguishability replaces the real setup, key generation, partial-decryption answers, and challenge encryption by their fake-mode counterparts. Selective security for fake setup then flips $\mu$. A second application of mode indistinguishability returns to the real experiment. Because the compiled scheme runs the real algorithms, it inherits the CRS, key, ciphertext, and share sizes of the EqDPE scheme, so compactness survives the compilation intact. The argument follows the strategy of [GY26b, GY26a], with the interface widened to account for maliciously generated public keys and simulated partial decryptions.

Lifting static DPE to full adaptive security for policies beyond 1-out-of-$N$ remains open, and the obstruction is worth stating. The Gentry-Waters compiler [GW09, KMW23, FWW23, HWW25] exploits a dichotomy special to broadcast encryption where relative to a recipient set, a secret key either recovers the entire message or is useless. Their two-key transformation can therefore give every user one key of each type and decide later which type shows up in the challenge. A DPE key falls into neither category. Its partial decryption may carry prescribed information that reveals nothing on its own and yet becomes useful once combined with other shares, so replacing such a key by an independently sampled useless one destroys the joint distribution that reconstruction depends on. The two-key argument does not appear to extend to a general monotone policy in a black-box way.

## 2.2 EqDPE for DNF Policies

Before diving into the construction we provide an overview of the main tool both of our constructions use.

**Workhorse: Equivocal Matrix Commitments.** Both constructions rely on the equivocal matrix commitment (EqMCOM) of [GY26a], which randomizes the commitment procedure of Wee's matrix commitments [Wee25] and adds an equivocation algorithm $\mathsf{EqCom}$. Let $\mathbf{B}$ be the structured matrix supplied by the $2m^2$-decomposed LWE assumption [AMR25], Assumption 3.5. A commitment to $\mathbf{M} \in \mathbb{Z}_q^{n\times L}$ is a short matrix $\mathbf{C}$ together with public opening information from which one derives a low-norm $\mathbf{Z}$ with $\mathbf{C}\mathbf{V}_L = \mathbf{M} - \mathbf{B}\mathbf{Z}$,

where the verification matrix $\mathbf{V}_L$ and the opening matrix have bounded norms. This identity is what lets a ciphertext carry the single component $\mathbf{s}^\top\mathbf{C}$ in place of one LWE sample per column of $\mathbf{M}$.

Equivocation supplies the proof mechanism. Whenever the columns of $\mathbf{M}$ can be written as $\mathbf{m}_i = \mathbf{B}(\mathbf{y}_i + \mathbf{r}_i)$ for a hidden wide Gaussian pre-image $\mathbf{y}_i$ and a known low-norm shift $\mathbf{r}_i$, commitment indistinguishability property of EqMCOM lets one replace an honest commitment by $\mathbf{C} = \mathbf{B}\mathbf{R}_\mathbf{C}$ for a short $\mathbf{R}_\mathbf{C}$ fixed before $\mathbf{M}$ is known, and compute the public opening information afterwards as $\mathsf{EqCom}([\mathbf{y}_i + \mathbf{r}_i]_{i\in[L]}, \mathbf{R}_\mathbf{C})$. The image $\mathbf{B}\mathbf{y}_i$ is statistically close to uniform while its pre-image stays hidden, so the reduction's only job is to arrange each challenge-dependent column in the form "hidden random image plus known short shift". The two constructions differ precisely in how they establish that invariant after the attacker has committed to a policy.

**Opening the clause structure.** Our DNF construction builds on the decomposed-LWE schemes of [AGY26, CW26a], which can be described through a secret-sharing layer. For equivocation it pays to open that layer and work directly with clause sums. Each user samples $\mathsf{sk}_i = \mathbf{R}_i \leftarrow \{\pm 1\}^{4m^5\times m}$ and publishes $\mathsf{pk}_i = \mathbf{T}_i = \mathbf{B}\mathbf{R}_i$. To encrypt under a DNF $\phi = \mathsf{conj}_1 \vee \cdots \vee \mathsf{conj}_K$, unlike [AGY26, CW26a], we do not use secret sharing. Instead, our observation is that as each clause in a DNF is a conjunction, it would require secret keys from all users to satisfy this clause. So, we could simply add all the public keys to generate a public key for this clause of sorts. Concretely, for the DNF formula $\phi$ with wire labeling $\rho$, the encryptor draws a seed $\mathsf{sd}$, derives a Gaussian mask $\mathbf{u}_w = \mathcal{H}(\mathsf{sd}, w)$ for each wire, sets the wire public key $\mathbf{t}_w = \mathbf{T}_{\rho(w)}\mathbf{u}_w$, and commits to the $K$-column matrix

$$\mathbf{M} = \left[\textstyle\sum_{w\in\mathsf{conj}_1} \mathbf{t}_w + \mathbf{p} \;\middle|\; \cdots \;\middle|\; \sum_{w\in\mathsf{conj}_K} \mathbf{t}_w + \mathbf{p}\right],$$

where $\mathbf{p}$ is the global mask under which the message is encrypted. The rest of the ciphertext is $\mathbf{c}_1^\top = \mathbf{s}^\top\mathbf{B}+\mathsf{err}$, $\mathbf{c}_2^\top = \mathbf{s}^\top\mathbf{C} + \mathsf{err}$, and $c_3 = \mathbf{s}^\top\mathbf{p} + \mathsf{err} + \lfloor q/2\rfloor\mu$. A partial decryption under $\mathsf{sk}_i = \mathbf{R}_i$ for this ciphertext is a smudged value of $\mathbf{c}_1^\top\mathbf{R}_i$. Given a set of partial decryptions for wires that satisfy clause $\mathsf{conj}_j$, as $\mathbf{c}_1^\top\mathbf{R}_{j,i} \approx \mathbf{s}^\top\mathbf{T}_{j,i}$, we can then add $\mathbf{s}^\top\mathbf{T}_{j,i}\mathbf{u}_{j,i} = \mathbf{s}^\top\sum_{w\in\mathsf{conj}_j}\mathbf{t}_w$. Then, using $\mathbf{V}_K$ and $\mathbf{Z}$ of EqMCOM, we can expand $\mathbf{c}_2^\top$ to get $\mathbf{c}_2^\top\mathbf{V}_K + \mathbf{c}_1^\top\mathbf{Z} \approx \mathbf{s}^\top\mathbf{M}$. From there, using the $j$-th column and subtracting the sum of partial decryptions we can recover a noisy copy of $\mathbf{s}^\top\mathbf{p}$, and removing it from $c_3$ reveals the message $\mu$.

Writing the commitment this way exposes exactly which value has to look random in the proof: for every clause, the sum $\sum_{w\in\mathsf{conj}_j}\mathbf{t}_w + \mathbf{p}$ must admit an equivocal decomposition of the format required for EqMCOM. Static admissibility supplies the combinatorial invariant that makes this possible. Because the malicious wires do not satisfy $\phi$, every clause retains at least one honest wire. No clause, corrupt assignment, or honest wire needs to be fixed at setup.

**Fake Mode and programming one honest wire per clause.** Fake key generation algorithm $\mathsf{SimKGen}$ samples every honest $\mathbf{T}_i$ with a gadget trapdoor. Once the attacker submits $(\phi, \rho)$, the $\mathsf{Equivocate}$ algorithm picks the one honest wire $w_j^*$ per clause, samples a target $\mathbf{d}_j^* = \mathbf{B}\mathbf{y}_j$, and programs the oracle at that wire to a short preimage of the residual target, $\mathcal{H}(\mathsf{sd}, w_j^*) \leftarrow \mathsf{SamplePre}\left(\mathbf{T}_{\rho(w_j^*)}, T_{\mathbf{T}_{\rho(w_j^*)}}, \mathbf{d}_j^* - \mathbf{p}, \sigma\right)$. The trapdoor makes the programmed value distributed as a fresh Gaussian mask, and the clause column collapses to $\mathbf{m}_j = \mathbf{T}_{\rho(w_j^*)}\mathcal{H}(\mathsf{sd}, w_j^*) + \sum_{w\in\mathsf{conj}_j\setminus\{w_j^*\}}\mathbf{t}_w + \mathbf{p} = \mathbf{B}\mathbf{y}_j + \sum_{w\in\mathsf{conj}_j\setminus\{w_j^*\}}\mathbf{t}_w$. For the wires other than the honest wire, attacker provides the secret key $\mathsf{sk}_i = \mathbf{R}_i$ for which we can choose the hash value $\mathbf{u}_w$. Then, this is the exact form EqMCOM requires as $\sum_{w\in\mathsf{conj}_j\setminus\{w_j^*\}}\mathbf{t}_w$ has a short pre-image with respect to $\mathbf{B}$. $\mathsf{Equivocate}$ thus computes $\mathbf{C} = \mathbf{B}\mathbf{R}_\mathbf{C}$ and generates the public opening information only after learning the DNF. By using security of decomposed LWE assumption, we can then set $\mathbf{c}_1^\top$ uniform. Another application of leftover hash lemma hides the message in $c_3$. Since the reduction needs only the existence of an honest wire in each eventual clause and not its identity at setup, the DNF and the honest/malicious placement may be chosen after viewing key-generation and partial-decryption queries.

Simulation of partial-decryption queries via $\mathsf{SimPartDec}$ needs a separate mechanism. This is because as highlighted in the above procedure, fake key generation $\mathsf{SimKGen}$ does not have any secret key. We can handle this similarly to [CW26a]. The ciphertext now carries a simulation-sound extractable NIZK certifying

that $\mathbf{c}_1^\top$ has a short-error LWE representation. From any fresh accepting proof we can use the NIZK extractor to extract $(\mathbf{s}, \mathbf{e})$ and answers the partial decryption query using public key $\mathbf{T}_i$ as $\mathbf{s}^\top \mathbf{T}_i + \widetilde{\mathsf{err}}$ where $\widetilde{\mathsf{err}}$ is smudging noise, which makes it statistically close to $\mathbf{c}_1^\top \mathbf{R}_i$ under a real key.

**Compactness.** Note that the keys are succinct as the per-wire expansion is deferred to encryptor. A secret-sharing based scheme like [AGY26, CW26a] would reserve a separate public-key component for every location at which a user might appear, and thus the keys would grow with an a-priori policy bound. However, in our scheme a user publishes only $\mathbf{T}_i$, and encryption expands it into as many wire values as the chosen DNF needs by multiplying with the fresh masks $\mathcal{H}(\mathsf{sd}, w)$. With EqMCOM compressing the clause columns, the public key, secret key, and partial decryption are of size $\mathsf{poly}(\lambda)$ and the ciphertext is of size $\mathsf{poly}(\lambda, \log N)$, independent of the number and widths of the clauses. Compared to [AGY26, CW26a], our public keys do not grow with any bound on the number of times a user's public key can be used in the DNF formula and our public keys are of size $\mathsf{poly}(\lambda)$ without any dependence of the DNF structure.

## 2.3 EqDPE for Threshold Policies

**Scaled sharing, and why EqMCOM does not simply drop in.** For a $t$-out-of-$N$ threshold policy, Afshar, Goyal, and Yadugiri (AGY) [AGY26] share the mask $\mathbf{p}$ itself into $N$ vectors $(\widehat{\mathbf{p}}_1, \ldots, \widehat{\mathbf{p}}_N)$ using a programmable scaled linear secret-sharing scheme [AGY26]. In this scheme, every authorized set $S$ admits short integer coefficients and a bounded nonzero scale such that $\sum_{i \in S} \delta_{S,i} \widehat{\mathbf{p}}_i = \gamma_S \mathbf{p}$ and $\delta_{S,i}, \gamma_S$ values are bounded. Allowing the reconstructed value to be scaled is what keeps the coefficients far smaller than direct Lagrange interpolation would, and hence the lattice noise below the modulus. The sharing also has two identically distributed programmable modes in which the first one fixes enough uniformly random coordinates and derives the secret, whereas the other fixes the shares of a maximally unauthorized set $C$ of size $t-1$ and completes them to a sharing of a chosen secret. AGY compresses the resulting columns with a matrix commitment, obtaining ciphertexts of size $\tau^6 \cdot \mathsf{poly}(\lambda, \log N)$ for $\tau = \min(t^2, N-t)$, improving to $\tau^2 \cdot \mathsf{poly}(\lambda, \log N)$ with a CRS and succinct LWE.

Swapping their matrix commitment for EqMCOM does not make the scheme equivocal. Their proof is selective, so the reduction knows which shares belong to corrupt users and programs those coordinates in advance. A static attacker that supplies the public keys and their positions only at challenge time, which leaves the reduction unable to decide which honest key should carry which programmable share. The positional uncertainty is the real obstacle, because a matrix commitment is ordered. Abstractly committing to $[\mathbf{a} \mid \mathbf{b}]$ is not the same operation as committing to $[\mathbf{b} \mid \mathbf{a}]$, so one cannot permute shares after the fact and expect the opening to survive.

**A large-key stepping stone.** We first relax key compactness and give each user one column per possible slot. That is, we sample secret keys $\mathbf{R}_i \leftarrow \{\pm 1\}^{4m^5 \times N}$ and publishing $\mathbf{T}_i = \mathbf{B}\mathbf{R}_i \in \mathbb{Z}_q^{n \times N}$. That is, both public and secret keys now grow with $N$, the maximum number of users encryption can support. A user occupying slot $j$ contributes the $j$-th column. Fake setup guesses the challenge threshold $t$, samples a uniformly random unauthorized set $C \subseteq [N]$ of size $t-1$, and invokes the second sharing mode so that every share indexed by $C$ has a known short preimage under $\mathbf{B}$. An honest key is then simulated as $\mathbf{T}_i = \widetilde{\mathbf{T}}_i - [\widehat{\mathbf{p}}_1 \mid \cdots \mid \widehat{\mathbf{p}}_N]$ for a random-looking $\widetilde{\mathbf{T}}_i$, so whichever slot $j$ the user ends up in, the column entering the commitment satisfies $\mathbf{t}_{i,j} + \widehat{\mathbf{p}}_j = \widetilde{\mathbf{t}}_{i,j}$. Expanding the key is what supplies enough independently programmable columns to cover every position the attacker might choose.

However, the corrupt positions are still unknown, and the programmed sharing can absorb malicious keys only at the coordinates in $C$. To resolve this we use the symmetry of threshold secret sharing. Note that in order for right reconstruction in $t$-out-of-$N$ threshold schemes, we only need a set of at least $t$ users and it doesn't matter which $t$ users are present. So, encryption may apply a uniformly random permutation $\Pi : [N] \to [N]$ to each of the coordinates, using column $\Pi(i)$ of the $i$-th key together with share $\widehat{\mathbf{p}}_{\Pi(i)}$. Specifically, instead of committing the matrix $\mathbf{M} = [\mathbf{t}_i + \widehat{\mathbf{p}}_i]_{i \in [N]}$, we commit the matrix $\mathbf{M} = [\mathbf{t}_{i,\Pi(i)} + \widehat{\mathbf{p}}_{\Pi(i)}]$. This way, the $i$-th user reveals the share $\widehat{\mathbf{p}}_{\Pi(i)}$. If we also provide $\Pi$ along with the ciphertext, we can correctly reconstruct the message $\mu$. However, as $\Pi$ is a random permutation, we employ

a random oracle to sample this permutation succinctly. In a bit more detail, we show that the Fisher-Yates shuffling algorithm [Dur64, Knu07] can be used in conjunction with a random oracle to sample this random permutation $\Pi$ (cf. Appendix A).

In the security experiment, admissibility guarantees attacker provides fewer than $t$ malicious keys, so after seeing their locations the simulator picks a permutation that maps the malicious slots into $C$ and then extends it uniformly to a permutation of $[N]$. Since $C$ was itself uniform and hidden, the result is distributed as an honest sample. Every commitment column now has the EqMCOM form. The honest column cancels its embedded share and leaves the hidden image $\widetilde{\mathbf{t}}_{i,\Pi(i)}$, while a malicious one sits at a coordinate of $C$ for which the simulator knows short preimages of both the adversarial column and the programmed share $\widehat{\mathbf{p}}_{\Pi(i)}$. This way, we can use EqMCOM to realize static security in threshold policies as well.

**Compressing the keys.** The slot-expanded scheme achieves static security but leaves each user with $N$ size public- and secret-keys and an equally wide partial decryption. We remove the dependence by deferring the expansion to pre-processing, exactly as in the DNF construction. A user publishes $\mathsf{pk}_i = (\mathbf{T}_i := \mathbf{B}\mathbf{R}_i, \mathsf{sd}_i)$ for a short secret key $\mathbf{R}_i \in \{\pm 1\}^{4m^5 \times m}$ and a seed $\mathsf{sd}_i \in \{0,1\}^\lambda$. The encryptor derives the effective wide key $\mathbf{T}_i \mathcal{H}_{\mathsf{pk}}(\mathsf{sd}_i) \in \mathbb{Z}_q^{n \times N}$ from a random oracle $\mathcal{H}_{\mathsf{pk}}$ that supplies the large Gaussian mask. Real partial decryptions stay short, and reconstruction applies the relevant column of $\mathcal{H}_{\mathsf{pk}}(\mathsf{sd}_i)$ publicly. Static security is what makes the deferral simulatable. An honest challenge key is never corrupted, so the simulator need not open $\mathbf{T}_i$ to a short $\mathbf{R}_i$ after publishing it, and can instead sample $\mathbf{T}_i$ with a gadget trapdoor and program $\mathcal{H}_{\mathsf{pk}}(\mathsf{sd}_i)$ to a Gaussian pre-image $\mathbf{U}_i$ such that $\mathbf{T}_i \mathbf{U}_i = \widetilde{\mathbf{T}}_i - [\widehat{\mathbf{p}}_1 \mid \cdots \mid \widehat{\mathbf{p}}_N]$, reproducing the distribution of the expanded key above. Deriving these columns only at encryption time also removes the a-priori bound $N$ from the global setup (Section 6.6). Public keys, partial decryptions are then of size $\tau^2 \cdot \mathsf{poly}(\lambda)$, and ciphertext is of the size $\tau^6 \cdot \mathsf{poly}(\lambda)$ under decomposed LWE in the random oracle model. Ciphertext size can be shortened to $\tau^2 \cdot \mathsf{poly}(\lambda)$ with a CRS under succinct LWE [Wee24]. Finally, we remark if we can construct a programmable $(\alpha, \beta)$-scaled LSSS as defined in [AGY26] with $|\alpha| = |\beta| = \mathsf{poly}(\lambda, \log N)$, we can construct an optimal silent threshold encryption from lattices.

**Roadmap.** We provide the necessary preliminaries including definition of EqMCOM in Section 3. We define our equivocal DPE framework in Section 4 and show the DPE-from-EqDPE compiler in Section 4.2. Next, we show the construction of statically secure EqDPE for DNFs in Section 5 and statically secure EqDPE for threshold policies in Section 6 respectively.

# 3 Preliminaries

**Notation.** By $\mathsf{PPT}$ we denote probabilistic polynomial-time. We denote the security parameter by $\lambda$ and the set of positive integers by $\mathbb{N}$. For any $a, b \in \{0\} \cup \mathbb{N}$, $a \leq b$, we denote by $[a, b]$, the set of all integers from $a$ to $b$ including $a$ and $b$. In other words, $[a, b] = \{a, \ldots, b\}$. We denote by $[n] := [1, n]$. We denote by $x \leftarrow \mathcal{X}$, the process of sampling an element $x$ from the set $\mathcal{X}$, with uniform probability. Similarly, for any $\mathsf{PPT}$ algorithm $\mathcal{A}$, $x \leftarrow \mathcal{A}(y)$ denotes the process of sampling $x$ from the output distribution of $\mathcal{A}$ when run on $y$ with randomly chosen coins. By $\mathsf{negl}(\cdot)$, we denote negligible functions. By $\mathsf{poly}(\cdot)$, we denote positive polynomials. All of our logarithms log are in base 2. We denote the power set $A$ by $2^A$. Consider two efficiently samplable probability distributions $\mathcal{X} = \{\mathcal{X}_\lambda\}_{\lambda \in \mathbb{N}}$ and $\mathcal{Y} = \{\mathcal{Y}_\lambda\}_{\lambda \in \mathbb{N}}$ over support $\Omega$. We say $\mathcal{X}$ and $\mathcal{Y}$ are computationally indistinguishable if for any non-uniform $\mathsf{PPT}$ distinguisher $\mathcal{D} = \{\mathcal{D}_\lambda\}_{\lambda \in \mathbb{N}}$, and for large enough $\lambda \in \mathbb{N}$, $\left|\Pr\left[b \leftarrow \mathcal{D}(1^\lambda, \alpha_b) \ : \ \alpha_0 \leftarrow \mathcal{X}_\lambda, \alpha_1 \leftarrow \mathcal{Y}_\lambda, b \leftarrow \{0,1\}\right] - 1/2\right| \leq \mathsf{negl}(\lambda)$. We write $\mathcal{X} \approx_c \mathcal{Y}$ to denote that distributions $\mathcal{X}$ and $\mathcal{Y}$ computationally indistinguishable. Similarly, by $\approx_s$, we denote statistical indistinguishability. Concretely, $\mathcal{X} \approx_s \mathcal{Y}$ if for large enough $\lambda \in \mathbb{N}$, $\mathsf{SD}(\mathcal{X}_\lambda, \mathcal{Y}_\lambda) = 1/2 \sum_{\alpha \in \Omega} |\Pr[\alpha \leftarrow \mathcal{X}_\lambda] - \Pr[\alpha \leftarrow \mathcal{Y}_\lambda]| \leq \mathsf{negl}(\lambda)$.

## 3.1 Lattice Preliminaries

For $q \in \mathbb{N}$, we denote the finite field with $q$ elements as $\mathbb{Z}_q$. In this work, if $q$ is prime, we view the field as integers from $[-\lfloor q/2 \rfloor, \lceil q/2 \rceil) \subset \mathbb{Z}$. Following popular convention, we denote column vectors by lowercase

bold letters, e.g., $\mathbf{x} \in \mathbb{Z}_q^n$ and denote matrices by uppercase bold letters, e.g., $\mathbf{A} \in \mathbb{Z}_q^{n \times m}$. We use $x_i$ to denote the $i$-th element in $\mathbf{x}$ for $i \in [n]$ and $a_{i,j}$ to denote the element in the $i$-th row and $j$-th column of matrix $\mathbf{A}$ for $i \in [n], j \in [m]$. We sometimes alternately write $\mathbf{x}[i] := x_i$ and $\mathbf{A}[i,j] := a_{i,j}$. We define $\mathbf{1}_n$ to be a vector of all ones, $\mathbf{0}_n$ to be a vector of all zeros, and $\mathbb{1}_{n,j}$ to denote the $n$-dimensional canonical basis vector that has 1 in the $j$-th position, i.e., $\mathbf{1}_n = [1\ 1\ \ldots\ 1]^\top$ and $\mathbb{1}_{n,j} = [\ \ldots\ 1\ \ldots]^\top$. We frequently drop $n$ from subscript in these vectors when dimension is clear. We denote the $n \times n$ identity matrix by $\mathbf{I}_n$ and drop the subscript when dimension is clear. For a matrix $\mathbf{A} \in \mathbb{Z}_q^{n \times m}$, $\mathsf{vec}(\mathbf{A}) \in \mathbb{Z}_q^{nm}$ denoted the vector obtained by concatenating the columns of $\mathbf{A}$. For any vector $\mathbf{a} \in \mathbb{Z}_q^n$ or matrix $\mathbf{A} \in \mathbb{Z}_q^{n \times m}$, we use $\|\mathbf{a}\|$ and $\|\mathbf{A}\|$ to denote the maximum of all the entries in $\mathbf{a}$ and $\mathbf{A}$ respectively. We use $\mathbf{A} \otimes \mathbf{B}$ to denote the Kronecker product between matrices $\mathbf{A}, \mathbf{B}$. The rounding function $\lceil \cdot \rfloor_p : \mathbb{Z}_q \to \mathbb{Z}_p$ is given by $\lceil x \rfloor_p = \lceil \frac{p}{q} x \rfloor \in \mathbb{Z}_p$. That is, if $x = y \cdot q/p + z$ where $z \in [-q/2p, q/2p)$, we have $\lceil x \rfloor_p = y$. The general rounding function $\lceil \cdot \rfloor$ is performed over $\mathbb{Z}_q$.

**Discrete Gaussian.** We write $\mathcal{D}_{\mathbb{Z},\sigma}$ to denote the discrete gaussian distribution with width parameter $\sigma > 0$. We state the following important lemma about the maximum norm of a value sampled from discrete gaussian distribution.

**Lemma 3.1** ([Lyu12, MP12])**.** For any $k > 0$, $\Pr[|x| > k\sigma : x \leftarrow \mathcal{D}_{\mathbb{Z},\sigma}] \leq 2\exp\left(\frac{-k^2}{2}\right)$.

**Truncated Discrete Gaussians.** The truncated discrete Gaussian distribution over $\mathbb{Z}$ with parameter $\sigma$, denoted by $\widetilde{\mathcal{D}}_{m,\mathbb{Z},\sigma}$, is same as the discrete Gaussian distribution $\mathcal{D}_\sigma$ except it outputs 0 whenever the $\ell_\infty$ norm exceeds $\sqrt{m} \cdot \sigma$. Note that, by definition, $\widetilde{\mathcal{D}}_{m,\mathbb{Z},\sigma}$ is $\sqrt{m} \cdot \sigma$-bounded. Also, note that $\widetilde{\mathcal{D}}_{m,\mathbb{Z},\sigma} \approx_s \mathcal{D}_{m,\mathbb{Z},\sigma}$.

**Gadget Matrix.** For positive integers $n, m, q$, and $m = \Omega(n \log q)$, we write $\mathbf{G} \in \mathbb{Z}_q^{n \times m}$ to denote the gadget matrix [MP12]. It is defined as $\mathbf{G} = (\mathbf{I}_n \otimes \mathbf{g}^\top)$, $\mathbf{g} = [1\ 2\ 2^2\ \ldots\ 2^{\lfloor \log q \rfloor}\ 0\ \ldots\ 0]^\top \in \mathbb{Z}_q^m$.

We write $\mathbf{G}^{-1}$ to denote the standard entry-wise bit decomposition. Note that $\mathbf{G}^{-1} : \mathbb{Z}_q^n \to \mathbb{Z}_q^m$ is an operation such that $\mathbf{G}\mathbf{G}^{-1}(\mathbf{a}) = \mathbf{a}$ and not inverse of $\mathbf{G}$. Output of $\mathbf{G}^{-1}$ is padded with 0s to match the dimensions. We often overload the notation and write $\mathbf{G}^{-1}(\mathbf{A})$ where $\mathbf{G}^{-1}$ is performed for each column.

**Leftover Hash Lemma.** We recall the (generalized) leftover hash lemma adapted from [ILL89, DRS04, GPV08, ABB10].

**Lemma 3.2** (Generalized Leftover Hash Lemma)**.** Let $n, m, q \in \mathbb{N}$ and $q$ is an odd prime.

1. $\{(\mathbf{B}, \mathbf{BR}, \mathbf{e}^\top\mathbf{R})\} \approx_s \{(\mathbf{B}, \mathbf{A}, \mathbf{e}^\top\mathbf{R})\}$ where $m = \Omega(n \log q)$, $\mathbf{B} \leftarrow \mathbb{Z}_q^{n \times m}, \mathbf{R} \leftarrow \{\pm 1\}^{m \times \ell}$(or $\mathbf{R} \leftarrow \{0,1\}^{m \times \ell}$), $\mathbf{A} \leftarrow \mathbb{Z}_q^{n \times \ell}$, $\ell = \mathsf{poly}(n)$ and $\mathbf{e}^\top \in \mathbb{Z}_q^m$ is any fixed vector.
2. $\{(\mathbf{B}, \mathbf{BR})\} \approx_s \{(\mathbf{B}, \mathbf{A})\}$ where $m = \Omega(n \log q)$, $\mathbf{B} \leftarrow \mathbb{Z}_q^{n \times m}, \mathbf{R} \leftarrow \mathcal{D}_{\mathbb{Z},\sigma}^{m \times \ell}, \mathbf{A} \leftarrow \mathbb{Z}_q^{n \times \ell}$, $\ell = \mathsf{poly}(n)$ and $\sigma = \omega(\sqrt{\log m})$.

**Gadget Trapdoors.** In this work, we also use gadget trapdoor framework which provides a way to sample a random-looking matrix $\mathbf{A}$ along with a trapdoor $T_\mathbf{A}$ such that $\|T_\mathbf{A}\| = 1$ and $\mathbf{A}T_\mathbf{A} = \mathbf{G}$. This framework also provides a useful property to sample short pre-images $\mathbf{x}$ for any $\mathbf{y}$ efficiently such that $\mathbf{Ax} = \mathbf{y}$. For completeness, we provide full definition in Appendix B.

**Lemma 3.3** (Smudging Lemma)**.** Let $B_1 = B_1(\lambda)$ and $B_2 = B_2(\lambda)$ be positive integers and let $e_1 \in [-B_1, B_1]$ be a fixed integer. Let $e_2 \leftarrow [-B_2, B_2]$ be chosen uniformly at random. Then the distribution of $e_2$ is statistically indistinguishable from that of $e_2 + e_1$ as long as $B_1/B_2 = \mathsf{negl}(\lambda)$.

## 3.2 Assumptions: LWE and $\ell$-Decomposed LWE

The Learning With Errors (LWE) assumption is defined as follows.

**Assumption 3.4** ([Reg05])**.** The LWE assumption with parameters $(n, m, q, \chi)$ states that

$$\left\{(\mathbf{A}, \mathbf{s}^\top \mathbf{A} + \mathbf{e}^\top)\right\} \approx_c \left\{(\mathbf{A}, \mathbf{c}^\top)\right\}$$

where $\mathbf{A} \leftarrow \mathbb{Z}_q^{n \times m}, \mathbf{s} \leftarrow \mathbb{Z}_q^n, \mathbf{e} \leftarrow \mathcal{D}_{\mathbb{Z},\chi}^m, \mathbf{c} \leftarrow \mathbb{Z}_q^m$.

We provide the $\ell$-decomposed LWE assumption which was originally proposed in [AMR25]. Our formulation is adapted from [Wee25, WW25].

**Assumption 3.5** ([AMR25], adapted from [Wee25, WW25])**.** Let $(n, m, q, \chi)$ be LWE parameters. Then, the $(\ell, \sigma)$-decomposed LWE assumption states that

$$\left\{(\mathbf{W}, \mathbf{D}, \mathbf{B}, \mathbf{s}^\top \mathbf{B} + \mathbf{e}^\top)\right\} \approx_c \left\{(\mathbf{W}, \mathbf{D}, \mathbf{B}, \mathbf{c}^\top)\right\}$$

where $\mathbf{W} \leftarrow \mathbb{Z}_q^{n \times \ell m}, \mathbf{D} \leftarrow \mathcal{D}_{\mathbb{Z},\sigma}^{m \times \ell m}, \mathbf{s} \leftarrow \mathbb{Z}_q^n, \mathbf{e} \leftarrow \mathcal{D}_{\mathbb{Z},\chi}^{\ell^2 m}, \mathbf{c} \leftarrow \mathbb{Z}_q^{\ell^2 m}$ and $\mathbf{B} = \mathbf{W}(\mathbf{I}_\ell \otimes \mathbf{D}) + \mathsf{vec}(\mathbf{I}_\ell)^\top \otimes \mathbf{G} \in \mathbb{Z}_q^{n \times \ell^2 m}$. When $\sigma = \mathsf{poly}(\lambda, \ell, m)$ we simply call this as $\ell$-decomposed LWE assumption.

**Leftover Hash Lemma for Structured B.** We remark that $\mathbf{B}$ defined in above assumption is a structured matrix uniquely defined by $\mathbf{W}, \mathbf{D}$. Thus, it might not be immediately clear that Lemma 3.2 works with $\mathbf{B}$. We provide the following lemmas which show that leftover hash lemma holds even with respect to $\mathbf{B}$.

**Lemma 3.6.** Lemma 3.2, Item 1 holds with respect to $\mathbf{B}$ even given $(\mathbf{W}, \mathbf{D})$.

*Proof.* Full proof is provided in Appendix B. □

## 3.3 Simulation Sound Extractable NIZK Arguments

**Syntax.** A simulation-sound extractable NIZK argument for the language $\mathcal{L} = \{x \ : \ \exists \ w, C(x, w) = 1\}$ consists of the following polynomial time algorithms.

$\mathsf{Setup}(1^\lambda) \to \mathsf{crs}$**.** The randomized *public-coin* setup algorithm takes as input a security parameter $\lambda$ and outputs a common random string $\mathsf{crs}$. The following algorithms take $\mathsf{crs}$ implicitly

$\mathsf{Prove}(x, w) \to \pi$**.** The randomized prover algorithm takes as input an instance $x$, a witness $w$, and outputs a proof $\pi$.

$\mathsf{Verify}(x, \pi) \to 0/1$**.** The deterministic verifier algorithm takes as input an instance $x$, a proof $\pi$, and outputs 0 (reject) or 1 (accept).

$\mathsf{SimSetup}(1^\lambda) \to (\mathsf{crs}, \mathsf{td})$**.** The randomized simulation setup algorithm takes as input security parameter $\lambda$ and outputs common reference string $\mathsf{crs}$ and trapdoor $\mathsf{td}$.

$\mathsf{SimProve}(\mathsf{td}, x) \to \pi$**.** The randomized simulation proving algorithm takes as input a trapdoor $\mathsf{td}$, instance $x$ and outputs proof $\pi$.

$\mathcal{E}(\mathsf{td}, x, \pi) \to w$**.** The possibly randomized extraction algorithm takes as input trapdoor $\mathsf{td}$, instance $x$, proof $\pi$, and outputs witness $w$.

**Definition 3.7** (NIZK)**.** A $\mathsf{NIZK}$ scheme $(\mathsf{Setup}, \mathsf{Prove}, \mathsf{Verify}, \mathsf{SimSetup}, \mathsf{SimProve}, \mathcal{E})$ for $\mathcal{L}$ is said to be a simulation-sound NIZK argument scheme if it satisfies the following properties.

**Completeness.** For any $\lambda \in \mathbb{N}$, $x \in \mathcal{L}, C(x, w) = 1$, $\mathsf{Verify}(x, \pi) = 1$ where $\mathsf{crs} \leftarrow \mathsf{Setup}(1^\lambda), \pi \leftarrow \mathsf{Prove}(x, w)$.

**Compactness.** There exists a polynomial $\mathsf{poly}(\lambda)$ such that size of the proof $\pi$ is $|\pi| = |w| + \mathsf{poly}(\lambda)$.

**Zero-Knowledge.** For any stateful admissible PPT adversary $\mathcal{A}$, there exists a negligible function $\mathsf{negl}(\cdot)$ such that for all $\lambda \in \mathbb{N}$:

$$|\Pr[\mathcal{A}^{\mathsf{Prove}(\mathsf{crs},\cdot,\cdot)}(\mathsf{crs}) = 1 \ : \ \mathsf{crs} \leftarrow \mathsf{Setup}(1^\lambda)] - |\Pr[\mathcal{A}^{\mathcal{O}(\mathsf{td},\cdot,\cdot)}(\mathsf{crs}) = 1 \ : \ (\mathsf{crs},\mathsf{td}) \leftarrow \mathsf{SimSetup}(1^\lambda)]| \leq \mathsf{negl}(\lambda)$$

where $\mathcal{O}(x,w)$ outputs $\mathsf{SimProve}(\mathsf{td},x)$ if $x \in \mathcal{L}$ ($C(x,w)=1$) and $\perp$ otherwise.

**Simulation Sound Straight-line Extraction.** For any stateful admissible PPT adversary $\mathcal{A}$, there exists a negligible function $\mathsf{negl}(\cdot)$ such that for all $\lambda \in \mathbb{N}$:

$$\Pr\left[\begin{array}{ll} \mathsf{Verify}(x^*,\pi^*)=1 \wedge & (\mathsf{crs},\mathsf{td}) \leftarrow \mathsf{SimProve}(1^\lambda), \\ C(x,w)=0 & : \ (x^*,\pi^*) \leftarrow \mathcal{A}^{\mathsf{SimProve}(\mathsf{td},\cdot)}(\mathsf{crs}), \\ & w \leftarrow \mathcal{E}(\mathsf{td},x^*,\pi^*) \end{array}\right] \leq \mathsf{negl}(\lambda).$$

A stateful PPT machine $\mathcal{A}$ is said to be admissible if $x^*$ is not one of the queries made to $\mathcal{S}$.

**Theorem 3.8** ([PS19, Wat24, BCD+25, WWW25, Sah99, DDO+01])**.** Assuming the hardness of LWE (Assumption 3.4), there exists a simulation-sound NIZK scheme for any NP language.

## 3.4 Equivocal Matrix Commitments

We provide the definition of equivocal matrix commitments as defined in [GY26a].

**Syntax.** An Equivocal Matrix Commitment schemes (EqMCOM) consists of the following polynomial time algorithms.

$\mathsf{Setup}(1^n, 1^m, q, \sigma, B) \to (\mathsf{pp}, \mathbf{W}, \mathbf{D}, \mathbf{B})$**.** The randomized *public-coin* setup algorithm takes as input LWE parameters $(n, m, q, \sigma)$ where $\sigma = \mathsf{poly}(\lambda, m)$, bound $B \in \mathbb{N}$, outputs public parameters $\mathsf{pp}$, $\mathbf{W} \leftarrow \mathbb{Z}_q^{n \times m_1}, \mathbf{D} \leftarrow \mathcal{D}_{\mathbb{Z},\sigma}^{m \times m_1}$, and $\mathbf{B} = \mathbf{W}(\mathbf{I}_{m_2} \otimes \mathbf{D}) + \mathsf{vec}(\mathbf{I}_{m_2})^\top \otimes \mathbf{G}$. The following algorithms take $(\mathsf{pp}, \mathbf{W}, \mathbf{D}, \mathbf{B})$ implicitly.

$\mathsf{Com}(\mathbf{M}) \to (\mathbf{C}, \mathsf{aux})$**.** The randomized *public-coin* commitment algorithm takes as input matrix $\mathbf{M} \in \mathbb{Z}_q^{n \times N}$, outputs commitment matrix $\mathbf{C} \in \mathbb{Z}_q^{n \times m}$ and auxiliary information $\mathsf{aux}$ (this contains the random coins sampled by this algorithm).

$\mathsf{Verify}(1^N) \to \mathbf{V}_N$**.** The deterministic verification algorithm takes as input column length $N$ and outputs verification matrix $\mathbf{V}_N \in \mathbb{Z}_q^{m \times N}$.

$\mathsf{Open}(\mathbf{M}, \mathsf{aux}) \to \mathbf{Z}$**.** The deterministic opening algorithm takes as input matrix $\mathbf{M} \in \mathbb{Z}_q^{n \times N}$, auxiliary information $\mathsf{aux}$, and outputs opening matrix $\mathbf{Z} \in \mathbb{Z}_q^{m_1 m_2 \times N}$.

$\mathsf{EqCom}(\mathbf{Y}, \mathbf{R_C}) \to \mathsf{aux}$**.** The randomized equivocal commitment algorithm takes as input low-norm matrix $\mathbf{Y} \in \mathbb{Z}_q^{m_1 m_2 \times N}$, $\mathbf{R_C} \in \mathbb{Z}_q^{m_1 m_2 \times m}$ such that $\|\mathbf{R_C}\| = 1$, and outputs auxiliary information $\mathsf{aux}$.

**Definition 3.9** (EqMCOM)**.** An EqMCOM scheme $(\mathsf{Setup}, \mathsf{Com}, \mathsf{Verify}, \mathsf{Open}, \mathsf{EqCom})$ should satisfy the following properties.

**Correctness.** For any $n, q \in \mathbb{N}, m = \Omega(n \log q), \sigma = \mathsf{poly}(\lambda, m), B \in \mathbb{N}, N = \mathsf{poly}(\lambda)$, any matrix $\mathbf{M} \in \mathbb{Z}_q^{n \times N}$,

$$\Pr\left[\begin{array}{ll} \mathbf{C}\mathbf{V}_N = \mathbf{M} - \mathbf{B}\mathbf{Z} & : \ (\mathsf{pp}, \mathbf{W}, \mathbf{D}, \mathbf{B}) \leftarrow \mathsf{Setup}(1^n, 1^m, q, \sigma, B), \\ & (\mathbf{C}, \mathsf{aux}) \leftarrow \mathsf{Com}(\mathbf{M}), \mathbf{V}_N = \mathsf{Verify}(1^N), \\ & \mathbf{Z} = \mathsf{Open}(\mathbf{M}, \mathsf{aux}) \end{array}\right] = 1$$

**Compactness.** There exist polynomials $p_1(\cdot), p_2(\cdot), \mathsf{poly}_{\mathsf{aux}}(\cdot,\cdot)$ such that for any $\lambda \in \mathbb{N}, N \leq 2^\lambda, \mathbf{M} \in \mathbb{Z}_q^{n\times N}, n, q \in \mathbb{N}, m = \Omega(n \log q), \sigma = \mathsf{poly}(\lambda, m)$, and $(\mathsf{pp}, \mathbf{W}, \mathbf{D}, \mathbf{B}) \leftarrow \mathsf{Setup}(1^n, 1^m, q, \sigma, B)$, $m_1 = p_1(m), m_2 = p_2(m), |\mathsf{aux}| \leq \mathsf{poly}_{\mathsf{aux}}(\lambda, \log N)$ where $(\mathbf{C}, \mathsf{aux}) \leftarrow \mathsf{Com}(\mathbf{M})$.

Moreover, we also need that $\|\mathbf{V}_N\| \leq O(\|\mathbf{D}\| m^4 \log q)$ and $\|\mathbf{Z}\| \leq O(\sqrt{m}\widetilde{\sigma})$ where $\mathbf{V}_N = \mathsf{Verify}(1^N)$, $\mathbf{Z} = \mathsf{Open}(\mathbf{M}, \mathsf{aux})$, and $\widetilde{\sigma}$ is implicit in $\mathsf{pp}$.

**Commitment Indistinguishability.** For any stateful PPT adversary $\mathcal{A}$, there exists a negligible function $\mathsf{negl}(\cdot)$ such that for any $\lambda \in \mathbb{N}$,

$$\left| \Pr \left[ b \leftarrow \mathcal{A}^{\mathcal{O}(\cdot,\cdot)}(\mathbf{C}^{(b)}, \mathsf{aux}^{(b)}) \quad : \quad \begin{array}{l} (1^n, 1^m, q, \sigma, B) \leftarrow \mathcal{A}(1^\lambda), h := 1, \mathcal{Y} := \emptyset, \\ (\mathsf{pp}, \mathbf{W}, \mathbf{D}, \mathbf{B}) \leftarrow \mathsf{Setup}(1^n, 1^m, q, \sigma, B), \\ (S, \mathbf{R}) \leftarrow \mathcal{A}^{\mathcal{O}(\cdot,\cdot)}(\mathsf{pp}, \mathbf{W}, \mathbf{D}, \mathbf{B}), \\ b \leftarrow \{0,1\}, \\ \text{If } b = 0, (\mathbf{C}^{(0)}, \mathsf{aux}^{(0)}) \leftarrow \mathsf{Com}(\mathbf{M}) \\ \text{If } b = 1, \mathbf{R}_{\mathbf{C}} \leftarrow \{\pm 1\}^{4m^5 \times m}, \mathbf{C}^{(1)} = \\ \mathbf{B}\mathbf{R}_{\mathbf{C}}, \mathsf{aux}^{(1)} \leftarrow \mathsf{EqCom}(\mathbf{Y} + \mathbf{R}, \mathbf{R}_{\mathbf{C}}) \end{array} \right] - \frac{1}{2} \right| \leq \mathsf{negl}(\lambda)$$

where $\mathbf{Y} = [\mathcal{Y}[h_1] \mid \ldots \mid \mathcal{Y}[h_N]]$ and $\mathbf{M} = \mathbf{B}(\mathbf{Y} + \mathbf{R})$. Here, $(h_1, \ldots, h_N)$ is the lexicographical ordering of $S$. $\mathcal{O}$ is a stateful oracle that responds to the following queries.

- $\mathsf{Gen}$. $\mathcal{A}$ queries for generation. Sample $\mathbf{y} \leftarrow \mathcal{D}_{\mathbb{Z},\widetilde{\sigma}}^{m_1 m_2}$. Compute $\mathbf{t} = \mathbf{B}\mathbf{y}$. Send $(\mathbf{t}, h)$ to $\mathcal{A}$. Store $\mathcal{Y}[h] := \mathbf{y}$ and update $h := h + 1$.
- $(\mathsf{Corr}, h')$. $\mathcal{A}$ queries for corruption with $h' \in [h]$. If there is no entry for $h'$ in $\mathcal{Y}$, send $\perp$ to $\mathcal{A}$. Otherwise, send $\mathcal{Y}[h']$ to $\mathcal{A}$. Remove entry corresponding to $h'$ from $\mathcal{Y}$.

A stateful PPT machine $\mathcal{A}$ is said to be admissible if $S \subseteq [h]$ and for each $h^* \in S$, there is an entry in $\mathcal{Y}$ and $\|\mathbf{R}\| \leq B$. Moreover, $\mathcal{A}$ doesn't corrupt any $h^* \in S$ in post-challenge queries.

**Theorem 3.10** ([GY26a]). There exists an equivocal matrix commitment scheme in the random oracle model.

# 4 (Equivocal) Distributed Monotone Policy Encryption

**Syntax.** A Distributed Monotone Encryption scheme (DPE) for policy class $\mathcal{P} = \{\mathcal{P}_\kappa\}_{\kappa\in\mathbb{N}}$ consists of the following polynomial-time algorithms.

$\mathsf{GSetup}(1^\lambda, 1^\kappa) \to \mathsf{crs}$. The possibly randomized *public-coin* global setup algorithm takes as input security parameter $\lambda$, policy class index $\kappa$, and outputs common random string $\mathsf{crs}$. The following algorithms take $\mathsf{crs}$ implicitly.

$\mathsf{KGen} \to (\mathsf{pk}, \mathsf{sk})$. The randomized key generation algorithm outputs public-secret key pair $(\mathsf{pk}, \mathsf{sk})$.

$\mathsf{PreProcess}(P, (\mathsf{pk}_i)_{i\in[N]}) \to (\mathsf{mpk}, \mathsf{aux})$. The *public-coin* randomized pre-processing algorithm takes as input policy $P \in \mathcal{P}_\kappa$, public keys $(\mathsf{pk}_i)_{i\in[N]}$, and outputs master public key $\mathsf{mpk}$, random coins $\mathsf{coin}$.

$\mathsf{Enc}(\mathsf{mpk}, \mathsf{coin}, \mu) \to \mathsf{ct}$. The randomized encryption algorithm takes as input master public key $\mathsf{mpk}$, random coins $\mathsf{coin}$, message $\mu \in \{0,1\}$, and outputs a ciphertext $\mathsf{ct}$.

$\mathsf{PartDec}(\mathsf{ct}, \mathsf{coin}, \mathsf{sk}) \to \mathsf{dsh}$. The possibly randomized partial decryption algorithm takes as input ciphertext $\mathsf{ct}$, random coins $\mathsf{coin}$, a secret key $\mathsf{sk}$ and outputs a decryption share $\mathsf{dsh}$.

$\mathsf{Recon}(P, (\mathsf{pk}_i)_{i\in[N]}, (\mathsf{dsh}_j)_{j\in S'}, \mathsf{ct}, \mathsf{coin}) \to \mu'$ The deterministic reconstruction algorithm takes as input policy $P$, public keys $(\mathsf{pk}_i)_{i\in[N]}$, decryption shares $(\mathsf{dsh}_j)_{j\in S'}$, ciphertext $\mathsf{ct}$, random coins $\mathsf{coin}$, and outputs $\mu' \in \{0,1\} \cup \{\perp\}$.

**Definition 4.1** (DPE)**.** A DPE scheme $(\mathsf{GSetup}, \mathsf{KGen}, \mathsf{PreProcess}, \mathsf{Enc}, \mathsf{PartDec}, \mathsf{Recon})$ is said to be an DPE scheme for $\mathcal{P}$ if it satisfies the following properties.

**Correctness.** For any $\lambda \in \mathbb{N}$ any $P \in \mathcal{P}_\kappa$, $N \leq 2^\lambda$, $S' \subseteq [N]$ such that $P(\mathsf{ind}_{S'}) = 1$, any $\mu \in \{0,1\}$,

$$\Pr\left[\mu = \mathsf{Recon}(P, (\mathsf{pk}_i)_i, (\mathsf{dsh}_j)_j, \mathsf{ct}, \mathsf{coin}) \quad : \quad \begin{array}{l} \mathsf{crs} \leftarrow \mathsf{GSetup}(1^\lambda), \\ \forall\, i \in [N], (\mathsf{pk}_i, \mathsf{sk}_i) \leftarrow \mathsf{KGen}, \\ (\mathsf{mpk}, \mathsf{coin}) \leftarrow \mathsf{PreProcess}(P, (\mathsf{pk}_i)_{i\in[N]}), \\ \mathsf{ct} \leftarrow \mathsf{Enc}(\mathsf{mpk}, \mathsf{coin}, \mu), \\ \forall\, j \in S', \mathsf{dsh}_j \leftarrow \mathsf{PartDec}(\mathsf{ct}, \mathsf{coin}, \mathsf{sk}_j) \end{array}\right] = 1$$

Here, $\mathsf{ind}_{S'} \in \{0,1\}^N$ is the indicator vector for the set $S' \subseteq [N]$.

**Compactness.** There exists a polynomial $\mathsf{poly}(\cdot,\cdot)$ such that $\forall\ \lambda \in \mathbb{N}, N \leq 2^\lambda$, $|\mathsf{ct}| + |\mathsf{coin}| = o(P) \cdot \mathsf{poly}(\lambda, \log N)$.

**Adaptive Security.** For any stateful admissible PPT adversary $\mathcal{A}$, there exists a negligible function such that $\forall\ \lambda \in \mathbb{N}$,

$$\left|\Pr\left[b \leftarrow \mathcal{A}^{\mathcal{O}(\cdot)}(\mathsf{ct}^*, \mathsf{coin}) \quad : \quad \begin{array}{l} \mathcal{K} := \emptyset, \mathsf{ctr} := 1, b \leftarrow \{0,1\}, \\ \mathsf{crs} \leftarrow \mathsf{GSetup}(1^\lambda), \\ (P, H_1, \ldots, H_N) \leftarrow \mathcal{A}^{\mathcal{O}(\cdot)}(\mathsf{crs}), \\ (\mathsf{mpk}, \mathsf{coin}) \leftarrow \mathsf{PreProcess}(P, (\mathsf{pk}_i)_{i\in[N]}), \\ \mathsf{ct}^* \leftarrow \mathsf{Enc}(\mathsf{mpk}, \mathsf{coin}, b) \end{array}\right] - \frac{1}{2}\right| \leq \mathsf{negl}(\lambda)$$

$\mathcal{O}$ is a stateful oracle that responds to the following queries.

- $\mathsf{KGen}$**.** $\mathcal{A}$ queries for key generation. $\mathcal{O}$ samples $(\mathsf{pk}_{\mathsf{ctr}}, \mathsf{sk}_{\mathsf{ctr}}) \leftarrow \mathsf{KGen}$, updates $\mathcal{K} := \mathcal{K} \cup \{(\mathsf{ctr}, \mathsf{pk}_{\mathsf{ctr}}, \mathsf{sk}_{\mathsf{ctr}}, 0)\}$ and sends $(\mathsf{ctr}, \mathsf{pk}_{\mathsf{ctr}})$ to $\mathcal{A}$. Then, $\mathcal{O}$ updates $\mathsf{ctr} := \mathsf{ctr} + 1$.
- $(\mathsf{Corr}, h')$**.** $\mathcal{A}$ queries for key corruption for handle $h' \in [\mathsf{ctr}]$. If there is no entry for $(h', \mathsf{pk}_{h'}, \mathsf{sk}_{h'}, 0)$ in $\mathcal{K}$ send $\perp$ to $\mathcal{A}$. Otherwise, $\mathcal{O}$ responds with $\mathsf{sk}_{h'}$ to $\mathcal{A}$ and updates the $h'$-th entry in $\mathcal{K}$ as $(h', \mathsf{pk}_{h'}, \mathsf{sk}_{h'}, 1)$.
- $(\mathsf{PartDec}, (\mathsf{ct}, \mathsf{coin}, h'))$**.** $\mathcal{A}$ queries for partial decryption for a ciphertext $\mathsf{ct}$ for handle $h' \in [\mathsf{ctr}]$. If there is no entry for $(h', \mathsf{pk}_{h'}, \mathsf{sk}_{h'}, 0)$ in $\mathcal{K}$ send $\perp$ to $\mathcal{A}$. Otherwise, $\mathcal{O}$ runs $\mathsf{dsh}_{h'} \leftarrow \mathsf{PartDec}(\mathsf{ct}, \mathsf{coin}, \mathsf{sk}_{h'})$ and sends $\mathsf{dsh}_{h'}$ to $\mathcal{A}$. Moreover, if $\mathsf{ct} = \mathsf{ct}^*$, now $h'$ will be considered as corrupted.

Here, $(\mathsf{pk}_i)_{i\in[N]}$ are derived as follows: each of $H_i$ for $i \in [N]$ is parsed as (1) $(\mathsf{ctr}, \perp, \perp)$ - in this case, we set $\mathsf{pk}_i = \mathsf{pk}_{\mathsf{ctr}}$ which corresponds to an honest / corrupted key or (2) $(\perp, \mathsf{pk}, \mathsf{sk})$ - in this case, we set $\mathsf{pk}_i = \mathsf{pk}$, which corresponds to a a malicious public key.

A stateful PPT machine $\mathcal{A}$ is said to be admissible if by the end of the security game, let $\mathcal{C}$ be the set of corrupted / malicious keys in the set $(\mathsf{pk}_i)_{i\in[N]}$ provided by $\mathcal{A}$. Then, $P(\mathsf{ind}_{\mathcal{C}}) = 0$ where $\mathsf{ind}_{\mathcal{C}}$ is the indicator vector for $\mathcal{C} \subseteq [N]$.

**Definition 4.2** (Static Security, [DJWW26])**.** An DPE scheme $(\mathsf{GSetup}, \mathsf{KGen}, \mathsf{Enc}, \mathsf{PartDec}, \mathsf{Dec})$ is said to be a statically secure DPE scheme if satisfies correctness and compactness of Definition 4.1 and security holds in the static security game which is the same as adaptive security game from Definition 4.1 except that $\mathcal{A}$ is no longer allowed any $(\mathsf{Corr}, h')$ queries ever and no $(\mathsf{PartDec}, (\mathsf{ct}^*, \mathsf{coin}, h'))$ in the post-challenge phase.

## 4.1 Equivocal DPE

In this section, we instantiate the equivocal encryption framework [GY26b, GY26a] in the context of DPE and define an Equivocal Distributed Monotone Policy Encryption scheme (EqDPE).

**Syntax.** An Equivocal Distributed Monotone Policy Encryption scheme (EqDPE) for $\mathcal{P} = \{\mathcal{P}_\lambda\}_{\lambda\in\mathbb{N}}$ consists of the following polynomial-time algorithms.

$\mathsf{GSetup}, \mathsf{KGen}, \mathsf{PreProcess}, \mathsf{Enc}, \mathsf{PartDec}, \mathsf{Recon}$**.** These are defined similarly to an DPE scheme (Definition 4.1).

$\mathsf{FakeSetup}(1^\lambda, 1^\kappa, \mu) \to (\widetilde{\mathsf{crs}}, \widetilde{\mathsf{ct}}, \mathsf{aux})$**.** The randomized fake setup algorithm takes as input security parameter $\lambda$, message $\mu \in \{0,1\}$, outputs fake CRS, $\widetilde{\mathsf{crs}}$, fake ciphertext $\widetilde{\mathsf{ct}}$, and auxiliary information $\mathsf{aux}$.

$\mathsf{SimKGen}(\mathsf{aux}, \mathsf{st}) \to (\mathsf{pk}, \mathsf{st}')$**.** The randomized *stateful* key generation simulation algorithm takes as input auxiliary information $\mathsf{aux}$, state $\mathsf{st}$, and outputs a public key, $\mathsf{pk}$, and updated state $\mathsf{st}'$.

$\mathsf{SimCorr}(\mathsf{aux}, \mathsf{st}, h) \to (\mathsf{sk}, \mathsf{st}')$**.** The randomized *stateful* corruption simulation algorithm takes as input auxiliary information $\mathsf{aux}$, state $\mathsf{st}$, handle $h$, and outputs a secret key, $\mathsf{sk}$, and updated state $\mathsf{st}'$.

$\mathsf{SimPartDec}(\mathsf{aux}, \mathsf{st}, \mathsf{ct}, \mathsf{coin}, h) \to (\mathsf{dsh}, \mathsf{st}')$**.** The randomized *stateful* partial decryption simulation algorithm takes as input auxiliary information $\mathsf{aux}$, state $\mathsf{st}$, ciphertext $\mathsf{ct}$, random coins $\mathsf{coin}$, handle $h$, and outputs a decryption share, $\mathsf{dsh}$, and updated state $\mathsf{st}'$.

$\mathsf{Equivocate}(\mathsf{aux}, \mathsf{st}, P, (H_1, \ldots, H_N)) \to (\mathsf{ct}, \mathsf{coin}, \mathsf{st}')$**.** The possibly randomized equivocation algorithm takes as input auxiliary information $\mathsf{aux}$, state $\mathsf{st}$, policy $P$, handles $(H_1, \ldots, H_N)$, and outputs ciphertext $\mathsf{ct}$, random coins $\mathsf{coin}$, and updated state $\mathsf{st}'$.

**Definition 4.3.** An EqDPE scheme ($\mathsf{GSetup}, \mathsf{KGen}, \mathsf{PreProcess}, \mathsf{Enc}, \mathsf{PartDec}, \mathsf{Recon}, \mathsf{FakeSetup}, \mathsf{SimKGen}, \mathsf{SimCorr}, \mathsf{SimPartDec}, \mathsf{Equivocate}$) should satisfy the following properties.

**Correctness and Compactness.** These are defined for ($\mathsf{GSetup}, \mathsf{KGen}, \mathsf{PreProcess}, \mathsf{Enc}, \mathsf{PartDec}, \mathsf{Recon}$) exactly as correctness and compactness for an DPE scheme (Definition 4.1).

**Mode Indistinguishability.** We define the following game between a stateful PPT adversary $\mathcal{A}$ and a challenger $\mathsf{Chal}$.

1. $\mathcal{A}$ sends $1^\kappa$, $\mu \in \{0,1\}$ to $\mathsf{Chal}$. $\mathsf{Chal}$ flips a coin $b \leftarrow \{0,1\}$. $\mathsf{Chal}$ initiates dictionary $\mathcal{K} := \emptyset$ and counter $\mathsf{ctr} := 1$.
2. If $b = 0$, $\mathsf{Chal}$ samples $\mathsf{crs}^{(0)} \leftarrow \mathsf{GSetup}(1^\lambda)$, and sends $\mathsf{crs}^{(0)}$ to $\mathcal{A}$.
   Otherwise, if $b = 1$, $\mathsf{Chal}$ samples $(\mathsf{crs}^{(1)}, \widetilde{\mathsf{ct}}, \mathsf{aux}) \leftarrow \mathsf{FakeSetup}(1^\lambda, 1^\kappa, \mu)$ and sends $\mathsf{crs}^{(1)}$ to $\mathcal{A}$. $\mathsf{Chal}$ initiates $\mathsf{st} := (\mathsf{ctr}, \mathcal{K})$.
3. $\mathcal{A}$ (polynomially-many times) issues the following queries.
   - $\mathsf{KGen}$**.** If $b = 0$, $\mathsf{Chal}$ computes $(\mathsf{pk}_{\mathsf{ctr}}, \mathsf{sk}_{\mathsf{ctr}}) \leftarrow \mathsf{KGen}$ and updates $\mathcal{K} := \mathcal{K} \cup \{(\mathsf{ctr}, \mathsf{pk}_{\mathsf{ctr}}, \mathsf{sk}_{\mathsf{ctr}}, 0)\}$. If $b = 1$, $\mathsf{Chal}$ computes $(\mathsf{pk}_{\mathsf{ctr}}, \mathsf{st}') \leftarrow \mathsf{SimKGen}(\mathsf{aux}, \mathsf{st})$ and updates $\mathsf{st} := \mathsf{st}'$. In either case, $\mathsf{Chal}$ sends $(\mathsf{ctr}, \mathsf{pk}_{\mathsf{ctr}})$ to $\mathcal{A}$ and updates $\mathsf{ctr} := \mathsf{ctr} + 1$.
   - $(\mathsf{Corr}, h')$**.** If there is no entry for handle $h'$ in $\mathcal{K}$, $\mathsf{Chal}$ sends $\perp$ to $\mathcal{A}$. Otherwise, if $b = 0$, $\mathsf{Chal}$ sends $\mathsf{sk}_{h'}$ and updates $h'$-th entry as $\{(h', \mathsf{pk}_{h'}, \mathsf{sk}_{h'}, 1)\}$. If $b = 1$, $\mathsf{Chal}$ computes $(\mathsf{sk}_{h'}, \mathsf{st}') \leftarrow \mathsf{SimCorr}(\mathsf{aux}, \mathsf{st}, h')$, updates $\mathsf{st} := \mathsf{st}'$, and sends $\mathsf{sk}_{h'}$.
   - $(\mathsf{PartDec}, (\mathsf{ct}, \mathsf{coin}, h'))$**.** If there is no entry for handle $h'$, $\mathsf{Chal}$ sends $\perp$ to $\mathcal{A}$. Otherwise, if $b = 0$, $\mathsf{Chal}$ sends $\mathsf{dsh}_{h'} \leftarrow \mathsf{PartDec}(\mathsf{ct}, \mathsf{coin}, \mathsf{sk}_{h'})$. If $b = 1$, $\mathsf{Chal}$ computes $(\mathsf{dsh}_{h'}, \mathsf{st}') \leftarrow \mathsf{SimPartDec}(\mathsf{aux}, \mathsf{st}, \mathsf{ct}, \mathsf{coin}, h')$, updates $\mathsf{st} := \mathsf{st}'$, and sends $\mathsf{dsh}_{h'}$.
4. $\mathcal{A}$ sends $(P, H_1, \ldots, H_N)$. $\mathsf{Chal}$ derives $(\mathsf{pk}_i)_{i \in [N]}$ as in Definition 4.1. If $b = 0$, $\mathsf{Chal}$ samples $(\mathsf{mpk}, \mathsf{coin}) \leftarrow \mathsf{PreProcess}(P, (\mathsf{pk}_i)_{i \in [N]})$ and $\mathsf{ct}^* \leftarrow \mathsf{Enc}(\mathsf{mpk}, \mathsf{coin}, \mu)$ and sends $(\mathsf{ct}^*, \mathsf{coin})$ to $\mathcal{A}$.
   Otherwise, if $b = 1$, $\mathsf{Chal}$ computes $(\mathsf{ct}^*, \mathsf{coin}, \mathsf{st}') \leftarrow \mathsf{Equivocate}(\mathsf{aux}, \mathsf{st}, P, (H_1, \ldots, H_N))$ and sends $\mathsf{ct}^*, \mathsf{coin}$ to $\mathcal{A}$. $\mathsf{Chal}$ also updates $\mathsf{st} := \mathsf{st}'$.
5. $\mathcal{A}$ resumes the queries of Step 3 except now, if $\mathsf{ct} = \mathsf{ct}^*$, now $h'$ will be considered as corrupted.
6. Finally $\mathcal{A}$ outputs its guess for $b$ as $b'$.

We say that EqDPE satisfies mode indistinguishability property if for any stateful admissible PPT adversary $\mathcal{A}$, there exists a negligible function $\mathsf{negl}(\cdot)$ such that for any $\lambda \in \mathbb{N}$,

$$\left| \Pr[b' = b] - \frac{1}{2} \right| \leq \mathsf{negl}(\lambda)$$

**Selective Security for Fake Setup.** For any stateful admissible PPT adversary $\mathcal{A}$, there exists a negligible function $\mathsf{negl}(\cdot)$ such that for any $\lambda \in \mathbb{N}$,

$$\left|\Pr\left[\ b \leftarrow \mathcal{A}(\widetilde{\mathsf{crs}}, \widetilde{\mathsf{ct}}, \mathsf{aux}) \quad : \quad \begin{array}{l} b \leftarrow \{0,1\}, 1^\kappa \leftarrow \mathcal{A}(1^\lambda), \\ (\widetilde{\mathsf{crs}}, \widetilde{\mathsf{ct}}, \mathsf{aux}) \leftarrow \mathsf{FakeSetup}(1^\lambda, 1^\kappa, b) \end{array}\ \right] - \frac{1}{2}\right| \leq \mathsf{negl}(\lambda)$$

## 4.2 DPE from EqDPE

**Construction 4.4.** We construct an DPE scheme for $\mathcal{P} = \{\mathcal{P}_\lambda\}_{\lambda\in\mathbb{N}}$ (Definition 4.1) using an EqDPE scheme (Definition 4.3) for $\mathcal{P}$ as follows.

$\mathsf{GSetup}(1^\lambda, 1^\kappa)$**.** Sample $\mathsf{dpe.crs} \leftarrow \mathsf{EqDPE.GSetup}(1^\lambda, 1^\kappa)$. Output $\mathsf{crs} := \mathsf{dpe.crs}$.

$\mathsf{KGen}$**.** Sample $(\mathsf{dpe.pk}, \mathsf{dpe.sk}) \leftarrow \mathsf{EqDPE.KGen}$. Output $(\mathsf{pk}, \mathsf{sk}) := (\mathsf{dpe.pk}, \mathsf{dpe.sk})$.

$\mathsf{PreProcess}(P, (\mathsf{pk}_i)_{i\in[N]})$**.** Parse $\mathsf{pk}_i$ as $\mathsf{dpe.pk}_i$ for $i \in [N]$. Sample $(\mathsf{dpe.mpk}, \mathsf{dpe.coin}) \leftarrow \mathsf{EqDPE.PreProcess}$ $(P, (\mathsf{dpe.pk}_i)_{i\in[N]})$.

Output $\mathsf{mpk} := \mathsf{dpe.mpk}$ and $\mathsf{coin} := \mathsf{dpe.coin}$.

$\mathsf{Enc}(\mathsf{mpk}, \mathsf{coin}, \mu)$**.** Parse $\mathsf{pk}_i$ as $\mathsf{dpe.pk}_i$ for $i \in [N]$, $\mathsf{mpk}$ as $\mathsf{dpe.mpk}$, and $\mathsf{coin}$ as $\mathsf{dpe.coin}$. Sample $\mathsf{dpe.ct} \leftarrow \mathsf{EqDPE.Enc}(\mathsf{dpe.mpk}, \mathsf{dpe.coin}, \mu)$. Output $\mathsf{ct} := \mathsf{dpe.ct}$.

$\mathsf{PartDec}(\mathsf{ct}, \mathsf{coin}, \mathsf{sk})$**.** Parse $\mathsf{sk}$ as $\mathsf{dpe.sk}$ and $\mathsf{ct}$ as $\mathsf{dpe.ct}$. Output $\mathsf{dsh} \leftarrow \mathsf{EqDPE.PartDec}(\mathsf{dpe.ct}, \mathsf{dpe.coin}, \mathsf{dpe.sk})$.

$\mathsf{Recon}(P, (\mathsf{pk}_i)_{i\in[N]}, (\mathsf{dsh}_j)_{j\in S'}, \mathsf{ct}, \mathsf{coin})$**.** Parse $\mathsf{pk}_i$ as $\mathsf{dpe.pk}_i$ for $i \in [N]$, $\mathsf{dsh}_j$ as $\mathsf{dpe.dsh}_j$ for $j \in S'$, $\mathsf{ct}$ as $\mathsf{dpe.ct}$, and $\mathsf{coin}$ as $\mathsf{dpe.coin}$.

Output $\mu' = \mathsf{EqDPE.Recon}(P, (\mathsf{dpe.pk}_i)_i, (\mathsf{dpe.dsh}_j)_j, \mathsf{dpe.ct}, \mathsf{dpe.coin})$.

**Correctness and Compactness.** This follows from correctness and compactness of EqDPE.

**Theorem 4.5.** If EqDPE is an EqDPE scheme (Definition 4.3) for $\mathcal{P} = \{\mathcal{P}_\kappa\}_{\kappa\in\mathbb{N}}$, then Construction 4.4 is an adaptively secure DPE scheme for $\mathcal{P}$ (Definition 4.1).

*Proof Sketch.* We prove this theorem using the following hybrids for an admissible adversary.

**Hybrid 0.** This is the adaptive security experiment from Definition 4.1 with $\mu = 0/1$.

**Hybrid 1.** In this hybrid, we will use $\mathsf{FakeSetup}(1^\lambda, 1^\kappa, \mu)$ to generate $(\widetilde{\mathsf{crs}}, \widetilde{\mathsf{ct}}, \mathsf{aux})$. We will also use $\mathsf{aux}$ and $\mathsf{SimKGen}$ to generate $\mathsf{pk}_{\mathsf{ctr}}$ for any key generation query, $\mathsf{SimCorr}$ to answer any corruption query, and $\mathsf{SimPartDec}$ to answer any partial decryption query from $\mathcal{A}$. Finally, when $\mathcal{A}$ sends $(P, H_1, \ldots, H_N)$, we will use $\mathsf{Equivocate}(\mathsf{aux}, \mathsf{st}, P, (H_1, \ldots, H_N))$ to generate the challenge ciphertext $\mathsf{ct}^*, \mathsf{coin}$. For each $b \in \{0,1\}$, Hybrids 0 and 1 are indistinguishable because of the mode indistinguishability property of EqDPE.

Observe that Hybrid 1 we can use selective security for fake setup of Definition 4.3 to change $\mu$. This proves that EqDPE yields an adaptively secure DPE scheme. We omit full proof for brevity. □

**Remark 4.6** (Static EqDPE)**.** Removing the corruption interface yields the static variant of EqDPE. Concretely, we drop the $\mathsf{SimCorr}$ algorithm from the syntax, remove the $(\mathsf{Corr}, h')$ query from the mode indistinguishability game and the oracle $\mathcal{O}'$ of Definition 4.3. We call the resulting notion a static EqDPE. The proof of Theorem 4.5 goes through exactly same with the corruption oracle removed, and now yields a statically secure DPE scheme (Definition 4.2).

# 5 Static EqDPE for DNFs from Decomposed LWE

**DNF Notation.** We define a (monotone) Disjunctive Normal Form (DNF) formula $\phi$ over $L$ variables and $W$ wires as a disjunction of $K$ many clauses, $\phi = \mathsf{conj}_1 \vee \ldots \vee \mathsf{conj}_K$, where for each $i \in [K]$, $\mathsf{conj}_i$ is a conjunction of $L_i$ many variables. That is, $\mathsf{conj}_i = x_1 \wedge \ldots \wedge x_{L_i}$ and each $x_\iota$ may be reused in another clause. Consequently, $\sum_{i=1}^{K} L_i = W$. For monotone-policy encryption, we also consider a wire labeling function $\rho : [W] \to [N]$ that maps each wire in DNF formula $\phi$ to and element in $[N]$. This function need not be injective and is used for mapping public keys to wires. We also support any policy class and thus take $\kappa$ to be empty.

**Building Blocks.** Our construction uses the following primitives.

- An equivocal matrix commitment scheme $\mathsf{EqMCOM}$ (Definition 3.9).
- A simulation-sound extractable NIZK (Definition 3.7) for the language $\mathcal{L} = \{x := (\mathbf{B}, \mathbf{c}_1^\top, \chi, \mathsf{ct}') \ : \ \exists\, w := (\mathbf{s}, \mathbf{e}), \mathbf{c}_1^\top = \mathbf{s}^\top \mathbf{B} + \mathbf{e}^\top \ \wedge \ \|\mathbf{e}\| \leq \sqrt{m}\chi\}$.
- Hash function $\mathcal{H} : \{0,1\}^\lambda \times \{0,1\}^\lambda \to \mathcal{D}_{\mathbb{Z},\sigma}^m$ which will be modeled as random oracle[3] in security proof.

**Parameters.** We provide the parameters used by our construction below.

- $n = \mathsf{poly}(\lambda)$ (Decomposed LWE security)
- $m = \Omega(n \log q)$ (Leftover hash lemma)
- $\chi = \mathsf{poly}(\lambda)$ (Decomposed LWE security)
- $\widetilde{\chi} > \lambda^{\omega(1)} m^{11/2} \chi$ (Smudging)
- $\sigma = \omega(\log m)$ (Decomposed LWE security)
- Odd prime $q > O(\lambda \cdot \mathsf{poly}(N) \cdot \chi \cdot \widetilde{\chi} \cdot \sigma \cdot \widetilde{\sigma} \cdot m^{10} \cdot \log q)$ (Correctness)

One way to instantiate these parameters is for constant $0 < \epsilon < 1/2$, $n = (\lambda)^{\Theta(1/\epsilon)} \cdot \mathsf{poly}(\log \lambda)$, $m = 4n \log q$, $\chi = \mathsf{poly}(n)$, $\sigma = \log^2 m$, and $q = \mathsf{poly}(n, 2^\lambda)$. This would require sub-exponential modulus-to-noise ratio.

**Construction 5.1.** We construct a static EqDPE scheme for DNFs assuming the hardness of $2m^2$-decomposed LWE using the above building blocks as follows.

$\mathsf{GSetup}(1^\lambda)$**.** Sample $(\mathsf{pp}, \mathbf{W}, \mathbf{D}, \mathbf{B}) \leftarrow \mathsf{EqMCOM.Setup}(1^n, 1^m, q, \sigma, \lambda^{\omega(1)})$. Sample $\mathbf{p} \leftarrow \mathbb{Z}_q^n$ and $\mathsf{nizk.crs} \leftarrow \mathsf{NIZK.Setup}(1^\lambda)$. Sample hash function $\mathcal{H} : \{0,1\}^\lambda \times \{0,1\}^\lambda \to \mathcal{D}_{\mathbb{Z},\sigma}^m$.

Output $\mathsf{crs} := (\mathsf{pp}, \mathbf{W}, \mathbf{D}, \mathbf{B}, \mathbf{p}, \mathsf{nizk.crs}, \mathcal{H})$.

$\mathsf{KGen}(\mathsf{crs})$**.** Sample $\mathbf{R} \leftarrow \{\pm 1\}^{4m^5 \times m}$ and compute $\mathbf{T} := \mathbf{BR}$.

Output $\mathsf{pk} := \mathbf{T}$ and $\mathsf{sk} := \mathbf{R}$.

$\mathsf{PreProcess}((\phi, \rho), (\mathsf{pk}_i)_{i \in [N]})$**.** Parse $\mathsf{pk}_i$ as $\mathbf{T}_i \in \mathbb{Z}_q^{n \times m}$ for each $i \in [N]$ and $\rho : [W] \to [N]$. Do the following.

1. Sample $\mathsf{sd} \leftarrow \{0,1\}^\lambda$ and for each $w \in [W]$, sample $\mathbf{u}_w \leftarrow \mathcal{H}(\mathsf{sd}, w)$.
2. Compute $\mathbf{t}_w = \mathbf{T}_{\rho(w)} \mathbf{u}_w$.
3. Let $K$ be the number of clauses in $\phi$ and each $i \in [K]$, let $L_i$ be the number of wires in each clause. Then, parse $\{\mathbf{t}_w\}_{w \in [W]}$ as $\{\mathbf{t}_{i,j}\}_{i \in [K], j \in [L_i]}$ according to $\phi$.
4. Compute matrix $\mathbf{M} = \left[ \left( \sum_{j=1}^{L_1} \mathbf{t}_{1,j} \right) + \mathbf{p} \mid \ldots \mid \left( \sum_{j=1}^{L_K} \mathbf{t}_{K,j} \right) + \mathbf{p} \right] \in \mathbb{Z}_q^{n \times K}$.
5. Sample $(\mathbf{C}, \mathsf{aux}) \leftarrow \mathsf{EqMCOM.Com}(\mathbf{M})$.

[3] We employ a random oracle that outputs elements from a discrete gaussian distribution over integers. This can be replaced by a random oracle that outputs uniformly random bit strings and an 'explainable' gaussian sampler as defined in [LW22]. Moreover in [LW22, Appendix B] showed that the rejection sampling of [GPV08] is statistically 'explainable'. So, using a random oracle that outputs discrete gaussian elements is a standard procedure.

Output $\mathsf{mpk} := \mathbf{C}, \mathsf{coin} := (\mathsf{aux}, \mathsf{sd})$.

$\mathsf{Enc}(\mathsf{mpk}, \mathsf{coin}, \mu)$. Parse $\mathsf{mpk}$ as $\mathbf{C} \in \mathbb{Z}_q^{n \times m}$, $\mathsf{coin}$ as $(\mathsf{aux}, \mathsf{sd}) \in \{0,1\}^{\mathsf{poly}(\lambda)} \times \{0,1\}^{\lambda}$. Do the following.

1. Sample $\mathbf{s} \leftarrow \mathbb{Z}_q^n, \mathbf{e} \leftarrow \mathcal{D}_{\mathbb{Z},\chi}^{4m^5}, \widetilde{\mathbf{e}} \leftarrow [-\sqrt{m}\widetilde{\chi}, \sqrt{m}\widetilde{\chi}]^m, \mathbf{r_p} \leftarrow \{\pm 1\}^{4m^5}$.
2. Compute $\mathbf{c}_1^\top := \mathbf{s}^\top \mathbf{B} + \mathbf{e}^\top$, $\mathbf{c}_2^\top := \mathbf{s}^\top \mathbf{C} + \widetilde{\mathbf{e}}^\top$, and $c_3 = \mathbf{s}^\top \mathbf{p} + \mathbf{e}^\top \mathbf{r_p} + \lfloor q/2 \rfloor \mu$.
3. Compute $\mathsf{nizk}.\pi \leftarrow \mathsf{NIZK.Prove}((\mathbf{B}, \mathbf{c}_1^\top, \chi, \mathbf{c}_2^\top, c_3, \mathsf{aux}, \mathsf{sd}), (\mathbf{s}, \mathbf{e}))$.

Output $\mathsf{ct} := (\mathbf{c}_1^\top, \mathbf{c}_2^\top, c_3, \mathsf{nizk}.\pi) \in \mathbb{Z}_q^{1 \times (4m^5+m+1)} \times \{0,1\}^{\mathsf{poly}_\pi(\lambda)}$.

$\mathsf{PartDec}(\mathsf{ct}, \mathsf{coin}, \mathsf{sk})$. Parse $\mathsf{ct}$ as $(\mathbf{c}_1^\top, \mathbf{c}_2^\top, c_3, \mathsf{nizk}.\pi)$, $\mathsf{coin}$ as $(\mathsf{aux}, \mathsf{sd})$ and $\mathsf{sk}$ as $\mathbf{R} \in \{\pm 1\}^{4m^5 \times m}$. If $\mathsf{NIZK.Verify}((\mathbf{B}, \mathbf{c}_1^\top, \chi, (\mathbf{c}_2^\top, c_3, \mathsf{aux}, \mathsf{sd})), \mathsf{nizk}.\pi) = 0$, output $\perp$. Otherwise, sample $\widehat{\mathbf{e}} \leftarrow [-\sqrt{m}\widetilde{\chi}, \sqrt{m}\widetilde{\chi}]^m$ and output $\mathsf{dsh} := \mathbf{c}_1^\top \mathbf{R} + \widehat{\mathbf{e}}^\top \in \mathbb{Z}_q$.

$\mathsf{Recon}((\phi, \rho), (\mathsf{pk}_i)_{i \in [N]}, (i^*, (\mathsf{dsh}_j)_{j \in S'}), \mathsf{ct}, \mathsf{coin})$. Parse $\mathsf{pk}_{i,j}$ as $\mathbf{t}_{i,j}$, $\mathsf{ct}$ as $(\mathbf{c}_1^\top, \mathbf{c}_2^\top, c_3, \mathsf{nizk}.\pi)$, and $\mathsf{coin}$ as $(\mathsf{aux}, \mathsf{sd})$. If $\mathsf{NIZK.Verify}((\mathbf{B}, \mathbf{c}_1^\top, \chi, \mathbf{c}_2^\top, \mathsf{aux}, \mathsf{sd}), \mathsf{nizk}.\pi) = 0$, output $\perp$. Let $i^* \in [K]$ be the clause that is satisfied by $S'$ (according to $\rho$). If there is no such $i^*$, abort and output $\perp$. Otherwise, do the following.

1. Using $\mathsf{sd}$, compute $\mathbf{t}_{i,j}, \mathbf{u}_{i,j}$ for each $i \in [K], j \in [L_i]$ like in steps 1-3 of $\mathsf{Enc}$.
2. Compute $\mathbf{M}$ as in step 4 of $\mathsf{Enc}$, $\mathbf{Z} = \mathsf{EqMCOM.Open}(\mathbf{M}, \mathsf{aux})$, and $\mathbf{V}_K = \mathsf{EqMCOM.Verify}(1^K)$.
3. Let $\mathbf{v}_{i^*}, \mathbf{z}_{i^*}$ denote the $i^*$-th columns of $\mathbf{V}_K, \mathbf{Z}$ respectively.
4. Compute $c_{i^*} = \mathbf{c}_2^\top \mathbf{v}_{i^*} + \mathbf{c}_1^\top \mathbf{z}_{i^*} - \sum_{j=1}^{L_{i^*}} \mathsf{dsh}_j \mathbf{u}_{i^*,j}$. Compute $y = c_3 - c_{i^*}$.

Output $\mu' := 0$ if $-q/4 < y < q/4$ and $\mu' := 1$ otherwise.

**Correctness.** Fix a clause $i^* \in [K]$ satisfied by a reconstructing set $S' \subseteq [N]$. For each wire $j \in [L_{i^*}]$ of clause $i^*$, let $\nu_j := \rho(w_{i^*,j}) \in S'$ be its labeling party, so that $\mathsf{pk}_{\nu_j} = \mathbf{T}_{\nu_j} = \mathbf{B}\mathbf{R}_{\nu_j}$ with $\mathbf{R}_{\nu_j} \in \{\pm 1\}^{4m^5 \times m}$. Writing $\mathbf{u}_{i^*,j} = \mathcal{H}(\mathsf{sd}, w_{i^*,j})$, the wire sub-key is $\mathbf{t}_{i^*,j} = \mathbf{T}_{\nu_j}\mathbf{u}_{i^*,j} = \mathbf{B}\mathbf{R}_{\nu_j}\mathbf{u}_{i^*,j} = \mathbf{B}\mathbf{r}_{i^*,j}$, $\mathbf{r}_{i^*,j} := \mathbf{R}_{\nu_j}\mathbf{u}_{i^*,j} \in \mathbb{Z}^{4m^5}$. By correctness of $\mathsf{EqMCOM}$, $\mathbf{C}\mathbf{V}_K = \mathbf{M} - \mathbf{B}\mathbf{Z}$, so for the $i^*$-th column $\mathbf{C}\mathbf{v}_{i^*} = \mathbf{m}_{i^*} - \mathbf{B}\mathbf{z}_{i^*}$ where $\mathbf{M}_{i^*} = \sum_j \mathbf{t}_{i^*,j} + \mathbf{p}$. Each party $\nu$ produces the share $\mathsf{dsh}_\nu = \mathbf{c}_1^\top \mathbf{R}_\nu + \widehat{\mathbf{e}}_\nu^\top \in \mathbb{Z}_q^{1 \times m}$ with $\mathbf{c}_1^\top = \mathbf{s}^\top \mathbf{B} + \mathbf{e}^\top$, and $\mathsf{Recon}$ combines them as $\sum_{j=1}^{L_{i^*}} \mathsf{dsh}_{\nu_j} \mathbf{u}_{i^*,j}$ (applying each party's share to the labeling wire's hash vector). Hence,

$$
\begin{aligned}
c_{i^*} &= \mathbf{c}_2^\top \mathbf{v}_{i^*} + \mathbf{c}_1^\top \mathbf{z}_{i^*} - \sum_{j=1}^{L_{i^*}} \mathsf{dsh}_{\nu_j} \mathbf{u}_{i^*,j} \\
&= \mathbf{s}^\top \mathbf{C}\mathbf{v}_{i^*} + \widetilde{\mathbf{e}}^\top \mathbf{v}_{i^*} + \mathbf{s}^\top \mathbf{B}\mathbf{z}_{i^*} + \mathbf{e}^\top \mathbf{z}_{i^*} - \sum_j \left( \mathbf{c}_1^\top \mathbf{R}_{\nu_j} \mathbf{u}_{i^*,j} + \widehat{\mathbf{e}}_{\nu_j}^\top \mathbf{u}_{i^*,j} \right) \\
&= \mathbf{s}^\top \mathbf{C}\mathbf{v}_{i^*} + \widetilde{\mathbf{e}}^\top \mathbf{v}_{i^*} + \mathbf{s}^\top \mathbf{B}\mathbf{z}_{i^*} + \mathbf{e}^\top \mathbf{z}_{i^*} - \sum_j \left( \mathbf{s}^\top \mathbf{B}\mathbf{r}_{i^*,j} + \mathbf{e}^\top \mathbf{r}_{i^*,j} + \widehat{\mathbf{e}}_{\nu_j}^\top \mathbf{u}_{i^*,j} \right) \\
&= \mathbf{s}^\top \left( \mathbf{C}\mathbf{v}_{i^*} + \mathbf{B}\mathbf{z}_{i^*} - \sum_j \mathbf{t}_{i^*,j} \right) + \underbrace{\widetilde{\mathbf{e}}^\top \mathbf{v}_{i^*} + \mathbf{e}^\top \mathbf{z}_{i^*} - \sum_j \mathbf{e}^\top \mathbf{r}_{i^*,j} - \sum_j \widehat{\mathbf{e}}_{\nu_j}^\top \mathbf{u}_{i^*,j}}_{\widetilde{e}} \\
&= \mathbf{s}^\top \mathbf{p} + \widetilde{e},
\end{aligned}
$$

Thus $y = c_3 - c_{i^*} = \lfloor q/2 \rfloor \mu + \mathbf{e}^\top \mathbf{r_p} - \widetilde{e}$, and $\mu' = \mu$ as long as $|\mathbf{e}^\top \mathbf{r_p} - \widetilde{e}| < q/4$. With high probability $\|\mathbf{e}\| \le \sqrt{m}\chi$, $\|\widetilde{\mathbf{e}}\| \le \sqrt{m}\widetilde{\chi}$, $\|\widehat{\mathbf{e}}_{\nu_j}\| \le \sqrt{m}\widetilde{\chi}$, $\|\mathbf{u}_{i^*,j}\| \le \sqrt{m}\sigma$, and $\|\mathbf{r_p}\| = 1$. Since $\mathbf{R}_{\nu_j} \in \{\pm 1\}^{4m^5 \times m}$, we have

$\|\mathbf{r}_{i^*,j}\| = \|\mathbf{R}_{\nu_j}\mathbf{u}_{i^*,j}\| \le m \cdot \sqrt{m}\sigma = m^{3/2}\sigma$. By compactness of $\mathsf{EqMCOM}$, $\|\mathbf{v}_{i^*}\| \le \|\mathbf{V}_K\| = O(\sigma m^{9/2}\log q)$ and $\|\mathbf{z}_{i^*}\| \le \|\mathbf{Z}\| = O(\sqrt{m}\widetilde{\sigma})$. Combining these (and using $L_{i^*} \le W$), $|\mathbf{e}^\top \mathbf{r}_{\mathbf{p}} - \widetilde{e}| \le O(W \cdot \chi\widetilde{\chi}\sigma\widetilde{\sigma}m^{10}\log q)$. Correctness holds whenever $q > O(\lambda \cdot W \cdot \chi\widetilde{\chi}\sigma\widetilde{\sigma}m^{10}\log q)$ (which is the constraint of the parameter table, with the number of wires $W$ in place of $N$).

**Compactness.** The size of $\mathsf{ct}$ and $\mathsf{coin}$ is $(4m^5+m+1)\lceil\log q\rceil + |\mathsf{aux}| + \lambda + |\pi|$. We have $|\mathsf{aux}| \le \mathsf{poly}(\lambda, \log K)$ by compactness of $\mathsf{EqMCOM}$, and $|\pi| \le \mathsf{poly}(\lambda)$ since the NIZK witness $(\mathbf{s}, \mathbf{e})$ has size $\mathsf{poly}(\lambda)$. Thus $|\mathsf{ct}| = O(m^5 \log q) + \mathsf{poly}(\lambda, \log N) = \mathsf{poly}(\lambda, \log N)$.

We also remark that, $|\mathsf{crs}| = \mathsf{poly}(\lambda, \log N)$, $|\mathsf{pk}| = nm\lceil\log q\rceil$, $|\mathsf{sk}| = 4m^6$, and $|\mathsf{dsh}| = m\lceil\log q\rceil$, all polynomially bounded in $\lambda$. Moreover, since $\mathsf{crs}$ is public-coin it can be replaced by a random oracle for an even more compact $\mathsf{crs}$.

## 5.1 Fake Setup, Key Generation and Partial Decryption Simulation, and Equivocation

We now describe our $\mathsf{FakeSetup}, \mathsf{SimKGen}, \mathsf{SimPartDec}$, and $\mathsf{Equivocate}$ algorithms. We use the simulated setup $\mathsf{NIZK.SimSetup}$ (which outputs a simulation trapdoor $\mathsf{nizk.td}$ allowing both proof simulation via $\mathsf{NIZK.SimProve}$ and witness extraction via $\mathsf{NIZK}.\mathcal{E}$), and treat the hash function $\mathcal{H}$ as a programmable random oracle, maintained as a dictionary $\mathsf{H}$.

$\mathsf{FakeSetup}(1^\lambda, \mu)$**.** Given $\mu \in \{0,1\}$, do the following.

1. Sample $(\mathsf{pp}, \mathbf{W}, \mathbf{D}, \mathbf{B}) \leftarrow \mathsf{EqMCOM.Setup}(1^n, 1^m, q, \sigma, \lambda^{\omega(1)})$, $(\mathsf{nizk.crs}, \mathsf{nizk.td}) \leftarrow \mathsf{NIZK.SimSetup}(1^\lambda)$.
2. Sample $\mathbf{s} \leftarrow \mathbb{Z}_q^n, \mathbf{e} \leftarrow \mathcal{D}_{\mathbb{Z},\chi}^{4m^5}, \mathbf{R}_{\mathbf{C}} \leftarrow \{\pm 1\}^{4m^5 \times m}, \mathbf{r}_{\mathbf{p}} \leftarrow \{\pm 1\}^{4m^5}, \mathbf{p} \leftarrow \mathbb{Z}_q^n$. Compute $\mathbf{c}_1^\top := \mathbf{s}^\top\mathbf{B} + \mathbf{e}^\top$ and $c_3 = \mathbf{s}^\top\mathbf{p} + \mathbf{e}^\top\mathbf{r}_{\mathbf{p}} + \lfloor q/2 \rfloor \mu$.
3. Set $\widetilde{\mathsf{crs}} := (\mathsf{pp}, \mathbf{W}, \mathbf{D}, \mathbf{B}, \mathbf{p}, \mathsf{nizk.crs}, \mathcal{H})$, $\widetilde{\mathsf{ct}} := (\mathbf{c}_1^\top, c_3)$, and $\mathsf{aux} := (\widetilde{\mathsf{crs}}, \widetilde{\mathsf{ct}}, \mathbf{R}_{\mathbf{C}}, \mathsf{nizk.td})$. Here $\mathcal{H}$ is a handle for random oracle queries.

Output $(\widetilde{\mathsf{crs}}, \widetilde{\mathsf{ct}}, \mathsf{aux})$.

$\mathsf{SimKGen}(\mathsf{aux}, \mathsf{st})$**.** Parse $\mathsf{st}$ as $(\mathsf{ctr}, \mathcal{K})$. Sample $(\mathbf{T}, T_{\mathbf{T}}) \leftarrow \mathsf{TrapGen}(1^n, 1^m, q)$. Set $\mathsf{pk} := \mathbf{T}$ and $\mathsf{st}' := (\mathsf{ctr}+1, \mathcal{K} \cup \{(\mathsf{ctr}, \mathbf{T}, T_{\mathbf{T}})\})$. Output $(\mathsf{pk}, \mathsf{st}')$.

$\mathsf{SimPartDec}(\mathsf{aux}, \mathsf{st}, \mathsf{ct}, \mathsf{coin}, h)$**.** Parse $\mathsf{ct}$ as $((\mathbf{c}_1')^\top, (\mathbf{c}_2')^\top, c_3', \mathsf{nizk}.\pi')$, $\mathsf{coin}$ as $(\mathsf{aux}', \mathsf{sd}')$, and retrieve $(h, \mathbf{T}_h, \cdot)$ from $\mathcal{K}$ in $\mathsf{st}$. If $\mathsf{NIZK.Verify}((\mathbf{B}, (\mathbf{c}_1')^\top, \chi, (\mathbf{c}')_2^\top, c_3', \mathsf{aux}', \mathsf{sd}'), \mathsf{nizk}.\pi') = 0$, output $\perp$. Otherwise, extract $(\mathbf{s}', \mathbf{e}') \leftarrow \mathsf{NIZK}.\mathcal{E}(\mathsf{nizk.td}, (\mathbf{B}, (\mathbf{c}_1')^\top, \chi, (\mathbf{c}')_2^\top, c_3', \mathsf{aux}', \mathsf{sd}'), \mathsf{nizk}.\pi')$, sample $\widehat{\mathbf{e}} \leftarrow [-\sqrt{m}\widetilde{\chi}, \sqrt{m}\widetilde{\chi}]^m$, and output $\mathsf{dsh} := (\mathbf{s}')^\top\mathbf{T}_h + \widehat{\mathbf{e}}^\top \in \mathbb{Z}_q^{1\times m}$.

$\mathsf{Equivocate}^{\mathsf{H}(\cdot)}(\mathsf{aux}, \mathsf{st}, (\phi, \rho), (H_1, \ldots, H_N))$**.** Parse $\mathsf{aux}$ as $(\widetilde{\mathsf{crs}}, \mathbf{c}_1^\top, \mathbf{R}_{\mathbf{C}}, \mathsf{nizk.td})$ and $\mathsf{st}$ as $(\mathsf{ctr}, \mathcal{K})$. Derive $(\mathsf{pk}_i)_{i\in[N]}$ from the handles as in Definition 4.1. For each clause $i \in [K]$, admissibility guarantees a single honest wire $(i, j_i^*)$. Let $\rho_i \in [N]$ be its labeling party (with $(\rho_i, \mathbf{T}_{\rho_i}, T_{\mathbf{T}_{\rho_i}}) \in \mathcal{K}$), and let the remaining wires of clause $i$ be labeled by malicious parties whose handles $S = (\perp, \mathbf{T}, \mathbf{R})$ carry a secret key $\mathbf{R}$. Sample $\mathsf{sd} \leftarrow \{0,1\}^\lambda$ (abort if $\mathsf{sd}$ already appears in $\mathsf{H}$) and do the following.

1. For each clause $i \in [K]$, sample $\mathbf{y}_i \leftarrow \mathcal{D}_{\mathbb{Z},\widetilde{\sigma}}^{4m^5}$ and set the target $\mathbf{d}_i^* := \mathbf{B}\mathbf{y}_i$. Program the honest wire's oracle value $\mathsf{H}(\mathsf{sd}, w_{i,j_i^*}) \leftarrow \mathsf{SamplePre}(\mathbf{T}_{\rho_i}, T_{\mathbf{T}_{\rho_i}}, \mathbf{d}_i^* - \mathbf{p}, \sigma)$, so that the honest sub-key is $\mathbf{t}_{i,j_i^*} = \mathbf{T}_{\rho_i} \cdot \mathsf{H}(\mathsf{sd}, w_{i,j_i^*}) = \mathbf{d}_i^* - \mathbf{p} = \mathbf{B}\mathbf{y}_i - \mathbf{p}$.
2. For every remaining wire $w$, sample $\mathsf{H}(\mathsf{sd}, w) \leftarrow \mathcal{D}_{\mathbb{Z},\sigma}^m$. Its sub-key is $\mathbf{t}_w = \mathbf{T}_{\rho(w)}\mathbf{u}_w = \mathbf{B}\mathbf{R}_{\rho(w)}\mathbf{u}_w = \mathbf{B}\mathbf{r}_w$ with $\mathbf{r}_w := \mathbf{R}_{\rho(w)}\mathbf{u}_w$ (known from the malicious secret key).
3. Compute $\mathbf{Y} := [\mathbf{y}_1 \mid \ldots \mid \mathbf{y}_K]$ and $\mathbf{R} := \left[\sum_{w\in\mathsf{conj}_1\setminus\{w_{1,j_1^*}\}} \mathbf{r}_w \mid \ldots \mid \sum_{w\in\mathsf{conj}_K\setminus\{w_{K,j_K^*}\}} \mathbf{r}_w\right]$, so that $\mathbf{M} := \mathbf{B}(\mathbf{Y} + \mathbf{R})$ agrees with the matrix $\mathbf{M}$ of $\mathsf{Enc}$.
4. Compute $\mathbf{C} := \mathbf{B}\mathbf{R}_{\mathbf{C}}$ and $\mathsf{eqcom.aux} \leftarrow \mathsf{EqMCOM.EqCom}(\mathbf{Y} + \mathbf{R}, \mathbf{R}_{\mathbf{C}})$.

5. Sample $\widetilde{\mathbf{e}} \leftarrow [-\sqrt{m}\widetilde{\chi}, \sqrt{m}\widetilde{\chi}]^m$ and $\mathsf{nizk.}\pi \leftarrow \mathsf{NIZK.SimProve}(\mathsf{nizk.td}, (\mathbf{B}, \mathbf{c}_1^\top, \chi, (\mathbf{c}_2^\top, c_3, \mathsf{eqcom.aux}, \mathsf{sd})))$.

Output $\mathsf{ct} := (\mathbf{c}_1^\top, \mathbf{c}_1^\top \mathbf{R}_{\mathbf{C}} + \widetilde{\mathbf{e}}^\top, c_3, \mathsf{nizk.}\pi)$, $\mathsf{coin} := (\mathsf{eqcom.aux}, \mathsf{sd})$, and $\mathsf{st}' := \mathsf{st}$.

## 5.2 Mode Indistinguishability Analysis

**Theorem 5.2** (Mode Indistinguishability.)**.** If EqMCOM satisfies commitment indistinguishability (Definition 3.9) and NIZK is a simulation-sound extractable NIZK for $\mathcal{L}$ (Definition 3.7), then Construction 5.1 satisfies the mode indistinguishability property (Definition 4.3) in the random oracle model.

*Proof.* We prove this theorem using the following hybrids.

**$\mathbf{Hyb}_0^{\mathcal{A}}(1^\lambda)$.** This is the mode indistinguishability experiment from Definition 4.3 with $b = 0, \mu = 0/1$.

- **Setup Phase.** Sample $(\mathsf{pp}, \mathbf{W}, \mathbf{D}, \mathbf{B}) \leftarrow \mathsf{EqMCOM.Setup}(1^n, 1^m, q, \sigma, \lambda^{\omega(1)})$. Sample $\mathbf{p} \leftarrow \mathbb{Z}_q^n$ and $\mathsf{nizk.crs} \leftarrow \mathsf{NIZK.Setup}(1^\lambda)$. Output $\mathsf{crs} := (\mathsf{pp}, \mathbf{W}, \mathbf{D}, \mathbf{B}, \mathbf{p}, \mathsf{nizk.crs}, \mathcal{H})$.

  Sample empty dictionaries $\mathsf{H}, \mathcal{K}$ and $\mathsf{ctr} := 1$.

- **Random Oracle Queries.** Given $(\mathsf{sd}, i) \in \{0,1\}^\lambda \times \{0,1\}^\lambda$, if there exists an entry for $(\mathsf{sd}, i)$ in $\mathsf{H}$, retrieve and send $\mathsf{H}[(\mathsf{sd}, i)]$. Otherwise, sample $\mathbf{u} \leftarrow \mathcal{D}_{\mathbb{Z},\sigma}^m$ and set $\mathsf{H}[(\mathsf{sd}, i)] := \mathbf{u}$. Reply with $\mathbf{u}$.

- **Key Generation Queries.** Sample $\mathbf{R} \leftarrow \{\pm 1\}^{4m^5 \times m}$ and compute $\mathbf{T} = \mathbf{BR}$. Update $\mathcal{K} := \mathcal{K} \cup \{(\mathsf{ctr}, \mathbf{T}, \mathbf{R}, 0)\}$. Send $(\mathsf{pk} := \mathbf{T}, \mathsf{ctr})$ to $\mathcal{A}$ and update $\mathsf{ctr} := \mathsf{ctr} + 1$.

- **Partial Decryption Queries.** Parse $\mathsf{ct}$ as $(\mathbf{c}_1^\top, \mathbf{c}_2^\top, c_3, \mathsf{nizk.}\pi)$ and $\mathsf{coin}$ as $(\mathsf{aux}, \mathsf{sd})$. If $\mathsf{NIZK.Verify}((\mathbf{B}, \mathbf{c}_1^\top, \chi, (\mathbf{c}_2^\top, c_3, \mathsf{aux}, \mathsf{sd})), \mathsf{nizk.}\pi) \neq 1$, abort and output $\perp$. Otherwise, Otherwise, sample $\widehat{\mathbf{e}} \leftarrow [-\sqrt{m}\widetilde{\chi}, \sqrt{m}\widetilde{\chi}]^m$ and output $\mathsf{dsh} = \mathbf{c}_1^\top \mathbf{R} + \widehat{\mathbf{e}}^\top$.

- **Challenge Encryption Query.** $\mathcal{A}$ sends $(\phi, \rho), (H_1, \ldots, H_N)$. Compute $\mathbf{T}_i$ for each $i \in [N]$ using $H_i$ like in Definition 4.3. Do the following.

  1. Sample $\mathsf{sd} \leftarrow \{0,1\}^\lambda$ and for each $w \in [W]$, sample $\mathbf{u}_w \leftarrow \mathcal{H}(\mathsf{sd}, w)$.
  2. Compute $\mathbf{t}_w = \mathbf{T}_{\rho(w)} \mathbf{u}_w$.
  3. Let $K$ be the number of clauses in $\phi$ and each $i \in [K]$, let $L_i$ be the number of wires in each clause. Then, parse $\{\mathbf{t}_w\}_{w \in [W]}$ as $\{\mathbf{t}_{i,j}\}_{i \in [K], j \in [L_i]}$ according to $\phi$.
  4. Compute matrix $\mathbf{M} = \left[\left(\sum_{j=1}^{L_1} \mathbf{t}_{1,j}\right) + \mathbf{p} \mid \ldots \mid \left(\sum_{j=1}^{L_K} \mathbf{t}_{K,j}\right) + \mathbf{p}\right] \in \mathbb{Z}_q^{n \times K}$.
  5. Sample $(\mathbf{C}, \mathsf{aux}) \leftarrow \mathsf{EqMCOM.Com}(\mathbf{M})$.
  6. Sample $\mathbf{s} \leftarrow \mathbb{Z}_q^n, \mathbf{e} \leftarrow \mathcal{D}_{\mathbb{Z},\chi}^{4m^5}, \widetilde{\mathbf{e}} \leftarrow [-\sqrt{m}\widetilde{\chi}, \sqrt{m}\widetilde{\chi}]^m, \mathbf{r}_{\mathbf{p}} \leftarrow \{\pm 1\}^{4m^5}$.
  7. Compute $\mathbf{c}_1^\top := \mathbf{s}^\top \mathbf{B} + \mathbf{e}^\top$, $\mathbf{c}_2^\top := \mathbf{s}^\top \mathbf{C} + \widetilde{\mathbf{e}}^\top$, and $c_3 = \mathbf{s}^\top \mathbf{p} + \mathbf{e}^\top \mathbf{r}_{\mathbf{p}} + \lfloor q/2 \rfloor \mu$.
  8. Compute $\mathsf{nizk.}\pi \leftarrow \mathsf{NIZK.Prove}((\mathbf{B}, \mathbf{c}_1^\top, \chi, \mathbf{c}_2^\top, c_3, \mathsf{aux}, \mathsf{sd}), (\mathbf{s}, \mathbf{e}))$.

  Send $\mathsf{ct}^* := (\mathbf{c}_1^\top, \mathbf{c}_2^\top, c_3, \mathsf{nizk.}\pi)$ and $\mathsf{coin} := (\mathsf{aux}, \mathsf{sd})$ to $\mathcal{A}$.

- **Partial Decryption Queries.** Perform these similarly to pre-challenge partial decryption queries.

- **Guess Phase.** Output whatever $\mathcal{A}$ outputs.

**$\mathbf{Hyb}_1^{\mathcal{A}}(1^\lambda)$.** This is the same as $\mathbf{Hyb}_0^{\mathcal{A}}(1^\lambda)$ except that we sample $(\mathsf{nizk.crs}, \mathsf{nizk.td}) \leftarrow \mathsf{NIZK.SimSetup}(1^\lambda)$ during setup, and sample the proof $\mathsf{nizk.}\pi$ in the challenge $\mathsf{ct}^*$ using $\mathsf{NIZK.SimProve}(\mathsf{nizk.td}, (\mathbf{B}, \mathbf{c}_1^\top, \chi, \mathbf{c}_2^\top, c_3, \mathsf{aux}, \mathsf{sd}))$ instead of NIZK.Prove.

$\mathbf{Hyb}_0^{\mathcal{A}}(1^\lambda)$ *and* $\mathbf{Hyb}_1^{\mathcal{A}}(1^\lambda)$ *are computationally indistinguishable by the computational zero-knowledge property of* NIZK *(Definition 3.7).*

$\mathbf{Hyb}_2^{\mathcal{A}}(1^\lambda)$. This is the same as $\mathbf{Hyb}_1^{\mathcal{A}}(1^\lambda)$ except that we answer each partial decryption query $(\mathsf{ct}', h)$ by extracting $(\mathbf{s}', \mathbf{e}') \leftarrow \mathsf{NIZK}.\mathcal{E}(\mathsf{nizk.td}, (\mathbf{B}, (\mathbf{c}_1')^\top, \chi, \mathbf{c}_2^\top, c_3, \mathsf{aux}, \mathsf{sd}), \pi')$ and returning $\mathsf{dsh} := (\mathbf{s}')^\top \mathbf{T}_h + \widehat{\mathbf{e}}^\top$ (rather than $(\mathbf{c}')^\top \mathbf{R}_h + \widehat{\mathbf{e}}^\top$).

$\mathbf{Hyb}_1^{\mathcal{A}}(1^\lambda)$ *and* $\mathbf{Hyb}_2^{\mathcal{A}}(1^\lambda)$ *are statistically close. By simulation-sound extractability, whenever* $\pi'$ *verifies the extractor returns* $(\mathbf{s}', \mathbf{e}')$ *with* $(\mathbf{c}')^\top = (\mathbf{s}')^\top \mathbf{B} + (\mathbf{e}')^\top$, $\|\mathbf{e}'\| \le \sqrt{m}\chi$. *Hence* $(\mathbf{c}')^\top \mathbf{R}_h = (\mathbf{s}')^\top (\mathbf{B}\mathbf{R}_h) + (\mathbf{e}')^\top \mathbf{R}_h = (\mathbf{s}')^\top \mathbf{T}_h + (\mathbf{e}')^\top \mathbf{R}_h$, *and since* $\|(\mathbf{e}')^\top \mathbf{R}_h\| \le 4m^{11/2}\chi$, *the two responses are statistically close by the smudging lemma (Lemma 3.3).*

$\mathbf{Hyb}_3^{\mathcal{A}}(1^\lambda)$. This is the same as $\mathbf{Hyb}_2^{\mathcal{A}}(1^\lambda)$ except that we answer key generation queries by sampling $(\mathbf{T}, T_{\mathbf{T}}) \leftarrow \mathsf{TrapGen}(1^n, 1^m, q)$ (storing $T_{\mathbf{T}}$).

$\mathbf{Hyb}_2^{\mathcal{A}}(1^\lambda)$ *and* $\mathbf{Hyb}_3^{\mathcal{A}}(1^\lambda)$ *are statistically close as* $\{(\mathbf{B}, \mathbf{B}\mathbf{R}) : \mathbf{R} \leftarrow \{\pm 1\}^{4m^5 \times m}\} \approx_s \{(\mathbf{B}, \mathbf{T}) : \mathbf{T} \leftarrow \mathbb{Z}_q^{n \times m}\}$ *by leftover hash lemma (Lemma B.3).*

$\mathbf{Hyb}_4^{\mathcal{A}}(1^\lambda)$. This is the same as $\mathbf{Hyb}_3^{\mathcal{A}}(1^\lambda)$ except that in the challenge we program each honest wire: sample $\mathbf{d}_i^* \leftarrow \mathbb{Z}_q^n$, set $\mathcal{H}(\mathsf{sd}, w_{i,j_i^*}) \leftarrow \mathsf{SamplePre}(\mathbf{T}_{\rho_i}, T_{\mathbf{T}_{\rho_i}}, \mathbf{d}_i^*, \sigma)$ so that $\mathbf{t}_{i,j_i^*} = \mathbf{d}_i^*$.

$\mathbf{Hyb}_3^{\mathcal{A}}(1^\lambda)$ *and* $\mathbf{Hyb}_4^{\mathcal{A}}(1^\lambda)$ *are statistically close by the gaussian pre-image distribution property of gadget trapdoors (Lemma B.6).*

$\mathbf{Hyb}_5^{\mathcal{A}}(1^\lambda)$. This is the same as $\mathbf{Hyb}_4^{\mathcal{A}}(1^\lambda)$ except that in the challenge we sample $\mathbf{y}_i \leftarrow \mathcal{D}_{\mathbb{Z},\widetilde{\sigma}}^{4m^5}$ and set $\mathbf{d}_i^* := \mathbf{B}\mathbf{y}_i$ (rather than $\mathbf{d}_i^* \leftarrow \mathbb{Z}_q^n$).

$\mathbf{Hyb}_4^{\mathcal{A}}(1^\lambda)$ *and* $\mathbf{Hyb}_5^{\mathcal{A}}(1^\lambda)$ *are statistically close by* $\{(\mathbf{B}, \mathbf{B}\mathbf{y}) : \mathbf{y} \leftarrow \mathcal{D}_{\mathbb{Z},\widetilde{\sigma}}^{4m^5}\} \approx_s \{(\mathbf{B}, \mathbf{d}^*) : \mathbf{d}^* \leftarrow \mathbb{Z}_q^n\}$ *(Lemma B.6).*

$\mathbf{Hyb}_6^{\mathcal{A}}(1^\lambda)$. This is the same as $\mathbf{Hyb}_5^{\mathcal{A}}(1^\lambda)$ except that in the challenge we sample $\mathbf{y}_i \leftarrow \mathcal{D}_{\mathbb{Z},\widetilde{\sigma}}^{4m^5}$ and set $\mathbf{d}_i^* := \mathbf{B}\mathbf{y}_i - \mathbf{p}$ (rather than just $\mathbf{d}_i^* \leftarrow \mathbb{Z}_q^n$).

$\mathbf{Hyb}_5^{\mathcal{A}}(1^\lambda)$ *and* $\mathbf{Hyb}_6^{\mathcal{A}}(1^\lambda)$ *are identical* $\{(\mathbf{B}, \mathbf{B}\mathbf{y}, \mathbf{B}\mathbf{y} + \mathbf{p})\} \equiv \{(\mathbf{B}, \mathbf{B}\mathbf{y} - \mathbf{p}, \mathbf{B}\mathbf{y})\}$. *Now the honest wire's sub-key is* $\mathbf{t}_{i,j_i^*} = \mathbf{B}\mathbf{y}_i - \mathbf{p}$, *so the* $i$*-th column of* $\mathbf{M}$ *equals* $\mathbf{B}\mathbf{y}_i + \sum_{w \in \mathsf{conj}_i \setminus \{w_{i,j_i^*}\}} \mathbf{B}\mathbf{r}_w$, *i.e.,* $\mathbf{M} = \mathbf{B}(\mathbf{Y} + \mathbf{R})$ *with* $\mathbf{Y} = [\mathbf{y}_1 \mid \ldots \mid \mathbf{y}_K]$ *and* $\mathbf{R} = [\sum_{w \in \mathsf{conj}_i \setminus \{w_{i,j_i^*}\}} \mathbf{r}_w]_{i \in [K]}$.

$\mathbf{Hyb}_7^{\mathcal{A}}(1^\lambda)$. This is the same as $\mathbf{Hyb}_6^{\mathcal{A}}(1^\lambda)$ except that during the challenge we sample $\mathbf{R}_{\mathbf{C}} \leftarrow \{\pm 1\}^{4m^5 \times m}$, set $\mathbf{C} := \mathbf{B}\mathbf{R}_{\mathbf{C}}$, and $\mathsf{aux}' \leftarrow \mathsf{EqMCOM.EqCom}(\mathbf{Y} + \mathbf{R}, \mathbf{R}_{\mathbf{C}})$, where $\mathbf{Y}, \mathbf{R}$ are as in $\mathbf{Hyb}_6^{\mathcal{A}}(1^\lambda)$ (so $\mathbf{M} = \mathbf{B}(\mathbf{Y} + \mathbf{R})$).

$\mathbf{Hyb}_6^{\mathcal{A}}(1^\lambda)$ *and* $\mathbf{Hyb}_7^{\mathcal{A}}(1^\lambda)$ *are computationally indistinguishable by the commitment indistinguishability property of* EqMCOM *(Definition 3.9). The reduction obtains each honest target* $\mathbf{d}_i^* = \mathbf{B}\mathbf{y}_i - \mathbf{p}$ *from the* EqMCOM *generation oracle (so it never needs the* $\mathbf{y}_i$*), programs the honest wire* $\mathcal{H}(\mathsf{sd}, w_{i,j_i^*}) \leftarrow \mathsf{SamplePre}(\mathbf{T}_{\rho_i}, T_{\mathbf{T}_{\rho_i}}, \mathbf{d}_i^*, \sigma)$ *using the trapdoo, and at challenge time submits the honest handles* $S = \{h_i\}_{i \in [K]}$ *together with* $\mathbf{R} = [\sum_{w \in \mathsf{conj}_i \setminus \{w_{i,j_i^*}\}} \mathbf{r}_w]_{i \in [K]}$. *Here* $\|\mathbf{R}\| \le W \cdot m^{3/2}\sigma \le \lambda^{\omega(1)}$, *the bound passed to* EqMCOM.Setup. *As the static game issues no corruption queries, the reduction makes no corruption queries to the* EqMCOM *oracle.*

$\mathbf{Hyb}_8^{\mathcal{A}}(1^\lambda)$. This is the same as $\mathbf{Hyb}_7^{\mathcal{A}}(1^\lambda)$ except that during the challenge we set $\mathbf{c}_2^\top := \mathbf{c}_1^\top \mathbf{R}_{\mathbf{C}} + \widetilde{\mathbf{e}}^\top$.

$\mathbf{Hyb}_7^{\mathcal{A}}(1^\lambda)$ *and* $\mathbf{Hyb}_8^{\mathcal{A}}(1^\lambda)$ *are statistically close by the smudging lemma (Lemma 3.3), since* $\mathbf{c}_1^\top \mathbf{R}_{\mathbf{C}} = \mathbf{s}^\top \mathbf{B}\mathbf{R}_{\mathbf{C}} + \mathbf{e}^\top \mathbf{R}_{\mathbf{C}}$ *and* $\|\mathbf{e}^\top \mathbf{R}_{\mathbf{C}}\| \le 4m^{11/2}\chi$.

$\mathbf{Hyb}_9^{\mathcal{A}}(1^\lambda)$. This is the mode indistinguishability experiment from Definition 4.3 with $b = 1$. The remaining changes are semantic. □

### 5.3 Selective Security for Fake Setup Analysis

**Theorem 5.3** (Selective Security for Fake Setup.)**.** If the $2m^2$-decomposed LWE assumption (Assumption 3.5) holds, then Construction 5.1 satisfies the selective security for fake setup property (Definition 4.3) in the random oracle model.

*Proof.* We prove this theorem using the following hybrids.

$\mathbf{Hyb}_0^{\mathcal{A}}(1^\lambda)$**.** This is the selective security for fake setup experiment from Definition 4.3 with $\mu = 0/1$.

- **Fake Setup.** Do the following.
  1. Sample $(\mathsf{pp}, \mathbf{W}, \mathbf{D}, \mathbf{B}) \leftarrow \mathsf{EqMCOM.Setup}(1^n, 1^m, q, \sigma, \lambda^{\omega(1)})$, $(\mathsf{nizk.crs}, \mathsf{nizk.td}) \leftarrow \mathsf{NIZK.SimSetup}(1^\lambda)$.
  2. Sample $\mathbf{s} \leftarrow \mathbb{Z}_q^n, \mathbf{e} \leftarrow \mathcal{D}_{\mathbb{Z},\chi}^{4m^5}, \mathbf{R_C} \leftarrow \{\pm 1\}^{4m^5 \times m}, \mathbf{r_p} \leftarrow \{\pm 1\}^{4m^5}, \mathbf{p} \leftarrow \mathbb{Z}_q^n$. Compute $\mathbf{c}_1^\top := \mathbf{s}^\top \mathbf{B} + \mathbf{e}^\top$ and $c_3 := \mathbf{s}^\top \mathbf{p} + \mathbf{e}^\top \mathbf{r_p} + \lfloor q/2 \rfloor \mu$.
  3. Set $\widetilde{\mathsf{crs}} := (\mathsf{pp}, \mathbf{W}, \mathbf{D}, \mathbf{B}, \mathbf{p}, \mathsf{nizk.crs}, \mathcal{H})$, $\widetilde{\mathsf{ct}} := (\mathbf{c}_1^\top, c_3)$, and $\mathsf{aux} := (\widetilde{\mathsf{crs}}, \widetilde{\mathsf{ct}}, \mathbf{R_C}, \mathsf{nizk.td})$.

  Send $(\widetilde{\mathsf{crs}}, \widetilde{\mathsf{ct}}, \mathsf{aux})$ to $\mathcal{A}$.

$\mathbf{Hyb}_1^{\mathcal{A}}(1^\lambda)$**.** This is the same as $\mathbf{Hyb}_0^{\mathcal{A}}(1^\lambda)$ except that we set $\mathbf{p} = \mathbf{B}\mathbf{r_p}$.

$\mathbf{Hyb}_0^{\mathcal{A}}(1^\lambda)$ *and* $\mathbf{Hyb}_1^{\mathcal{A}}(1^\lambda)$ *are statistically close by leftover hash lemma (Lemma B.3).*

$\mathbf{Hyb}_2^{\mathcal{A}}(1^\lambda)$**.** This is the same as $\mathbf{Hyb}_1^{\mathcal{A}}(1^\lambda)$ except that we set $c_3 = \mathbf{c}_1^\top \mathbf{r_p} + \lfloor q/2 \rfloor \mu$.

$\mathbf{Hyb}_1^{\mathcal{A}}(1^\lambda)$ *and* $\mathbf{Hyb}_2^{\mathcal{A}}(1^\lambda)$ *are identically distributed as* $\mathbf{p} = \mathbf{B}\mathbf{r_p}$.

$\mathbf{Hyb}_3^{\mathcal{A}}(1^\lambda)$ This is the same as $\mathbf{Hyb}_2^{\mathcal{A}}(1^\lambda)$ except that we sample $\mathbf{c}_1^\top \leftarrow \mathbb{Z}_q^{1 \times 4m^5}$.

*If the $2m^2$-decomposed LWE assumption holds, it is easy to see that* $\mathbf{Hyb}_2^{\mathcal{A}}(1^\lambda)$ *and* $\mathbf{Hyb}_3^{\mathcal{A}}(1^\lambda)$ *are computationally indistinguishable.*

$\mathbf{Hyb}_4^{\mathcal{A}}(1^\lambda)$**.** This is the same $\mathbf{Hyb}_3^{\mathcal{A}}(1^\lambda)$ except that we sample $\mathbf{p} \leftarrow \mathbb{Z}_q^n$ and $c_3 \leftarrow \mathbb{Z}_q$. Note that in hybrid, the advantage of any adversary is 0.

$\mathbf{Hyb}_3^{\mathcal{A}}(1^\lambda)$ *and* $\mathbf{Hyb}_4^{\mathcal{A}}(1^\lambda)$ *are statistically close by leftover hash lemma (Lemma B.3) as* $[\mathbf{B}^\top \mid \mathbf{c}_1]^\top \mathbf{r_p} \approx_s [\mathbf{p}^\top \mid c_3]$. □

**Corollary 5.4.** Assuming the hardness of the $2m^2$-decomposed LWE assumption (Assumption 3.5), there exists a statically secure DPE scheme for DNFs (Definition 4.2) in the random oracle model.

*Proof.* By Theorem 5.2 and Theorem 5.3, Construction 5.1 is a static EqDPE scheme (Definition 4.3, specialized to the static setting as in the remark following Theorem 4.5). Applying Theorem 4.5 in that specialization yields a statically secure DPE scheme for DNFs in the random oracle model. □

# 6 Static Silent Threshold Encryption from Decomposed LWE

In this section, we provide the construction of a statically secure silent threshold encryption for $\tau$-out-of-$N$ threshold policies. In Section 6.1, we recall the notion of programmable $(\alpha, \beta)$-LSSS as defined in [AGY26] which we will use for our construction. In Section 6.2, we construct a scheme with large public and secret keys where $|\mathsf{pk}| = |\mathsf{sk}| = O_{\lambda,\tau}(N)$, i.e., public keys grow as long as the bound on the number of users supported by encryption (ignoring the bound implicit in modulus). Later in Section 6.6, we show a way to remove this dependence.

**Threshold Policy Notation.** We take the bound $N$ which is the number of users encryption in our scheme supports during $\mathsf{GSetup}$. This restriction can be removed using similar techniques in Section 5 (cf. Section 6.6).

## 6.1 Preliminary: Programmable $(\alpha, \beta)$-Scaled Linear Secret Sharing Schemes

We recall the notion of a programmable $(\alpha, \beta)$-Scaled Linear Secret Sharing Scheme (Scaled LSSS) from [AGY26]. Unlike a standard LSSS, where an authorized set reconstructs the secret exactly, a Scaled LSSS only guarantees reconstruction of a *scaled* secret $\gamma_S \cdot k$, where the reconstruction coefficients $\delta_{S,i}$ and the set-dependent scaling factor $\gamma_S$ are both small over the integers. This relaxation preserves the natural (unmodified) form of the shares, and in particular the uniform distribution of unauthorized shares, which is what enables its programmability. We specialize the syntax for threshold policies.

**Syntax.** A programmable $(\alpha, \beta)$-Scaled LSSS for threshold access policies consists of the following polynomial-time algorithms.

$\mathsf{ProgShare}_1(1^N, 1^\tau, T, (\widetilde{\mathbf{sh}}_j)_{j\in T}) \to (k, \mathbf{sh}_1, \ldots, \mathbf{sh}_N)$**.** The deterministic programmed sharing algorithm in Mode 1 takes a set $T \subseteq [N]$, $|T| = \tau$, programmed share values $(\widetilde{\mathbf{sh}}_j)_{j\in T}$, and outputs a secret $k \in \mathbb{Z}_q$ and party shares $(\mathbf{sh}_1, \ldots, \mathbf{sh}_N)$.

$\mathsf{ProgShare}_2(1^N, 1^\tau, C, (\widetilde{\mathbf{sh}}_i)_{i\in C}, k) \to (k, \mathbf{sh}_1, \ldots, \mathbf{sh}_N)$**.** The possibly randomized programmed sharing algorithm in Mode 2 takes an invalid party set $C \subseteq [N]$, $|C| = \tau - 1$, programmed shares $(\widetilde{\mathbf{sh}}_i)_{i\in C}$, and a secret $k \in \mathbb{Z}_q$. It outputs the same secret $k$ and party shares $(\mathbf{sh}_1, \ldots, \mathbf{sh}_N)$.

$\mathsf{GetCoeffs}(1^N, 1^\tau, S) \to \big((\delta_{S,i})_{i\in S}, \gamma_S\big)$**.** The deterministic coefficient-generation algorithm takes an authorized set $S \subseteq [N]$ and outputs scaled reconstruction coefficients $\delta_{S,i} \in \mathbb{Z}$ and a nonzero scaling factor $\gamma_S \in \mathbb{Z}$.

$\mathsf{Recon}(1^N, 1^\tau, S, (\mathbf{sh}_i)_{i\in S}) \to v$**.** The deterministic scaled reconstruction algorithm computes $\big((\delta_{S,i})_{i\in S}, \gamma_S\big) = \mathsf{GetCoeffs}(\mathbb{A}, S)$ and outputs $v := \sum_{i\in S} \delta_{S,i} \cdot \mathbf{sh}_i \in \mathbb{Z}_q$.

**Definition 6.1** (Programmable $(\alpha, \beta)$-Scaled LSSS)**.** Let $\alpha = \alpha(N, \tau, q)$ and $\beta = \beta(N, \tau, q)$. A tuple $(\mathsf{GetCoeffs}, \mathsf{Recon}, \mathsf{ProgShare}_1, \mathsf{ProgShare}_2)$ is a programmable $(\alpha, \beta)$-Scaled LSSS for $\mathbb{A}$ if it satisfies the following properties.

**Correctness (Scaled Reconstruction).** For every authorized set $S \subseteq [N]$, $|S| = \tau$, every secret $k \in \mathbb{Z}_q$, and every $(\mathbf{sh}_1, \ldots, \mathbf{sh}_N)$, letting $\big((\delta_{S,i})_{i\in S}, \gamma_S\big) = \mathsf{GetCoeffs}(1^N, 1^\tau, S)$,

$$\mathsf{Recon}(1^N, 1^\tau, S, (\mathbf{sh}_i)_{i\in S}) = \sum_{i\in S} \langle \delta_{S,i}, \mathbf{sh}_i \rangle = \gamma_S \cdot k.$$

**Compactness ($(\alpha, \beta)$-Bounded Coefficients).** For every authorized set $S \subseteq [N], |S| = \tau$, the coefficients output by $\mathsf{GetCoeffs}(1^N, 1^\tau, S)$ satisfy

$$\|\delta_{S,i}\|_\infty < \alpha \quad \forall\, i \in S, \qquad 0 < |\gamma_S| < \beta.$$

**Programmability.** The scheme additionally satisfies:

**Consistency.** For every set $T \subseteq [N], |T| \geq \tau$, every maximally invalid share set $T \subseteq [L^*]$, every set $C \subseteq [N], |C| = \tau - 1$, every $(\widetilde{\mathbf{sh}}_j)_{j\in T}$, every $(\widetilde{\mathbf{sh}}_i)_{i\in C}$, and every $k \in \mathbb{Z}_q$, if we let

$$(k_0, \mathbf{sh}_{0,1}, \ldots, \mathbf{sh}_{0,N}) = \mathsf{ProgShare}_1(1^N, 1^\tau, T, (\widetilde{Sh}_j)_{j\in T}),$$
$$(k_1, \mathbf{sh}_{1,1}, \ldots, \mathbf{sh}_{1,N}) \leftarrow \mathsf{ProgShare}_2(1^N, 1^\tau, C, (\widetilde{\mathbf{sh}}_i)_{i\in C}, k),$$

then for both $b \in \{0, 1\}$ the output shares reconstruct the output secret, i.e. for $A \subseteq [N], |A| \geq \tau$, $\sum_{i\in A} \langle \delta_{A,i}, \mathbf{sh}_{b,i} \rangle = \gamma_A \cdot k_b$. Moreover the outputs are consistent with the programmed inputs. For Mode 1, if $j \in T$, $\mathbf{sh}_{1,j} = \widetilde{\mathbf{sh}}_j$. For Mode 2, $k_1 = k$ and $\mathbf{sh}_{1,i} = \widetilde{\mathbf{sh}}_i$ for every $i \in C$.

**Mode Indistinguishability.** For every set $T \subseteq [N], |T| \geq \tau$, every set $C \subseteq [N], |C| = \tau - 1$, the following two distributions are identical:

$$\left\{ (k_0, \mathbf{sh}_{0,1}, \ldots, \mathbf{sh}_{0,N}) \ : \ \begin{array}{l} (\widetilde{\mathbf{sh}}_j)_{j\in T} \leftarrow \mathbb{Z}_q, \widetilde{w}_{j^*} \leftarrow \mathbb{Z}_q, \\ (k_0, \ldots) = \mathsf{ProgShare}_1(1^N, 1^\tau, T, (\widetilde{\mathbf{sh}}_j)_{j\in T}) \end{array} \right\}$$
$$\equiv \left\{ (k_1, \mathbf{sh}_{1,1}, \ldots, \mathbf{sh}_{1,N}) \ : \ \begin{array}{l} (\widetilde{\mathbf{sh}}_i)_{i\in C} \leftarrow \mathbb{Z}_q^{|C|}, k \leftarrow \mathbb{Z}_q, \\ (k_1, \ldots) \leftarrow \mathsf{ProgShare}_2(1^N, 1^\tau, C, (\widetilde{\mathbf{sh}}_i)_{i\in C}, k) \end{array} \right\}.$$

**Theorem 6.2** (Scaled LSSS for Threshold Policies [AGY26])**.** Let $N \in \mathbb{N}$, $\tau \in [N]$, and let $q$ be a prime with $q > N$. For the $\tau$-out-of-$N$ threshold access structures, there exist:

1. a programmable $(N^{\tau^2/2}, N^{\tau^2/2})$-Scaled LSSS, and
2. a programmable $(N^{N-\tau}, N^{N-\tau})$-Scaled LSSS.

In particular, taking the better of the two, there is a programmable $(\alpha, \beta)$-Scaled LSSS for threshold access policies with $\alpha = \beta = N^{\min(\tau^2/2,\, N-\tau)}$, and hence $\log \alpha, \log \beta \leq \min(\tau^2, N - \tau) \cdot \log N$.

## 6.2 STE with Large Public Keys

**Building Blocks.** Our construction uses the following primitives.

- An equivocal matrix commitment scheme $\mathsf{EqMCOM}$ (Definition 3.9).
- A simulation-sound extractable NIZK (Definition 3.7) for the language $\mathcal{L} = \{x := (\mathbf{B}, \mathbf{c}_1^\top, \chi, p, \mathsf{ct}') \ : \ \exists\ w := (\mathbf{s}, \mathbf{e}), \mathbf{c}_1^\top = \mathbf{s}^\top \mathbf{B} + p\mathbf{e}^\top \ \wedge \ \|\mathbf{e}\| \leq \sqrt{m}\chi\}$.
- Hash function $\mathcal{H}_{\mathsf{sh}} : \{0,1\}^\lambda \times \{0,1\}^\lambda \to \mathbb{Z}_q^n$ which will be modeled as a random oracle in security proof.
- Hash function $\mathcal{H}_\Pi : \{0,1\}^\lambda \times \{0,1\}^\lambda \to \{0,1\}$ which will be modeled as a random oracle in security proof. This is used for sampling a random permutation of $[N]$ using Fisher-Yates shuffling which is explainable. We show this in Appendix A.
- An explainable shuffler $(\mathsf{Shuffle.Sample}, \mathsf{Shuffle.Explain})$ (Definition A.1).
- A programmable $(\alpha, \beta)$-scaled LSSS (Definition 6.1).

**Parameters.** We provide the LWE parameters used in our construction below.

- $n = \mathsf{poly}(\lambda), \chi = \mathsf{poly}(\lambda)$ (Decomposed LWE Security)
- $m = \Omega(n \log q)$ (Leftover hash lemma)
- $\sigma = \mathsf{poly}(\lambda, m)$ (Decomposed LWE assumption)
- $\widetilde{\chi} > \lambda^{\omega(1)} \cdot m^{11/2} \cdot \chi$ (Smudging)
- Odd primes $p > 3N$, $q > 4pO(N \cdot \alpha \cdot \beta \cdot \chi \cdot \widetilde{\chi} \cdot \sigma \cdot \widetilde{\sigma} \cdot m^6 \cdot \log q)$ (Correctness)

**Construction 6.3.** We provide the construction of a silent threshold encryption scheme using the above building blocks as follows.

$\mathsf{GSetup}(1^\lambda, 1^N)$**.** Sample $(\mathsf{pp}, \mathbf{W}, \mathbf{D}, \mathbf{B}) \leftarrow \mathsf{EqMCOM.Setup}(1^n, 1^m, q, \sigma, 1)$. Sample $\mathsf{nizk.crs} \leftarrow \mathsf{NIZK.Setup}(1^\lambda)$. Sample hash functions $\mathcal{H}_{\mathsf{sh}} : \{0,1\}^\lambda \times \{0,1\}^\lambda \to \mathbb{Z}_q^n$, $\mathcal{H}_\Pi : \{0,1\}^\lambda \times \{0,1\}^\lambda \to \{0,1\}$.

Output $\mathsf{crs} := (N, \mathbf{W}, \mathbf{D}, \mathbf{B}, \mathsf{nizk.crs}, \mathcal{H}_{\mathsf{sh}}, \mathcal{H}_\Pi)$.

$\mathsf{KGen}(\mathsf{crs})$**.** Sample $\mathbf{R} \leftarrow \{\pm 1\}^{4m^5 \times N}$ and compute $\mathbf{T} = \mathbf{BR}$. Output $(\mathsf{pk}, \mathsf{sk}) := (\mathbf{T}, \mathbf{R})$.

$\mathsf{PreProcess}(1^\tau, (\mathsf{pk}_1, \ldots, \mathsf{pk}_N))$**.** Parse each $\mathsf{pk}_i$ as $\mathbf{T}_i \in \mathbb{Z}_q^{n \times N}$ for each $i \in [N]$ and $\tau \leq N$ as the threshold. Do the following.

1. Sample $\mathsf{sd}_\Pi \leftarrow \{0,1\}^\lambda, \mathsf{sd}_{\mathsf{sh}} \leftarrow \{0,1\}^\lambda$.

2. Compute $\widetilde{\mathbf{p}}_i = \mathcal{H}_{\mathsf{sh}}(\mathsf{sd}_{\mathsf{sh}}, i)$ for each $i \in [\tau]$ and $(\mathbf{p}, \widehat{\mathbf{p}}_1, \ldots, \widehat{\mathbf{p}}_N) = \mathsf{ProgShare}_1(1^N, 1^\tau, (\widetilde{\mathbf{p}}_i)_{i\in[\tau]})$.
3. Let $\rho = \mathsf{poly}(\lambda, N)$ be the length of random coins used by $\mathsf{Shuffle.Sample}$ for random shuffle of $[N]$. Compute $r_i = \mathcal{H}_\Pi(\mathsf{sd}_\Pi, i)$ for $i \in [\rho]$ and set $r = (r_1, \ldots, r_\rho)$. Compute $\Pi : [N] \to [N]$ where $\Pi = \mathsf{Shuffle.Sample}(1^\lambda; r)$.
4. Construct matrix $\mathbf{M}_{\mathsf{pk}} := [\mathbf{t}_{1,\Pi(1)} \mid \ldots \mid \mathbf{t}_{N,\Pi(N)}]$ where $\mathbf{t}_{i,j}$ is the $j$-th column of $\mathbf{T}_i$.
5. Construct matrix $\mathbf{M}_{\mathsf{sh}} := [\widehat{\mathbf{p}}_{\Pi(1)} \mid \ldots \mid \widehat{\mathbf{p}}_{\Pi(N)}]$.
6. Let $\mathbf{M} = \mathbf{M}_{\mathsf{pk}} + \mathbf{M}_{\mathsf{sh}}$.
7. Sample $(\mathbf{C}, \mathsf{aux}) \leftarrow \mathsf{EqMCOM.Com}(\mathbf{M})$.

Output $\mathsf{mpk} := \mathbf{C}$, $\mathsf{coin} := (\mathsf{aux}, \mathsf{sd}_{\mathsf{sh}}, \mathsf{sd}_\Pi)$.

$\mathsf{Enc}(\mathsf{mpk}, \mathsf{coin}, \mu)$**.** Parse $\mathsf{mpk}$ as $\mathbf{C} \in \mathbb{Z}_q^{n\times m}$ and $\mathsf{coin}$ as $(\mathsf{aux}, \mathsf{sd}_{\mathsf{sh}}, \mathsf{sd}_\Pi) \in \{0,1\}^{\mathsf{poly}(\lambda)+2\lambda}$. Do the following.

1. Sample $\mathbf{s} \leftarrow \mathbb{Z}_q^n, \mathbf{e} \leftarrow \mathcal{D}_{\mathbb{Z},\chi}^{4m^5}, \mathbf{r}_{\mathbf{p}} \leftarrow \{\pm 1\}^{4m^5}, \widetilde{\mathbf{e}} \leftarrow [-\sqrt{m}\widetilde{\chi}, \sqrt{m}\widetilde{\chi}]^m$.
2. Compute $\mathbf{c}_1^\top = \mathbf{s}^\top\mathbf{B} + p\mathbf{e}^\top, \mathbf{c}_2^\top = \mathbf{s}^\top\mathbf{C} + p\widetilde{\mathbf{e}}^\top$, and $c_3 = \mathbf{s}^\top\mathbf{p} + p\mathbf{e}^\top\mathbf{r}_{\mathbf{p}} + \mu$.
3. Let $\mathsf{ct}' := (\mathbf{c}_2^\top, c_3, \mathsf{aux}, \mathsf{sd}_{\mathsf{sh}}, \mathsf{sd}_\Pi)$. Sample $\mathsf{nizk}.\pi \leftarrow \mathsf{NIZK.Prove}((\mathbf{B}, \mathbf{c}_1^\top, \chi, p, \mathsf{ct}'), (\mathbf{s}, \mathbf{e}))$.

Output $\mathsf{ct} := (\mathbf{c}_1^\top, \mathbf{c}_2^\top, c_3, \mathsf{nizk}.\pi)$.

$\mathsf{PartDec}(\mathsf{ct}, \mathsf{coin}, \mathsf{sk})$**.** Parse $\mathsf{ct}$ as $(\mathbf{c}_1^\top, \mathbf{c}_2^\top, c_3, \mathsf{nizk}.\pi)$, $\mathsf{coin}$ as $(\mathsf{aux}, \mathsf{sd}_{\mathsf{sh}}, \mathsf{sd}_\Pi)$, and $\mathsf{sk}$ as $\mathbf{R} \in \{\pm 1\}^{4m^5\times N}$. If $\mathsf{NIZK.Verify}((\mathbf{B}, \mathbf{c}_1^\top, \chi, p, \mathsf{ct}'), \mathsf{nizk}.\pi) \neq 1$, abort and output $\perp$ where $\mathsf{ct}' := (\mathbf{c}_2^\top, c_3, \mathsf{aux}, \mathsf{sd}_{\mathsf{sh}}, \mathsf{sd}_\Pi)$. Otherwise, sample $\widehat{\mathbf{e}} \leftarrow [-\sqrt{m}\widetilde{\chi}, \sqrt{m}\widetilde{\chi}]^N$ and output $\mathsf{dsh} := \mathbf{c}_1^\top\mathbf{R} + p\widehat{\mathbf{e}}^\top$.

$\mathsf{Recon}(1^\tau, (\mathsf{pk}_1, \ldots, \mathsf{pk}_N), (j, \mathsf{dsh}_j)_{j\in S'}, \mathsf{ct}, \mathsf{coin})$**.** Parse $\mathsf{pk}_i$ as $\mathbf{T}_i \in \mathbb{Z}_q^{n\times N}$ for each $i \in [N]$. If $|S'| < \tau$, abort and output $\perp$. Otherwise, parse $\mathsf{dsh}_j$ as a vector $\mathbf{d}_j \in \mathbb{Z}_q^N$. Consider the lexicographically smallest $\tau$-many shares in $S'$ as $S^*$. Parse $\mathsf{ct}$ as $(\mathbf{c}_1^\top, \mathbf{c}_2^\top, c_3, \mathsf{nizk}.\pi)$ and $\mathsf{coin}$ as $(\mathsf{aux}, \mathsf{sd}_{\mathsf{sh}}, \mathsf{sd}_\Pi)$. Do the following.

1. Using $\mathsf{sd}_{\mathsf{sh}}, \mathsf{sd}_\Pi$, compute $(\mathbf{p}, \widehat{\mathbf{p}}_1, \ldots, \widehat{\mathbf{p}}_N)$ and $\Pi : [N] \to [N]$ like in steps 2 and 3 of $\mathsf{Enc}$.
2. Construct matrix $\mathbf{M}$ like in steps 4-6 of $\mathsf{Enc}$.
3. Compute $\mathbf{V}_N = \mathsf{EqMCOM.Verify}(1^N)$ and $\mathbf{Z} = \mathsf{EqMCOM.Open}(\mathbf{M}, \mathsf{aux})$. Let $\mathbf{v}_j, \mathbf{z}_j$ denote the $j$-th columns of $\mathbf{V}_N, \mathbf{Z}$ respectively.
4. Compute $((\delta_{\Pi(S^*),\Pi(i)})_{i\in S^*}, \gamma_{\Pi(S^*)}) = \mathsf{GetCoeffs}(1^N, 1^\tau, \Pi(S^*))$ where $\Pi(S^*) = \{\Pi(k) \ : \ k \in S^*\}$.
5. Compute $y' = \gamma_{\Pi(S^*)}c_3 - \Big(\sum_{j\in S^*} \delta_{\Pi(S^*),\Pi(j)} \cdot (\mathbf{c}_2^\top\mathbf{v}_j + \mathbf{c}_1^\top\mathbf{z}_j - \mathsf{dsh}_{j,\Pi(j)})\Big)$.
6. Compute $y = y' \mod p$.

Output $(\gamma_{\Pi(S^*)})^{-1} \cdot y$.

**Correctness.** Note that $\mathsf{Recon}$ outputs $\perp$ if $|S'| < \tau$. So, we will only focus on the case when $|S'| \geq \tau$. For each $i \in S$, we have that $\mathsf{pk}_i = \mathbf{T}_i, \mathsf{sk}_i = \mathbf{R}_i$ where $\mathbf{T}_i = \mathbf{B}\mathbf{R}_i$. By correctness of $\mathsf{EqMCOM}$, we have $\mathbf{C}\mathbf{V}_N = \mathbf{M} - \mathbf{B}\mathbf{Z}$ where $(\mathbf{C}, \mathsf{aux}) \leftarrow \mathsf{EqMCOM.Com}(\mathbf{M}), \mathbf{Z} = \mathsf{MCOM.Open}(\mathbf{M}, \mathsf{aux}), \mathbf{V}_N = \mathsf{EqMCOM.Verify}(1^N)$. Here, $\mathbf{M}$ is computed like in steps 2-6 of $\mathsf{Enc}$ using $\mathsf{sd}_\Pi, \mathsf{sd}_{\mathsf{sh}}$ provided by $\mathsf{Enc}$. Thus,

$$\begin{aligned}\mathbf{c}_2^\top\mathbf{v}_j + \mathbf{c}_1^\top\mathbf{z}_j - \mathsf{dsh}_{j,\Pi(j)} &= \mathbf{s}^\top(\mathbf{C}\mathbf{v}_j + \mathbf{B}\mathbf{z}_j - \mathbf{t}_{j,\Pi(j)}) + p(\widetilde{\mathbf{e}}^\top\mathbf{v}_j + \mathbf{e}^\top\mathbf{z}_j - \mathbf{e}^\top\mathbf{r}_{j,\Pi(j)} - \widehat{e}_{j,\Pi(j)}) \\ &= \mathbf{s}^\top(\widehat{\mathbf{p}}_{\Pi(j)}) + p(\widetilde{\mathbf{e}}^\top\mathbf{v}_j + \mathbf{e}^\top\mathbf{z}_j - \mathbf{e}^\top\mathbf{r}_{j,\Pi(j)} - \widehat{e}_{j,\Pi(j)})\end{aligned}$$

In $\mathsf{Recon}$, as $|S'| \geq \tau, |S^*| = \tau$, we have that $\sum_{j\in S^*} \delta_{\Pi(S^*),\Pi(j)}\widehat{\mathbf{p}}_{\Pi(j)} = \gamma_{\Pi(S^*)}\mathbf{p}$ as $\mathbf{p}$ is the secret of programmable $(\alpha, \beta)$-Scaled LSSS. Thus, $y' = \gamma_{\Pi(S^*)}\mu + p\big(\gamma_{\Pi(S^*)}\mathbf{e}^\top\mathbf{r}_{\mathbf{p}} - \sum_{j\in S^*} \delta_{\Pi(S^*),\Pi(j)}(\widetilde{\mathbf{e}}^\top\mathbf{v}_j + \mathbf{e}^\top\mathbf{z}_j - \mathbf{e}^\top\mathbf{r}_{j,\Pi(j)} - \widehat{e}_{j,\Pi(j)})\big)$.

With high probability, $\|\mathbf{z}_j\| \le O(\sqrt{m}\widetilde{\sigma})$, $\|\mathbf{e}\| \le \sqrt{m}\chi$, $\left\|\widetilde{\mathbf{e}}^\top\right\| = \left\|\widehat{e}_{j,\Pi(j)}\right\| \le \sqrt{m}\widetilde{\chi}$. Moreover, $\left\|\mathbf{r}_{j,\Pi(j)}\right\| = \|\mathbf{r}_\mathbf{p}\| = 1$ and $\|\mathbf{v}_j\| \le O(\sigma m^{9/2}\log q)$. We have error term in $y'$ is bounded by $O(N\alpha\beta\chi\widetilde{\chi}\sigma\widetilde{\sigma}m^6\log q)$

If $q > 4pO(N\alpha\beta\chi\widetilde{\chi}\sigma\widetilde{\sigma}m^6\log q)$, then $y' \mod q = y'$ and thus $y = \gamma_{\Pi(S^*)}\mu \mod p$ and correctness holds.

**Compactness.** Size of ciphertext is $|\mathsf{ct}| + |\mathsf{coin}| = (4m^5 + m + 1)\lceil\log q\rceil + (n + 4m^5)\lceil\log q\rceil + \mathsf{poly}(\lambda) + \mathsf{poly}(\lambda, \log N) + 2\lambda$. Thus, $|\mathsf{ct}| = O(m^5\log q) + \mathsf{poly}(\lambda, \log N) = O(n^5\log^6 q) + \mathsf{poly}(\lambda, \log N)$. Hence, we have that ciphertext remains compact if $\tau = o(N^{1/12})$ or $\tau = N - o(N^{1/6})$.

**Remark 6.4** (Succinct LWE)**.** By further assuming a common reference string, we can use the succinct LWE assumption [Wee24] to get an even succinct $|\mathsf{ct}| + |\mathsf{coin}| = O(m\log q) + \mathsf{poly}(\lambda, \log N)$. This would mean, we get succinct ciphertexts whenever $\tau = o(N^{1/4})$ or $\tau = N - o(N^{1/2})$.

**Remark 6.5** (Optimal STE)**.** If there exists can construct a programmable $(\alpha, \beta)$-scaled LSSS (Definition 6.1) with $|\alpha| = |\beta| = \mathsf{poly}(\lambda, \log N)$, there exists an optimal silent threshold encryption from lattices. However, constructing such a scheme is still an open problem.

## 6.3 Fake Setup, Key Generation and Partial Decryption Simulation, and Equivocation

We now describe our $\mathsf{FakeSetup}, \mathsf{SimKGen}, \mathsf{SimPartDec}$, and $\mathsf{Equivocate}$ algorithms. We use the simulated setup $\mathsf{NIZK.SimSetup}$ (which outputs a simulation trapdoor $\mathsf{nizk.td}$ allowing both proof simulation via $\mathsf{NIZK.SimProve}$ and witness extraction via $\mathsf{NIZK}.\mathcal{E}$), and treat the hash functions $\mathcal{H}_{\mathsf{sh}}, \mathcal{H}_\Pi$ as programmable random oracles, maintained as a dictionary $\mathsf{H}_{\mathsf{sh}}, \mathsf{H}_\Pi$ respectively.

$\mathsf{FakeSetup}(1^\lambda, 1^N, \mu)$**.** Sample $(\mathsf{pp}, \mathbf{W}, \mathbf{D}, \mathbf{B}) \leftarrow \mathsf{EqMCOM.Setup}(1^n, 1^m, q, \sigma, 1)$. Sample $\tau^* \leftarrow [N]$. Sample $(\mathsf{nizk.crs}, \mathsf{nizk.td}) \leftarrow \mathsf{NIZK.SimSetup}(1^\lambda)$.

Sample $\mathbf{p} \leftarrow \mathbb{Z}_q^n$ and random $C \subseteq [N], |C| = \tau^* - 1$. Sample $\widehat{\mathbf{y}}_i \leftarrow \mathcal{D}_{\mathbb{Z},\widetilde{\sigma}}^{4m^5}$ and each $\widetilde{\mathbf{p}}_i = \mathbf{B}\widehat{\mathbf{y}}_i$ for $i \in C$. Sample $(\mathbf{p}, \widehat{\mathbf{p}}_1, \ldots, \widehat{\mathbf{p}}_N) \leftarrow \mathsf{ProgShare}_2(1^n, 1^{\tau^*}, (\widetilde{\mathbf{p}}_i)_{i\in C}, \mathbf{p})$.

Sample $\mathbf{s} \leftarrow \mathbb{Z}_q^n, \mathbf{e} \leftarrow \mathcal{D}_{\mathbb{Z},\chi}^{4m^5}, \mathbf{r}_\mathbf{p} \leftarrow \{\pm 1\}^{4m^5}, \mathbf{R}_\mathbf{C} \leftarrow \{\pm 1\}^{4m^5\times m}$. Compute $\mathbf{c}_1^\top := \mathbf{s}^\top\mathbf{B} + p\mathbf{e}^\top, c_3 = \mathbf{s}^\top\mathbf{p} + p\mathbf{e}^\top\mathbf{r}_\mathbf{p} + \mu$.

Output $\widetilde{\mathsf{crs}} := (\mathsf{pp}, \mathbf{W}, \mathbf{D}, \mathbf{B}, \mathsf{nizk.crs}, \mathcal{H}_{\mathsf{sh}}, \mathcal{H}_\Pi), \widetilde{\mathsf{ct}} := (\mathbf{c}_1^\top, c_3), \mathsf{aux} := (\widetilde{\mathsf{crs}}, \widetilde{\mathsf{ct}}, C, (\widehat{\mathbf{y}}_i)_{i\in C}, (\widehat{\mathbf{p}}_1, \ldots, \widehat{\mathbf{p}}_N), \mathbf{R}_\mathbf{C}, \mathsf{nizk.td})$. Here, $\mathcal{H}_{\mathsf{sh}}, \mathcal{H}_\Pi$ are handles for random oracle queries.

$\mathsf{SimKGen}(\mathsf{aux}, \mathsf{st})$**.** Parse $\mathsf{st}$ as $(\mathsf{ctr}, \mathcal{K})$. Sample $\widetilde{\mathbf{Y}} \leftarrow \mathcal{D}_{\mathbb{Z},\widetilde{\sigma}}^{4m^5\times N}$ and compute $\widetilde{\mathbf{T}} = \mathbf{B}\widetilde{\mathbf{Y}}$. Compute $\mathbf{T} = \widetilde{\mathbf{T}} - [\widehat{\mathbf{p}}_1 \mid \ldots \widehat{\mathbf{p}}_N]$ where $(\widehat{\mathbf{p}}_1, \ldots, \widehat{\mathbf{p}}_N)$ are retrieved from $\mathsf{aux}$.

Set $\mathsf{pk} := \mathbf{T}, \mathsf{st}' := (\mathsf{ctr} + 1, \mathcal{K} \cup \{(\mathsf{ctr}, \mathbf{T}, \widetilde{\mathbf{Y}})\})$. Output $(\mathsf{pk}, \mathsf{st}')$.

$\mathsf{SimPartDec}(\mathsf{aux}, \mathsf{st}, \mathsf{ct}, \mathsf{coin}, h)$**.** Parse $\mathsf{ct}$ as $(\mathbf{c}_1^\top, \mathbf{c}_2^\top, c_3, \mathsf{nizk}.\pi)$ and $\mathsf{coin}$ as $(\mathsf{aux}, \mathsf{sd}_{\mathsf{sh}}, \mathsf{sd}_\Pi)$. If $\mathsf{NIZK.Verify}((\mathbf{B}, \mathbf{c}_1^\top, \chi, p, (\mathbf{c}_1^\top, c_3, \mathsf{aux}, \mathsf{sd}_{\mathsf{sh}}, \mathsf{sd}_\Pi)), \mathsf{nizk}.\pi) \neq 1$, output $\perp$. Otherwise, sample $(\mathbf{s}', \mathbf{e}') \leftarrow \mathsf{NIZK}.\mathcal{E}(\mathsf{nizk.td}, (\mathbf{B}, \mathbf{c}_1^\top, \chi, p, (\mathbf{c}_1^\top, c_3, \mathsf{aux}, \mathsf{sd}_{\mathsf{sh}}, \mathsf{sd}_\Pi)), \mathsf{nizk}.\pi)$. Then, sample $\widehat{\mathbf{e}} \leftarrow [-\sqrt{m}\widetilde{\chi}, \sqrt{m}\widetilde{\chi}]^N$ and output $\mathsf{dsh} := (\mathbf{s}')^\top\mathbf{T}_h + p\widehat{\mathbf{e}}^\top$.

$\mathsf{Equivocate}^{\mathsf{H}_{\mathsf{sh}}(\cdot),\mathsf{H}_\Pi(\cdot)}(\mathsf{aux}, \mathsf{st}, 1^\tau, (H_1, \ldots, H_N))$**.** Parse $\mathsf{st}$ as $(\mathsf{ctr}, \mathcal{K})$ and $\mathsf{aux}$ as $(\widetilde{\mathsf{crs}}, \widetilde{\mathsf{ct}}, \mathbf{R}_\mathbf{C}, \mathsf{nizk.td})$. For each $i \in [N]$, parse $H_i$ as honest $(\mathsf{ctr}_i, \perp, \perp)$ or malicious $(\perp, \mathsf{pk}_i, \mathsf{sk}_i)$. If there is no entry for honest $\mathsf{ctr}_i$ in $\mathcal{K}$, abort. If for corrupted $(\perp, \mathsf{pk}_i, \mathsf{sk}_i) = (\perp, \mathbf{T}_i, \mathbf{R}_i) \in \{\perp\} \times \mathbb{Z}_q^{n\times N} \times \{\pm 1\}^{4m^5\times N}$, $\mathbf{T}_i \neq \mathbf{B}\mathbf{R}_i$, abort. Otherwise, for each honest $i$, retrieve $\widetilde{\mathbf{Y}}_i$ from $\mathcal{K}$. Do the following.

1. Sample $\mathsf{sd}_{\mathsf{sh}}, \mathsf{sd}_\Pi$. If there is any entry for $(\mathsf{sd}_{\mathsf{sh}}, \iota)$ for any $\iota \in \{0,1\}^\lambda$ in $\mathsf{H}_{\mathsf{sh}}$ or any entry for $(\mathsf{sd}_\Pi, \iota)$ for any $\iota \in \{0,1\}^\lambda$ in $\mathsf{H}_\Pi$, abort.
2. Otherwise, construct permutation $\Pi : [N] \to [N]$ where for each malicious $i$, $\Pi(i) \in C$ (one-to-one mapping) and for the remaining honest $i$, randomly assign $[N] \setminus C$ (one-to-one mapping).

3. Compute $r' \in \{0,1\}^\rho, r' \leftarrow \mathsf{Shuffle.Explain}(1^N, \Pi)$. For each $j \in [\rho]$, set $\mathsf{H}_\Pi[(\mathsf{sd}_\Pi, j)] := r'_j$.
4. For each $j \in [\tau]$, set $\mathsf{H}_{\mathsf{sh}}[(\mathsf{sd}_{\mathsf{sh}}, j)] := \widehat{\mathbf{p}}_j$.
5. Compute matrix $\mathbf{Y} = [\mathbf{y}_1 \mid \ldots \mid \mathbf{y}_N]$ where $\mathbf{y}_i = \widetilde{\mathbf{y}}_{i,\Pi(i)}$ if $i$ is honest and $\mathbf{y}_i = \mathbf{r}_{i,\Pi(i)} + \widehat{\mathbf{y}}_{\Pi(i)}$ if $i$ is malicious.
6. Sample $\mathsf{eqcom.aux} \leftarrow \mathsf{EqMCOM.EqCom}(\mathbf{Y}, \mathbf{R}_\mathbf{C})$. Sample $\widetilde{\mathbf{e}} \leftarrow [-\sqrt{m}\widetilde{\chi}, \sqrt{m}\widetilde{\chi}]^m$. Compute $\mathbf{c}_2^\top = \mathbf{c}_1^\top \mathbf{R}_\mathbf{C} + p\widetilde{\mathbf{e}}^\top$.
7. Sample $\mathsf{nizk}.\pi \leftarrow \mathsf{NIZK.SimProve}(\mathsf{nizk.td}, (\mathbf{B}, \mathbf{c}_1^\top, \chi, p, (\mathbf{c}_2^\top, c_3, \mathsf{eqcom.aux}, \mathsf{sd}_{\mathsf{sh}}, \mathsf{sd}_\Pi)))$.

Output $\mathsf{ct} := (\mathbf{c}_1^\top, \mathbf{c}_2^\top, c_3, \mathsf{nizk}.\pi)$, $\mathsf{coin} := (\mathsf{eqcom.aux}, \mathsf{sd}_{\mathsf{sh}}, \mathsf{sd}_\Pi)$, and $\mathsf{st}' := \mathsf{st}$.

## 6.4 Mode Indistinguishability

**Theorem 6.6.** If EqMCOM satisfies commitment indistinguishability (Definition 3.9) and NIZK is a simulation sound extractable NIZK for $\mathcal{L}$, then Construction 6.3 satisfies mode indistinguishability property in the random oracle model.

*Proof.* We prove this using the following hybrids.

**$\mathbf{Hyb}_0^\mathcal{A}(1^\lambda)$.** This is the mode indistinguishability experiment with $b = 0$.

- **Setup Phase.** $\mathcal{A}$ sends $1^N, \mu$. Sample $(\mathsf{pp}, \mathbf{W}, \mathbf{D}, \mathbf{B}) \leftarrow \mathsf{EqMCOM.Setup}(1^n, 1^m, q, \sigma, 1)$. Sample $\mathsf{nizk.crs} \leftarrow \mathsf{NIZK.Setup}(1^\lambda)$.

  Send $\mathsf{crs} := (N, \mathbf{W}, \mathbf{D}, \mathbf{B}, \mathsf{nizk.crs}, \mathcal{H}_{\mathsf{sh}}, \mathcal{H}_\Pi)$ to $\mathcal{A}$. Here $\mathcal{H}_{\mathsf{sh}}, \mathcal{H}_\Pi$ are handles for random oracle queries.

  Initiate empty dictionaries $\mathsf{H}_{\mathsf{sh}}, \mathsf{H}_\Pi, \mathcal{K}$, and counter $\mathsf{ctr} := 1$.

- **Random Oracle Queries.** If the query is for $\mathcal{H}_{\mathsf{sh}}$, given $(\mathsf{sd}, i) \in \{0,1\}^\lambda \times \{0,1\}^\lambda$, if there exists an entry for $(\mathsf{sd}, i)$ in $\mathsf{H}_{\mathsf{sh}}$, retrieve and return $\mathsf{H}[(\mathsf{sd}, i)]$. Otherwise, sample $\widetilde{\mathbf{p}} \leftarrow \mathbb{Z}_q^n$ and set $\mathsf{H}[(\mathsf{sd}, i)] := \widetilde{\mathbf{p}}$ and return $\widetilde{\mathbf{p}}$.

  If the query is for $\mathcal{H}_\Pi$, given $(\mathsf{sd}, i) \in \{0,1\}^\lambda \times \{0,1\}^\lambda$, if there exists an entry for $(\mathsf{sd}, i)$ in $\mathsf{H}_\Pi$, retrieve and return $\mathsf{H}[(\mathsf{sd}, i)]$. Otherwise, sample $\beta \leftarrow \{0,1\}$ and set $\mathsf{H}[(\mathsf{sd}, i)] := \{0,1\}$ and return $\{0,1\}$.

- **Key Generation Queries.** Sample $\mathbf{R} \leftarrow \{\pm 1\}^{4m^5 \times N}$ and compute $\mathbf{T} = \mathbf{BR}$. Set $(\mathsf{pk}, \mathsf{sk}) = (\mathbf{T}, \mathbf{R})$ and update $\mathcal{K} := \mathcal{K} \cup \{(\mathsf{ctr}, \mathsf{pk}, \mathsf{sk}, 0)\}$. Send $(\mathsf{pk}, \mathsf{ctr})$ to $\mathcal{A}$. Update $\mathsf{ctr} := \mathsf{ctr} + 1$.

- **Partial Decryption Queries.** Given $\mathsf{ct}, \mathsf{coin}, h'$, retrieve $\mathsf{sk}_{h'} = \mathbf{R}_{h'} \in \{\pm 1\}^{4m^5 \times N}$ from $\mathcal{K}$. Parse $\mathsf{ct}$ as $(\mathbf{c}_1^\top, \mathbf{c}_2^\top, c_3, \mathsf{nizk}.\pi)$ and $\mathsf{coin}$ as $(\mathsf{aux}, \mathsf{sd}_{\mathsf{sh}}, \mathsf{sd}_\Pi)$. If $\mathsf{NIZK.Verify}((\mathbf{B}, \mathbf{c}_1^\top, \chi, p, \mathsf{ct}'), \mathsf{nizk}.\pi) \neq 1$, abort and output $\perp$ where $\mathsf{ct}' := (\mathbf{c}_2^\top, c_3, \mathsf{aux}, \mathsf{sd}_{\mathsf{sh}}, \mathsf{sd}_\Pi)$. Otherwise, sample $\widehat{\mathbf{e}} \leftarrow [-\sqrt{m}\widetilde{\chi}, \sqrt{m}\widetilde{\chi}]^N$ and output $\mathsf{dsh}_{h'} := \mathbf{c}_1^\top \mathbf{R}_{h'} + p\widehat{\mathbf{e}}^\top$.

- **Challenge Encryption Query.** $\mathcal{A}$ sends $1^\tau, (H_1, \ldots, H_N)$ such that $\tau \in [N]$. For each $i \in [N]$, parse each $H_i$ as $(\mathsf{ctr}_i, \perp, \perp)$ or $(\perp, \mathsf{pk}_i, \mathsf{sk}_i)$. If there is not entry for $(\mathsf{ctr}_i, \mathsf{pk}_{\mathsf{ctr}_i}, \mathsf{sk}_{\mathsf{ctr}_i}, 0)$ in $\mathcal{K}$, output a random bit. Otherwise, if $(\mathsf{pk}_i, \mathsf{sk}_i) = (\mathbf{T}_i, \mathbf{R}_i) \notin \mathbb{Z}_q^{n \times N} \times \{\pm 1\}^{4m^5 \times N}$ or $\mathbf{T}_i \neq \mathbf{BR}_i$, output a random bit. Otherwise, do the following.

  1. Sample $\mathsf{sd}_\Pi \leftarrow \{0,1\}^\lambda, \mathsf{sd}_{\mathsf{sh}} \leftarrow \{0,1\}^\lambda$.
  2. Compute $\widetilde{\mathbf{p}}_i = \mathcal{H}_{\mathsf{sh}}(\mathsf{sd}_{\mathsf{sh}}, i)$ for each $i \in [\tau]$ and $(\mathbf{p}, \widehat{\mathbf{p}}_1, \ldots, \widehat{\mathbf{p}}_N) = \mathsf{ProgShare}_1(1^N, 1^\tau, (\widetilde{\mathbf{p}}_i)_{i \in [\tau]})$.
  3. Let $\rho = \mathsf{poly}(\lambda, \log N)$ be the length of random coins used by $\mathsf{Shuffle.Sample}$ for random shuffle of $[N]$. Compute $r_i = \mathcal{H}_\Pi(\mathsf{sd}_\Pi, i)$ for $i \in [\rho]$ and set $r = (r_1, \ldots, r_\rho)$. Compute $\Pi : [N] \to [N]$ where $\Pi = \mathsf{Shuffle.Sample}(1^\lambda; r)$.
  4. Construct matrix $\mathbf{M}_{\mathsf{pk}} := [\mathbf{t}_{1,\Pi(1)} \mid \ldots \mid \mathbf{t}_{N,\Pi(N)}]$ where $\mathbf{t}_{i,j}$ is the $j$-th column of $\mathbf{T}_i$.

5. Construct matrix $\mathbf{M}_{\mathsf{sh}} := [\widehat{\mathbf{p}}_{\Pi(1)} \mid \ldots \mid \widehat{\mathbf{p}}_{\Pi(N)}]$.
6. Let $\mathbf{M} = \mathbf{M}_{\mathsf{pk}} + \mathbf{M}_{\mathsf{sh}}$.
7. Sample $(\mathbf{C}, \mathsf{aux}) \leftarrow \mathsf{EqMCOM.Com}(\mathbf{M})$.
8. Sample $\mathbf{s} \leftarrow \mathbb{Z}_q^n, \mathbf{e} \leftarrow \mathcal{D}_{\mathbb{Z},\chi}^{4m^5}, \mathbf{r}_{\mathbf{p}} \leftarrow \{\pm 1\}^{4m^5}, \widetilde{\mathbf{e}} \leftarrow [-\sqrt{m}\widetilde{\chi}, \sqrt{m}\widetilde{\chi}]^m$.
9. Compute $\mathbf{c}_1^\top = \mathbf{s}^\top \mathbf{B} + p\mathbf{e}^\top, \mathbf{c}_2^\top = \mathbf{s}^\top \mathbf{C} + p\widetilde{\mathbf{e}}^\top$, and $c_3 = \mathbf{s}^\top \mathbf{p} + p\mathbf{e}^\top \mathbf{r}_{\mathbf{p}} + \mu$.
10. Let $\mathsf{ct}' := (\mathbf{c}_2^\top, c_3, \mathsf{aux}, \mathsf{sd}_{\mathsf{sh}}, \mathsf{sd}_\Pi)$. Sample $\mathsf{nizk}.\pi \leftarrow \mathsf{NIZK.Prove}((\mathbf{B}, \mathbf{c}_1^\top, \chi, p, \mathsf{ct}'), (\mathbf{s}, \mathbf{e}))$.

Send $\mathsf{ct}^* := (\mathbf{c}_1^\top, \mathbf{c}_2^\top, c_3, \mathsf{nizk}.\pi)$ and $\mathsf{coin} := (\mathsf{aux}, \mathsf{sd}_{\mathsf{sh}}, \mathsf{sd}_\Pi)$ to $\mathcal{A}$.

- **Key Generation and Partial Decryption Queries.** Perform these similar to pre-challenge queries.
- **Guess Phase.** Output whatever $\mathcal{A}$ outputs.

$\mathbf{Hyb}_1^{\mathcal{A}}(1^\lambda)$**.** This is the same as $\mathbf{Hyb}_0^{\mathcal{A}}(1^\lambda)$ except that during challenge encryption phase, we abort if the sampled $\mathsf{sd}_{\mathsf{sh}}$ or $\mathsf{sd}_\Pi$ is such that if there exists an entry for $(\mathsf{sd}_{\mathsf{sh}}, \iota)$ for anu $\iota \in \{0,1\}^\lambda$ in $\mathsf{H}_{\mathsf{sh}}$ or if there exists an entry for $(\mathsf{sd}_\Pi, \iota)$ for anu $\iota \in \{0,1\}^\lambda$ in $\mathsf{H}_\Pi$, we abort and output a random bit.

*As $\mathcal{A}$ is a* PPT *algorithm there exists polynomials $Q_{\mathsf{sh}} = Q_{\mathsf{sh}}(\lambda), Q_\Pi = Q_\Pi(\lambda)$ such that it makes at most $Q_{\mathsf{sh}}, Q_\Pi$ many queries to $\mathcal{H}_{\mathsf{sh}}, \mathcal{H}_\Pi$ respectively. Thus, the statistical distance between $\mathbf{Hyb}_0^{\mathcal{A}}(1^\lambda)$ and $\mathbf{Hyb}_1^{\mathcal{A}}(1^\lambda)$ is at most $(Q_{\mathsf{sh}} + Q_\Pi)/2^\lambda$.*

$\mathbf{Hyb}_2^{\mathcal{A}}(1^\lambda)$**.** This is the same as $\mathbf{Hyb}_1^{\mathcal{A}}(1^\lambda)$ except that during challenge encryption phase, we sample $\widetilde{\mathbf{p}}_1, \ldots, \widetilde{\mathbf{p}}_\tau \leftarrow \mathbb{Z}_q^n$ and set $\mathsf{H}[(\mathsf{sd}_{\mathsf{sh}}, 1)] := \widetilde{\mathbf{p}}_1, \ldots, \mathsf{H}[(\mathsf{sd}_{\mathsf{sh}}, \tau)] := \widetilde{\mathbf{p}}_\tau$. And, we sample $(r_1, \ldots, r_\rho) \leftarrow \{0,1\}^\rho$ and set $\mathsf{H}_\Pi[(\mathsf{sd}_\Pi, 1)] := r_1, \ldots, \mathsf{H}_\Pi[(\mathsf{sd}_\Pi, \rho)] := r_\rho$ in each of the dictionaries.

$\mathbf{Hyb}_1^{\mathcal{A}}(1^\lambda)$ *and* $\mathbf{Hyb}_2^{\mathcal{A}}(1^\lambda)$ *are identically distributed as in both hybrids, we program the random oracles with uniformly random values.*

$\mathbf{Hyb}_3^{\mathcal{A}}(1^\lambda)$**.** This is the same as $\mathbf{Hyb}_2^{\mathcal{A}}(1^\lambda)$ except that during Setup phase, we sample $(\mathsf{nizk.crs}, \mathsf{nizk.td}) \leftarrow \mathsf{NIZK.SimSetup}(1^\lambda)$ and during challenge encryption phase, we sample $\mathsf{nizk}.\pi \leftarrow \mathsf{NIZK.SimProve}(\mathsf{nizk.td}, (\mathbf{B}, \mathbf{c}_1^\top, \chi, p, \mathsf{ct}'))$.

$\mathbf{Hyb}_2^{\mathcal{A}}(1^\lambda)$ *and* $\mathbf{Hyb}_3^{\mathcal{A}}(1^\lambda)$ *are computationally indistinguishable by the computational zero-knowledge property of* NIZK *(Definition 3.7).*

$\mathbf{Hyb}_4^{\mathcal{A}}(1^\lambda)$**.** This is the same as $\mathbf{Hyb}_3^{\mathcal{A}}(1^\lambda)$ except that during partial decryption queries, we compute $(\mathbf{s}', \mathbf{e}') \leftarrow \mathsf{NIZK}.\mathcal{E}(\mathsf{nizk.td}, (\mathbf{B}, \mathbf{c}_1^\top, \chi, p, \mathsf{ct}'), \mathsf{nizk}.\pi)$ for the given $(\mathsf{ct}, h')$. If $\mathbf{c}_1^\top \neq (\mathbf{s}')^\top \mathbf{B} + p(\mathbf{e}')^\top$ or $\|\mathbf{e}'\| > \sqrt{m}\chi$, we abort and output a random bit.

$\mathbf{Hyb}_3^{\mathcal{A}}(1^\lambda)$ *and* $\mathbf{Hyb}_4^{\mathcal{A}}(1^\lambda)$ *are computationally indistinguishable by the simulation-sound extraction property of* NIZK *(Definition 3.7).*

$\mathbf{Hyb}_5^{\mathcal{A}}(1^\lambda)$**.** This is the same as $\mathbf{Hyb}_4^{\mathcal{A}}(1^\lambda)$ except that during partial decryption queries, sample $\widehat{\mathbf{e}} \leftarrow [-\sqrt{m}\widetilde{\chi}, \sqrt{m}\widetilde{\chi}]^N$ and set $\mathsf{dsh}_{h'} := (\mathbf{s}')^\top \mathbf{T}_{h'} + p\widehat{\mathbf{e}}^\top$.

$\mathbf{Hyb}_4^{\mathcal{A}}(1^\lambda)$ *and* $\mathbf{Hyb}_5^{\mathcal{A}}(1^\lambda)$ *are statistically close by the smudging lemma (Lemma 3.3). The difference between these hybrids is that in previous hybrid, we set* $\mathsf{dsh}_{h'} = (\mathbf{s}')^\top \mathbf{B}\mathbf{R}_{h'} + p(\mathbf{e}')^\top \mathbf{R}_{h'} + p\widehat{\mathbf{e}}^\top = (\mathbf{s}')^\top \mathbf{T}_{h'} + p((\mathbf{e}')^\top \mathbf{R}_{h'} + \widehat{\mathbf{e}}^\top)$ *and in this we set* $\mathsf{dsh}_{h'} := (\mathbf{s}')^\top \mathbf{T}_{h'} + p\widehat{\mathbf{e}}^\top$*. As* $\widehat{\mathbf{e}}^\top \approx_s (\mathbf{e}')^\top \mathbf{R}_{h'} + \widehat{\mathbf{e}}^\top$ *by smudging lemma* $(\widetilde{\chi} > \lambda^{\omega(1)} m^{11/2} \chi)$*, the hybrids are statistically close.*

$\mathbf{Hyb}_6^{\mathcal{A}}(1^\lambda)$**.** This is the same as $\mathbf{Hyb}_5^{\mathcal{A}}(1^\lambda)$ except that during challenge encryption phase, we sample a random permutation $\Pi : [N] \to [N]$ and then do $r' \leftarrow \mathsf{Shuffle.Explain}(1^N, \Pi)$ and program this $r'$ in $\mathsf{H}_\Pi$.

$\mathbf{Hyb}_5^{\mathcal{A}}(1^\lambda)$ *and* $\mathbf{Hyb}_6^{\mathcal{A}}(1^\lambda)$ *are statistically close by explainability property of* Shuffle *(Appendix A).*

$\mathbf{Hyb}_7^{\mathcal{A}}(1^\lambda)$. This is the same as $\mathbf{Hyb}_6^{\mathcal{A}}(1^\lambda)$ except that during challenge encryption phase, we sample a random subset $C \subseteq [N], |C| = \tau - 1$ and assign each malicious public key $i$, $\Pi(i) \in C$ (one-to-one mapping).

$\mathbf{Hyb}_6^{\mathcal{A}}(1^\lambda)$ *and* $\mathbf{Hyb}_7^{\mathcal{A}}(1^\lambda)$ *are identically distributed as sampling a random permutation* $\Pi : [N] \to [N]$ *and sampling a smaller permutation over* $C$ *and then figuring out the rest of the permutation over* $[N] \setminus C$ *results in uniformly random permutation.*

$\mathbf{Hyb}_8^{\mathcal{A}}(1^\lambda)$. This is the same $\mathbf{Hyb}_7^{\mathcal{A}}(1^\lambda)$ except that during challenge encryption phase, we sample $\mathbf{p}, (\widetilde{\mathbf{p}}_i)_{i \in C} \leftarrow \mathbb{Z}_q^n$ and compute $(\mathbf{p}, \widehat{\mathbf{p}}_1, \ldots, \widehat{\mathbf{p}}_N) \leftarrow \mathsf{ProgShare}_2(1^N, 1^\tau, (\widetilde{\mathbf{p}}_i)_{i \in C}, \mathbf{p})$. We will then set $\widehat{\mathbf{p}}_1, \ldots, \widehat{\mathbf{p}}_\tau$ in $\mathsf{H}_{\mathsf{sh}}$.

$\mathbf{Hyb}_7^{\mathcal{A}}(1^\lambda)$ *and* $\mathbf{Hyb}_8^{\mathcal{A}}(1^\lambda)$ *are identically distributed by mode indistinguishability and consistency property of programmable* $(\alpha, \beta)$*-LSSS (Definition 6.1).*

$\mathbf{Hyb}_9^{\mathcal{A}}(1^\lambda)$. This is the same $\mathbf{Hyb}_8^{\mathcal{A}}(1^\lambda)$ except that during setup phase, we guess the challenge $\tau \in [N]$, sample $C \subseteq [N], |C| = \tau - 1$, $\mathbf{p}, (\widetilde{\mathbf{p}}_i)_{i \in C} \leftarrow \mathbb{Z}_q^n$ and compute $(\mathbf{p}, \widehat{\mathbf{p}}_1, \ldots, \widehat{\mathbf{p}}_N) \leftarrow \mathsf{ProgShare}_2(1^N, 1^\tau, (\widetilde{\mathbf{p}}_i)_{i \in C}, \mathbf{p})$. We will then use $C$ in challenge encryption to sample $\Pi$ like in $\mathbf{Hyb}_7^{\mathcal{A}}(1^\lambda)$ (assign the malicious entries in $C$ and uniformly random assignment for other). Then, program $\widehat{\mathbf{p}}_1, \ldots, \widehat{\mathbf{p}}_\tau$ in $\mathsf{H}_{\mathsf{sh}}$.

$\mathbf{Hyb}_8^{\mathcal{A}}(1^\lambda)$ *and* $\mathbf{Hyb}_9^{\mathcal{A}}(1^\lambda)$ *are identically distributed as* $C$ *has nothing to do with the malicious set that* $\mathcal{A}$ *provides. We can sample this a-priori. Note that we need to guess* $\tau \in [N]$ *provided by* $\mathcal{A}$ *early in setup phase. This will result in loss of* $1/N$ *in advantage as we have to abort if reduction's guess is wrong.*

$\mathbf{Hyb}_{10}^{\mathcal{A}}(1^\lambda)$. This is the same as $\mathbf{Hyb}_9^{\mathcal{A}}(1^\lambda)$ except that during key generation phase before challenge encryption query is issued, we set $\mathbf{T} = \widetilde{\mathbf{T}} - [\widehat{\mathbf{p}}_1 \mid \ldots \mid \widehat{\mathbf{p}}_N]$ where $\widetilde{\mathbf{T}} \leftarrow \mathbb{Z}_q^{n \times N}$.

$\mathbf{Hyb}_9^{\mathcal{A}}(1^\lambda)$ *and* $\mathbf{Hyb}_{10}^{\mathcal{A}}(1^\lambda)$ *are statistically close by leftover hash lemma (Lemma B.3). In previous hybrid, we set* $\mathbf{T} = \mathbf{BR}$ *and in this one, we set is as* $\mathbf{T} = \widetilde{\mathbf{T}} - [\widehat{\mathbf{p}}_1 \mid \ldots \mid \widehat{\mathbf{p}}_N]$ *whose marginal distribution is uniformly random. Thus, it holds by leftover hash lemma that both hybrids are statistically close.*

$\mathbf{Hyb}_{11}^{\mathcal{A}}(1^\lambda)$. This is the same as $\mathbf{Hyb}_{10}^{\mathcal{A}}(1^\lambda)$ except that during challenge encryption query phase, we set $\mathbf{M}$ as

$$\mathbf{M} = [\mathbf{m}_1 \mid \ldots \mid \mathbf{m}_N], \text{ for } j \in [N], \mathbf{m}_j = \begin{cases} \widetilde{\mathbf{t}}_{j,\Pi(j)} & \text{if } j \text{ is honest} \\ \widehat{\mathbf{p}}_{\Pi(j)} + \mathbf{B}\mathbf{r}_{j,\Pi(j)} & \text{if } j \text{ is malicious} \end{cases}$$

where $\widetilde{\mathbf{t}}_{j,\Pi(j)}$ is the $\Pi(j)$-th column of the honest key $\widetilde{\mathbf{T}}_j$ which is randomly sampled by the challenger and $\mathbf{r}_{j,\Pi(j)}$ is the $\Pi(j)$-th column of $\mathsf{sk}_j$ provided by $\mathcal{A}$.

$\mathbf{Hyb}_{10}^{\mathcal{A}}(1^\lambda)$ *and* $\mathbf{Hyb}_{11}^{\mathcal{A}}(1^\lambda)$ *are identically distributed. This is because for honest keys, for any* $k \in [N]$*, the* $k$*-th column is* $\widetilde{\mathbf{t}}_k - \widehat{\mathbf{p}}_k$*. So, the* $k$*-th column of* $\mathbf{M}$ *for honest* $k$ *in challenge encryption phase is* $\mathbf{t}_{k,\Pi(k)} + \widehat{\mathbf{p}}_{\Pi(k)} = \widetilde{\mathbf{t}}_k$*. Thus, both hybrids are identically distributed.*

$\mathbf{Hyb}_{12}^{\mathcal{A}}(1^\lambda)$. This is the same as $\mathbf{Hyb}_{11}^{\mathcal{A}}(1^\lambda)$ except that we sample $\widetilde{\mathbf{Y}} \leftarrow \mathcal{D}_{\mathbb{Z},\widetilde{\sigma}}^{4m^5 \times N}$ and set $\widetilde{\mathbf{T}} = \mathbf{B}\widetilde{\mathbf{Y}}$ during each key generation phase before challenge encryption and during setup phase, we set $\widetilde{\mathbf{p}}_i = \mathbf{B}\widehat{\mathbf{y}}_i$ where $\widehat{\mathbf{y}}_i \leftarrow \mathcal{D}_{\mathbb{Z},\widetilde{\sigma}}^{4m^5}$ for each $i \in C$.

$\mathbf{Hyb}_{11}^{\mathcal{A}}(1^\lambda)$ *and* $\mathbf{Hyb}_{12}^{\mathcal{A}}(1^\lambda)$ *are statistically close by gaussian pre-image properties Lemma B.6.*

$\mathbf{Hyb}_{13}^{\mathcal{A}}(1^\lambda)$. This is the same as $\mathbf{Hyb}_{12}^{\mathcal{A}}(1^\lambda)$ except that during challenge encryption phase, we sample $\mathbf{R}_{\mathbf{C}} \leftarrow \{\pm 1\}^{4m^5 \times m}$ and set $\mathbf{C} = \mathbf{B}\mathbf{R}_{\mathbf{C}}$ and $\mathsf{aux} \leftarrow \mathsf{EqMCOM.EqCom}(\mathbf{Y}, \mathbf{R}_{\mathbf{C}})$ where,

$$\mathbf{Y} = [\mathbf{y}_1 \mid \ldots \mid \mathbf{y}_N] \text{ for } j \in [N], \mathbf{y}_j = \begin{cases} \widetilde{\mathbf{y}}_{j,\Pi(j)} & \text{if } j \text{ is honest} \\ \widehat{\mathbf{y}}_{\Pi(j)} + \mathbf{r}_{j,\Pi(j)} & \text{if } j \text{ is malicious} \end{cases}$$

For honest parties, $\widetilde{\mathbf{y}}_{j,\Pi(j)}$ can be retrieved from the pre-challenge queries issued.

$\mathbf{Hyb}_{12}^{\mathcal{A}}(1^\lambda)$ *and* $\mathbf{Hyb}_{13}^{\mathcal{A}}(1^\lambda)$ *are computationally indistinguishable by commitment indistinguishability property of* EqMCOM. *Specifically, all the* $\widetilde{\mathbf{T}}$ *and* $(\widehat{\mathbf{p}}_i)_{i\in C}$ *are derived from the challenger of* EqMCOM *commitment indistinguishability game. Then, without any corruptions, we can use the commitment indistinguishability property to argue indistinguishability.*

**$\mathbf{Hyb}_{14}^{\mathcal{A}}(1^\lambda)$.** This is the mode indistinguishability game with $b = 1$.

$\mathbf{Hyb}_{13}^{\mathcal{A}}(1^\lambda)$ *and* $\mathbf{Hyb}_{14}^{\mathcal{A}}(1^\lambda)$ *are statistically indistinguishable by smudging lemma (Lemma 3.3). The difference between these hybrids is that in previous hybrid, we set* $\mathbf{c}_2^\top = \mathbf{s}^\top\mathbf{C} + p\widetilde{\mathbf{e}}^\top = \mathbf{s}^\top\mathbf{B}\mathbf{R}_\mathbf{C} + p\widetilde{\mathbf{e}}^\top$ *and in this we set* $\mathbf{c}_2^\top := \mathbf{c}_1^\top\mathbf{R}_\mathbf{C} + p\widetilde{\mathbf{e}}^\top = \mathbf{s}^\top\mathbf{B}\mathbf{R}_\mathbf{C} + p(\widetilde{\mathbf{e}}^\top + \mathbf{e}^\top\mathbf{R}_\mathbf{C})$. *As* $\widetilde{\mathbf{e}}^\top \approx_s \mathbf{e}^\top\mathbf{R}_\mathbf{C} + \widetilde{\mathbf{e}}^\top$ *by smudging lemma* ($\widetilde{\chi} > \lambda^{\omega(1)}m^{11/2}\chi$)*, the hybrids are statistically close.*

□

## 6.5 Selective Security for Fake Setup

**Theorem 6.7** (Selective Setup for Fake Setup)**.** Assuming the hardness of $2m^2$-decomposed LWE assumption (Assumption 3.5), Construction 6.3 satisfies selective setup for fake setup.

*Proof.* We prove this using the following hybrids.

**$\mathbf{Hyb}_0^{\mathcal{A}}(1^\lambda)$.** This is the selective security for fake setup experiment with $b = 0/1$.

- **Fake Setup.** Sample $(\mathsf{pp}, \mathbf{W}, \mathbf{D}, \mathbf{B}) \leftarrow \mathsf{EqMCOM.Setup}(1^n, 1^m, q, \sigma, 1)$. Sample $\tau^* \leftarrow [N]$. Sample $(\mathsf{nizk.crs}, \mathsf{nizk.td}) \leftarrow \mathsf{NIZK.SimSetup}(1^\lambda)$.

  Sample $\mathbf{p} \leftarrow \mathbb{Z}_q^n$ and random $C \subseteq [N], |C| = \tau^* - 1$. Sample $\widehat{\mathbf{y}}_i \leftarrow \mathcal{D}_{\mathbb{Z},\widetilde{\sigma}}^{4m^5}$ and each $\widetilde{\mathbf{p}}_i = \mathbf{B}\widehat{\mathbf{y}}_i$ for $i \in C$. Sample $(\mathbf{p}, \widehat{\mathbf{p}}_1, \ldots, \widehat{\mathbf{p}}_N) \leftarrow \mathsf{ProgShare}_2(1^n, 1^{\tau^*}, (\widetilde{\mathbf{p}}_i)_{i\in C}, \mathbf{p})$.

  Sample $\mathbf{s} \leftarrow \mathbb{Z}_q^n, \mathbf{e} \leftarrow \mathcal{D}_{\mathbb{Z},\chi}^{4m^5}, \mathbf{r}_\mathbf{p} \leftarrow \{\pm 1\}^{4m^5}, \mathbf{R}_\mathbf{C} \leftarrow \{\pm 1\}^{4m^5\times m}$. Compute $\mathbf{c}_1^\top := \mathbf{s}^\top\mathbf{B} + p\mathbf{e}^\top, c_3 = \mathbf{s}^\top\mathbf{p} + p\mathbf{e}^\top\mathbf{r}_\mathbf{p} + \mu$.

  Send $\widetilde{\mathsf{crs}} := (\mathsf{pp}, \mathbf{W}, \mathbf{D}, \mathbf{B}, \mathsf{nizk.crs}, \mathcal{H}_{\mathsf{sh}}, \mathcal{H}_\Pi), \widetilde{\mathsf{ct}} := (\mathbf{c}_1^\top, c_3), \mathsf{aux} := (\widetilde{\mathsf{crs}}, \widetilde{\mathsf{ct}}, C, (\widehat{\mathbf{y}}_i)_{i\in C}, (\widehat{\mathbf{p}}_1, \ldots, \widehat{\mathbf{p}}_N), \mathbf{R}_\mathbf{C}, \mathsf{nizk.td})$ to $\mathcal{A}$.

**$\mathbf{Hyb}_1^{\mathcal{A}}(1^\lambda)$.** This is the same as $\mathbf{Hyb}_0^{\mathcal{A}}(1^\lambda)$ except that we set $\mathbf{p} = \mathbf{B}\mathbf{r}_\mathbf{p}$.

$\mathbf{Hyb}_0^{\mathcal{A}}(1^\lambda)$ *and* $\mathbf{Hyb}_1^{\mathcal{A}}(1^\lambda)$ *are statistically close by leftover hash lemma (Lemma B.3).*

**$\mathbf{Hyb}_2^{\mathcal{A}}(1^\lambda)$.** This is the same as $\mathbf{Hyb}_1^{\mathcal{A}}(1^\lambda)$ except that we set $c_3 = \mathbf{c}_1^\top\mathbf{r}_\mathbf{p} + \mu$.

$\mathbf{Hyb}_1^{\mathcal{A}}(1^\lambda)$ *and* $\mathbf{Hyb}_2^{\mathcal{A}}(1^\lambda)$ *are identically distributed as* $\mathbf{p} = \mathbf{B}\mathbf{r}_\mathbf{p}$.

**$\mathbf{Hyb}_3^{\mathcal{A}}(1^\lambda)$** This is the same as $\mathbf{Hyb}_2^{\mathcal{A}}(1^\lambda)$ except that we sample $\mathbf{c}_1^\top \leftarrow \mathbb{Z}_q^{1\times 4m^5}$.

*If the* $2m^2$*-decomposed LWE assumption holds, it is easy to see that* $\mathbf{Hyb}_2^{\mathcal{A}}(1^\lambda)$ *and* $\mathbf{Hyb}_3^{\mathcal{A}}(1^\lambda)$ *are computationally indistinguishable.*

**$\mathbf{Hyb}_4^{\mathcal{A}}(1^\lambda)$.** This is the same $\mathbf{Hyb}_3^{\mathcal{A}}(1^\lambda)$ except that we sample $\mathbf{p} \leftarrow \mathbb{Z}_q^n$ and $c_3 \leftarrow \mathbb{Z}_q$. Note that in hybrid, the advantage of any adversary is 0.

$\mathbf{Hyb}_3^{\mathcal{A}}(1^\lambda)$ *and* $\mathbf{Hyb}_4^{\mathcal{A}}(1^\lambda)$ *are statistically close by leftover hash lemma (Lemma B.3) as* $[\mathbf{B}^\top \mid \mathbf{c}_1]^\top\mathbf{r}_\mathbf{p} \approx_s [\mathbf{p}^\top \mid c_3]$.

□

**Corollary 6.8.** Assuming the hardness of $2m^2$-decomposed LWE (Assumption 3.5), there exists a silent threshold encryption scheme for $\tau$-out-of-$N$ threshold policies with sub-linear size ciphertexts for $\tau = o(N^{1/12})$ or $\tau = N - o(N^{1/6})$.

*Proof.* By Theorem 6.6 and Theorem 6.7, Construction 6.3 is a static EqDPE scheme (Definition 4.3, specialized to the static setting as in the remark following Theorem 4.5) for threshold policies. Applying Theorem 4.5 in that specialization yields a statically secure DPE scheme for threshold policies in the random oracle model. □

### 6.6 STE with Short Public Keys and More

In previous section, we constructed an STE scheme with large public and secret keys where $|\mathsf{pk}|$ and $|\mathsf{sk}|$ grows with $N$, where $N$ is a bound on the number of users that the encryption algorithm supports. We provide two natural optimization for this scheme.

**Shortening public and secret keys.** The idea is to rely on a random oracle $\mathcal{H}_{\mathsf{pk}} : \{0,1\}^\lambda \to \mathcal{D}_{\mathbb{Z},\sigma}^{m\times N}$. This will be initiated during setup when the bound $N$ is known. Now, during key generation ($\mathsf{KGen}$), we sample $\mathbf{R} \leftarrow \{\pm 1\}^{4m^5\times m}$, a seed $\mathsf{sd} \leftarrow \{0,1\}^\lambda$ and set our public key to be $(\mathbf{T} := \mathbf{BR}, \mathsf{sd})$ and secret key to be $\mathbf{R}$. Note that now $|\mathsf{pk}|$ and $|\mathsf{sk}|$ do not depend on $N$ other than what is implicit in the modulus. During pre-processing, we can set $\mathbf{t}_{i,\Pi(i)}$ to be the $\Pi(i)$-th column of $\mathbf{T}_i \cdot \mathcal{H}_{\mathsf{pk}}(\mathsf{sd}_i) \in \mathbb{Z}_q^{n\times N}$ which is publicly computable given $\mathsf{sd}_i$. This way, we can argue correctness and compactness. Although this will increase the error bound, it will not affect too much the modulus $q$.

For security proof, not that any key generated by challenger is not corrupted in static security game. Thus, we can set $\mathbf{T} \leftarrow \mathbb{Z}_q^{n\times m}$ for each public key generated by challenger. Then, we can sample $\mathbf{T}$ with a trapdoor $T_{\mathbf{T}}$ and program the random oracle $\mathcal{H}_{\mathsf{pk}}$ with the $\mathsf{sd}$-th entry $\mathbf{U} \in \mathbb{Z}^{m\times N}$ such that $\mathbf{TU} = \widetilde{\mathbf{T}} - [\widehat{\mathbf{p}}_1 \mid \ldots \mid \widehat{\mathbf{p}}_N]$, i.e., $\mathbf{U} \leftarrow \mathbf{T}_\sigma^{-1}(\widetilde{\mathbf{T}} - [\widehat{\mathbf{p}}_1 \mid \ldots \mid \widehat{\mathbf{p}}_N])$. This can be done efficiently as $\mathbf{T}$ is sampled with a trapdoor. We have exact distribution we need in the previous section's security proof too. Additional care needs to be taken to ensure that each $\mathsf{sd}$ sampled during key generation is unique for honest keys which can be done easily. Hence, by relying on a random oracle, we can compress the public/secret keys in the previous construction. It is easy to formally verify the security of this construction which we omit for brevity.

**Removing bound $N$ from $\mathsf{GSetup}$.** In Construction 6.3, we require a bound $N$ to sample large-enough public keys which can help in equivocating any permutation $\Pi$. However, using the technique in Section 5, similar to how we made public/secret keys compact as above, it could be possible to defer the generation of $\mathbf{t}_i$ to $\mathsf{Enc}$ using $\mathcal{H}_{\mathsf{pk}}$ in $\mathsf{Enc}$. This would yield an STE scheme without needing a bound in $\mathsf{GSetup}$.

## References


[ABB10] Shweta Agrawal, Dan Boneh, and Xavier Boyen. Efficient lattice (H)IBE in the standard model. In Henri Gilbert, editor, *Advances in Cryptology – EUROCRYPT 2010*, volume 6110 of *Lecture Notes in Computer Science*, pages 553–572, French Riviera, May 30 – June 3, 2010. Springer Berlin Heidelberg, Germany. 2, 9, 42

[ABSV15] Prabhanjan Ananth, Zvika Brakerski, Gil Segev, and Vinod Vaikuntanathan. From selective to adaptive security in functional encryption. In Rosario Gennaro and Matthew J. B. Robshaw, editors, *Advances in Cryptology – CRYPTO 2015, Part II*, volume 9216 of *Lecture Notes in Computer Science*, pages 657–677, Santa Barbara, CA, USA, August 16–20, 2015. Springer Berlin Heidelberg, Germany. 2

[ADM+24] Gennaro Avitabile, Nico Döttling, Bernardo Magri, Christos Sakkas, and Stella Wohnig. Signature-based witness encryption with compact ciphertext. In Kai-Min Chung and Yu Sasaki, editors, *Advances in Cryptology – ASIACRYPT 2024, Part I*, volume 15484 of *Lecture Notes in Computer Science*, pages 3–31, Kolkata, India, December 9–13, 2024. Springer, Singapore, Singapore. 1, 2, 3, 4

[AGY26] Abtin Afshar, Rishab Goyal, and Saikumar Yadugiri. Silent distributed cryptography for dnfs and threshold policies from lattices, 2026. Cryptology ePrint Archive, Paper 2026/1547. 2, 3, 4, 6, 7, 8, 21, 22, 23

[AMR25] Damiano Abram, Giulio Malavolta, and Lawrence Roy. Key-homomorphic computations for RAM: Fully succinct randomised encodings and more. In Yael Tauman Kalai and Seny F. Kamara, editors, *Advances in Cryptology – CRYPTO 2025, Part III*, volume 16002 of *Lecture Notes in Computer Science*, pages 236–268, Santa Barbara, CA, USA, August 17–21, 2025. Springer, Cham, Switzerland. 3, 5, 10

[AMR26] Damiano Abram, Giulio Malavolta, and Lawrence Roy. How to authenticate a non-deterministic computation. Cryptology ePrint Archive, Paper 2026/741, 2026. 2, 3

[BB04] Dan Boneh and Xavier Boyen. Efficient selective-ID secure identity based encryption without random oracles. In Christian Cachin and Jan Camenisch, editors, *Advances in Cryptology – EUROCRYPT 2004*, volume 3027 of *Lecture Notes in Computer Science*, pages 223–238, Interlaken, Switzerland, May 2–6, 2004. Springer Berlin Heidelberg, Germany. 2

[BCD+25] Pedro Branco, Arka Rai Choudhuri, Nico Döttling, Abhishek Jain, Giulio Malavolta, and Akshayaram Srinivasan. Black-box non-interactive zero knowledge from vector trapdoor hash. In Serge Fehr and Pierre-Alain Fouque, editors, *Advances in Cryptology – EUROCRYPT 2025, Part IV*, volume 15604 of *Lecture Notes in Computer Science*, pages 64–92, Madrid, Spain, May 4–8, 2025. Springer, Cham, Switzerland. 11

[BCF+25] Jan Bormet, Arka Rai Choudhuri, Sebastian Faust, Sanjam Garg, Hussien Othman, Guru-Vamsi Policharla, Ziyan Qu, and Mingyuan Wang. Beast-mev: Batched threshold encryption with silent setup for mev prevention. *Cryptology ePrint Archive*, 2025. 1

[BF01] Dan Boneh and Matt Franklin. Identity-based encryption from the weil pairing. In *Annual international cryptology conference*, pages 213–229. Springer, 2001. 2

[BGW05] Dan Boneh, Craig Gentry, and Brent Waters. Collusion resistant broadcast encryption with short ciphertexts and private keys. In Victor Shoup, editor, *Advances in Cryptology – CRYPTO 2005*, volume 3621 of *Lecture Notes in Computer Science*, pages 258–275, Santa Barbara, CA, USA, August 14–18, 2005. Springer Berlin Heidelberg, Germany. 2

[BP23] LTAN Brandão and Rene Peralta. Nist first call for multi-party threshold schemes. *doi*, 10:6028, 2023. 1

[CHKP10] David Cash, Dennis Hofheinz, Eike Kiltz, and Chris Peikert. Bonsai trees, or how to delegate a lattice basis. In Henri Gilbert, editor, *Advances in Cryptology – EUROCRYPT 2010*, volume 6110 of *Lecture Notes in Computer Science*, pages 523–552, French Riviera, May 30 – June 3, 2010. Springer Berlin Heidelberg, Germany. 2

[CHW25] Jeffrey Champion, Yao-Ching Hsieh, and David J. Wu. Registered ABE and adaptively-secure broadcast encryption from succinct LWE. In Yael Tauman Kalai and Seny F. Kamara, editors, *Advances in Cryptology – CRYPTO 2025, Part III*, volume 16002 of *Lecture Notes in Computer Science*, pages 3–34, Santa Barbara, CA, USA, August 17–21, 2025. Springer, Cham, Switzerland. 2, 3, 37

[CW26a] Jeffrey Champion and David J. Wu. Distributed monotone-policy encryption for DNFs from lattices. In Joan Daemen and Emmanuel Thomé, editors, *Advances in Cryptology – EUROCRYPT 2026, Part V*, volume 16545 of *Lecture Notes in Computer Science*, pages 334–365, Rome, Italy, May 10–14, 2026. Springer, Cham, Switzerland. 2, 3, 4, 6, 7

[CW26b] Jeffrey Champion and David J. Wu. Optimal distributed monotone-policy encryption for DNFs and more from lattices. Cryptology ePrint Archive, Paper 2026/1464, 2026. 3, 4

[DDO+01] Alfredo De Santis, Giovanni Di Crescenzo, Rafail Ostrovsky, Giuseppe Persiano, and Amit Sahai. Robust non-interactive zero knowledge. In Joe Kilian, editor, *Advances in Cryptology – CRYPTO 2001*, volume 2139 of *Lecture Notes in Computer Science*, pages 566–598, Santa Barbara, CA, USA, August 19–23, 2001. Springer Berlin Heidelberg, Germany. 11

[DG17a] Nico Döttling and Sanjam Garg. From selective IBE to full IBE and selective HIBE. In Yael Kalai and Leonid Reyzin, editors, *TCC 2017: 15th Theory of Cryptography Conference, Part I*, volume 10677 of *Lecture Notes in Computer Science*, pages 372–408, Baltimore, MD, USA, November 12–15, 2017. Springer, Cham, Switzerland. 2

[DG17b] Nico Döttling and Sanjam Garg. Identity-based encryption from the Diffie-Hellman assumption. In Jonathan Katz and Hovav Shacham, editors, *Advances in Cryptology – CRYPTO 2017, Part I*, volume 10401 of *Lecture Notes in Computer Science*, pages 537–569, Santa Barbara, CA, USA, August 20–24, 2017. Springer, Cham, Switzerland. 2

[DHMR07] Vanesa Daza, Javier Herranz, Paz Morillo, and Carla Rafols. Cca2-secure threshold broadcast encryption with shorter ciphertexts. In *International Conference on Provable Security*, pages 35–50. Springer, 2007. 1, 3

[DHMR08] Vanesa Daza, Javier Herranz, Paz Morillo, and Carla Ràfols. Ad-hoc threshold broadcast encryption with shorter ciphertexts. *Electronic Notes in Theoretical Computer Science*, 192(2):3–15, 2008. 1, 3

[DJWW26] Lalita Devadas, Abhishek Jain, Brent Waters, and David J. Wu. Succinct witness encryption for batch languages and applications. In Goichiro Hanaoka and Bo-Yin Yang, editors, *Advances in Cryptology – ASIACRYPT 2025*, pages 130–162, Singapore, 2026. Springer Nature Singapore. 1, 2, 3, 4, 13

[DKW21] Pratish Datta, Ilan Komargodski, and Brent Waters. Decentralized multi-authority abe for dnf s from lwe. In *Annual international conference on the theory and applications of cryptographic techniques*, pages 177–209. Springer, 2021. 41

[DRS04] Yevgeniy Dodis, Leonid Reyzin, and Adam Smith. Fuzzy extractors: How to generate strong keys from biometrics and other noisy data. In Christian Cachin and Jan Camenisch, editors, *Advances in Cryptology – EUROCRYPT 2004*, volume 3027 of *Lecture Notes in Computer Science*, pages 523–540, Interlaken, Switzerland, May 2–6, 2004. Springer Berlin Heidelberg, Germany. 9

[Dur64] Richard Durstenfeld. Algorithm 235: Random permutation. *Commun. ACM*, 7(7):420, July 1964. 8, 35

[FN94] Amos Fiat and Moni Naor. Broadcast encryption. In Douglas R. Stinson, editor, *Advances in Cryptology – CRYPTO'93*, volume 773 of *Lecture Notes in Computer Science*, pages 480–491, Santa Barbara, CA, USA, August 22–26, 1994. Springer Berlin Heidelberg, Germany. 2

[FWW23] Cody Freitag, Brent Waters, and David J. Wu. How to use (plain) witness encryption: Registered ABE, flexible broadcast, and more. In Helena Handschuh and Anna Lysyanskaya, editors, *Advances in Cryptology – CRYPTO 2023, Part IV*, volume 14084 of *Lecture Notes in Computer Science*, pages 498–531, Santa Barbara, CA, USA, August 20–24, 2023. Springer, Cham, Switzerland. 5

[GGSW13] Sanjam Garg, Craig Gentry, Amit Sahai, and Brent Waters. Witness encryption and its applications. In Dan Boneh, Tim Roughgarden, and Joan Feigenbaum, editors, *45th Annual ACM Symposium on Theory of Computing*, pages 467–476, Palo Alto, CA, USA, June 1–4, 2013. ACM Press. 3

[GKPW24] Sanjam Garg, Dimitris Kolonelos, Guru-Vamsi Policharla, and Mingyuan Wang. Threshold encryption with silent setup. In Leonid Reyzin and Douglas Stebila, editors, *Advances in Cryptology – CRYPTO 2024, Part VII*, volume 14926 of *Lecture Notes in Computer Science*, pages 352–386, Santa Barbara, CA, USA, August 18–22, 2024. Springer, Cham, Switzerland. 1, 2, 3, 4

[GKR25] Rishab Goyal, Venkata Koppula, and Mahesh Sreekumar Rajasree. A note on adaptive security in hierarchical identity-based encryption. In Yael Tauman Kalai and Seny F. Kamara, editors, *Advances in Cryptology – CRYPTO 2025, Part III*, volume 16002 of *Lecture Notes in Computer Science*, pages 101–136, Santa Barbara, CA, USA, August 17–21, 2025. Springer, Cham, Switzerland. 2

[GPSW06] Vipul Goyal, Omkant Pandey, Amit Sahai, and Brent Waters. Attribute-based encryption for fine-grained access control of encrypted data. In *Proceedings of the 13th ACM conference on Computer and communications security*, pages 89–98, 2006. 2

[GPV08] Craig Gentry, Chris Peikert, and Vinod Vaikuntanathan. Trapdoors for hard lattices and new cryptographic constructions. In Richard E. Ladner and Cynthia Dwork, editors, *40th Annual ACM Symposium on Theory of Computing*, pages 197–206, Victoria, BC, Canada, May 17–20, 2008. ACM Press. 2, 9, 16, 37

[GW09] Craig Gentry and Brent Waters. Adaptive security in broadcast encryption systems (with short ciphertexts). In Antoine Joux, editor, *Advances in Cryptology – EUROCRYPT 2009*, volume 5479 of *Lecture Notes in Computer Science*, pages 171–188, Cologne, Germany, April 26–30, 2009. Springer Berlin Heidelberg, Germany. 2, 3, 5

[GY26a] Rishab Goyal and Saikumar Yadugiri. Adaptively-secure flexible and identity-based broadcast encryption from decomposed LWE. Cryptology ePrint Archive, Paper 2026/862, 2026. 2, 3, 4, 5, 11, 12, 13, 37

[GY26b] Rishab Goyal and Saikumar Yadugiri. Equivocal broadcast encryption: Adaptively-secure optimal distributed broadcast encryption from lattices. To appear at CRYPTO 2026, 2026. 2, 3, 4, 5, 13

[HWW25] Yao-Ching Hsieh, Brent Waters, and David J. Wu. A generic approach to adaptively-secure broadcast encryption in the plain model. In Serge Fehr and Pierre-Alain Fouque, editors, *Advances in Cryptology – EUROCRYPT 2025, Part III*, volume 15603 of *Lecture Notes in Computer Science*, pages 336–365, Madrid, Spain, May 4–8, 2025. Springer, Cham, Switzerland. 3, 5

[ILL89] Russell Impagliazzo, Leonid A. Levin, and Michael Luby. Pseudo-random generation from one-way functions (extended abstracts). In *21st Annual ACM Symposium on Theory of Computing*, pages 12–24, Seattle, WA, USA, May 15–17, 1989. ACM Press. 9

[KMW23] Dimitris Kolonelos, Giulio Malavolta, and Hoeteck Wee. Distributed broadcast encryption from bilinear groups. In Jian Guo and Ron Steinfeld, editors, *Advances in Cryptology – ASIACRYPT 2023, Part V*, volume 14442 of *Lecture Notes in Computer Science*, pages 407–441, Guangzhou, China, December 4–8, 2023. Springer, Singapore, Singapore. 2, 3, 5

[Knu07] D Knut. Semi-numerical algorithms. the art of computer programming. vol. 2, 2007. 8, 35

[LOS+10] Allison B. Lewko, Tatsuaki Okamoto, Amit Sahai, Katsuyuki Takashima, and Brent Waters. Fully secure functional encryption: Attribute-based encryption and (hierarchical) inner product encryption. In Henri Gilbert, editor, *Advances in Cryptology – EUROCRYPT 2010*, volume 6110 of *Lecture Notes in Computer Science*, pages 62–91, French Riviera, May 30 – June 3, 2010. Springer Berlin Heidelberg, Germany. 2

[LW22] George Lu and Brent Waters. How to sample a discrete gaussian (and more) from a random oracle. In Eike Kiltz and Vinod Vaikuntanathan, editors, *TCC 2022: 20th Theory of Cryptography Conference, Part II*, volume 13748 of *Lecture Notes in Computer Science*, pages 653–682, Chicago, IL, USA, November 7–10, 2022. Springer, Cham, Switzerland. 16

[Lyu12] Vadim Lyubashevsky. Lattice signatures without trapdoors. In David Pointcheval and Thomas Johansson, editors, *Advances in Cryptology – EUROCRYPT 2012*, volume 7237 of *Lecture Notes in Computer Science*, pages 738–755, Cambridge, UK, April 15–19, 2012. Springer Berlin Heidelberg, Germany. 9

[MP12] Daniele Micciancio and Chris Peikert. Trapdoors for lattices: Simpler, tighter, faster, smaller. In David Pointcheval and Thomas Johansson, editors, *Advances in Cryptology – EUROCRYPT 2012*, volume 7237 of *Lecture Notes in Computer Science*, pages 700–718, Cambridge, UK, April 15–19, 2012. Springer Berlin Heidelberg, Germany. 9, 37

[PS19] Chris Peikert and Sina Shiehian. Noninteractive zero knowledge for NP from (plain) learning with errors. In Alexandra Boldyreva and Daniele Micciancio, editors, *Advances in Cryptology – CRYPTO 2019, Part I*, volume 11692 of *Lecture Notes in Computer Science*, pages 89–114, Santa Barbara, CA, USA, August 18–22, 2019. Springer, Cham, Switzerland. 11

[Reg05] Oded Regev. On lattices, learning with errors, random linear codes, and cryptography. In Harold N. Gabow and Ronald Fagin, editors, *37th Annual ACM Symposium on Theory of Computing*, pages 84–93, Baltimore, MA, USA, May 22–24, 2005. ACM Press. 10

[RSY21] Leonid Reyzin, Adam Smith, and Sophia Yakoubov. Turning hate into love: compact homomorphic ad hoc threshold encryption for scalable mpc. In *International Symposium on Cyber Security Cryptography and Machine Learning*, pages 361–378. Springer, 2021. 1, 2, 3, 4

[Sah99] Amit Sahai. Non-malleable non-interactive zero knowledge and adaptive chosen-ciphertext security. In *40th Annual Symposium on Foundations of Computer Science*, pages 543–553, New York, NY, USA, October 17–19, 1999. IEEE Computer Society Press. 11

[Wat05] Brent R. Waters. Efficient identity-based encryption without random oracles. In Ronald Cramer, editor, *Advances in Cryptology – EUROCRYPT 2005*, volume 3494 of *Lecture Notes in Computer Science*, pages 114–127, Aarhus, Denmark, May 22–26, 2005. Springer Berlin Heidelberg, Germany. 2

[Wat09] Brent Waters. Dual system encryption: Realizing fully secure IBE and HIBE under simple assumptions. In Shai Halevi, editor, *Advances in Cryptology – CRYPTO 2009*, volume 5677 of *Lecture Notes in Computer Science*, pages 619–636, Santa Barbara, CA, USA, August 16–20, 2009. Springer Berlin Heidelberg, Germany. 2

[Wat11] Brent Waters. Ciphertext-policy attribute-based encryption: An expressive, efficient, and provably secure realization. In *International workshop on public key cryptography*, pages 53–70. Springer, 2011. 4

[Wat15] Brent Waters. A punctured programming approach to adaptively secure functional encryption. In Rosario Gennaro and Matthew J. B. Robshaw, editors, *Advances in Cryptology – CRYPTO 2015, Part II*, volume 9216 of *Lecture Notes in Computer Science*, pages 678–697, Santa Barbara, CA, USA, August 16–20, 2015. Springer Berlin Heidelberg, Germany. 2

[Wat24] Brent Waters. A new approach for non-interactive zero-knowledge from learning with errors. In Bojan Mohar, Igor Shinkar, and Ryan O'Donnell, editors, *56th Annual ACM Symposium on Theory of Computing*, pages 399–410, Vancouver, BC, Canada, June 24–28, 2024. ACM Press. 11

[Wee24] Hoeteck Wee. Circuit ABE with $\mathsf{poly}(\mathsf{depth}, \lambda)$-sized ciphertexts and keys from lattices. In Leonid Reyzin and Douglas Stebila, editors, *Advances in Cryptology – CRYPTO 2024, Part III*, volume 14922 of *Lecture Notes in Computer Science*, pages 178–209, Santa Barbara, CA, USA, August 18–22, 2024. Springer, Cham, Switzerland. 3, 8, 25

[Wee25] Hoeteck Wee. Almost optimal KP and CP-ABE for circuits from succinct LWE. In Serge Fehr and Pierre-Alain Fouque, editors, *Advances in Cryptology – EUROCRYPT 2025, Part III*, volume 15603 of *Lecture Notes in Computer Science*, pages 34–62, Madrid, Spain, May 4–8, 2025. Springer, Cham, Switzerland. 3, 5, 10

[WQZD10] Qianhong Wu, Bo Qin, Lei Zhang, and Josep Domingo-Ferrer. Ad hoc broadcast encryption (poster presentation). In Ehab Al-Shaer, Angelos D. Keromytis, and Vitaly Shmatikov, editors, *ACM CCS 2010: 17th Conference on Computer and Communications Security*, pages 741–743, Chicago, Illinois, USA, October 4–8, 2010. ACM Press. 1

[WW25] Hoeteck Wee and David J. Wu. Unbounded distributed broadcast encryption and registered ABE from succinct LWE. In Yael Tauman Kalai and Seny F. Kamara, editors, *Advances in Cryptology – CRYPTO 2025, Part III*, volume 16002 of *Lecture Notes in Computer Science*, pages 204–235, Santa Barbara, CA, USA, August 17–21, 2025. Springer, Cham, Switzerland. 10, 37

[WW26] Brent Waters and David J. Wu. Silent threshold cryptography from pairings: Expressive policies in the plain model. Cryptology ePrint Archive, Report 2025/1547, To appear in Eurocrypt, 2026. 1, 2, 3, 4

[WWW25] Brent Waters, Hoeteck Wee, and David J. Wu. New techniques for preimage sampling: Improved NIZKs and more from LWE. In Serge Fehr and Pierre-Alain Fouque, editors, *Advances in Cryptology – EUROCRYPT 2025, Part IV*, volume 15604 of *Lecture Notes in Computer Science*, pages 3–33, Madrid, Spain, May 4–8, 2025. Springer, Cham, Switzerland. 11

# A Explainability of Fisher-Yates Shuffling Algorithm

In this section, we show that the Fisher-Yates shuffle [Knu07, Dur64] is statistically explainable.

**Notation.** We write $\mathcal{S}_N$ for the symmetric group on $[N]$ and represent $\pi \in \mathcal{S}_N$ as the array $(\pi[1], \ldots, \pi[N])$. For $\ell \in \mathbb{N}$ we set $\mathcal{R}_{N,\ell} := (\{0,1\}^\ell)^{N-1}$ and parse $r = H_N \| \cdots \| H_2 \in \mathcal{R}_{N,\ell}$, identifying each block $H_i \in \{0,1\}^\ell$ with an integer in $\{0, \ldots, 2^\ell - 1\}$. We write $\mathcal{J}_N := \{1,2\} \times \{1,2,3\} \times \ldots \times \{1, \ldots, N\}$, whose elements $\mathbf{j} = (j_N, \ldots, j_2)$ we call digit vectors. Usually Fisher-Yates works over these digit vectors. Note $|\mathcal{J}_N| = N!$. For a finite set $S$, $\mathcal{U}(S)$ is the uniform distribution on $S$, and $\mathsf{SD}(\cdot,\cdot)$ denotes statistical distance.

**Syntax.** An explainable shuffler has the following polynomial-time algorithms.

$\mathsf{Shuffle}(1^N; r) \to \pi$**.** The randomized shuffle algorithm takes as input $1^N$ which denotes set $[N]$, random coins $r$, and outputs permutation $\pi : [N] \to [N]$.

$\mathsf{Explain}(1^N, \pi) \to r$**.** The randomized shuffling explanation algorithm takes as input $1^N$ which denotes set $[N]$, permutation $\pi : [N] \to [N]$, and outputs random coins $r$.

**Definition A.1** (Explainable Shuffler)**.** Let $\rho = \rho(\lambda, N)$ be a randomness length. An explainable shuffler $(\mathsf{Sample}, \mathsf{Explain})$ satisfies the following properties.

**Sampling distribution:** For any $\lambda \in \mathbb{N}$, $N = N(\lambda)$,

$$\{\pi \ : \ \pi = \mathsf{Shuffle}(1^N; r), r \leftarrow \{0,1\}^\rho\} \approx_s \{\pi \ : \ \pi \leftarrow \mathcal{U}(\mathcal{S}_N)\}$$

**Explainability:** For any $\lambda \in \mathbb{N}$, $N = N(\lambda)$,

$$\{(\pi, r) \ : \ \pi = \mathsf{Shuffle}(1^N; r), r \leftarrow \{0,1\}^\rho\} \approx_s \{(\pi, r) \ : \ \pi \leftarrow \mathcal{U}(\mathcal{S}_N), r \leftarrow \mathsf{Explain}(1^N, \pi)\}$$

**Construction A.2.** Let $\ell$ be a block length with $2^\ell \geq N$. Concretely, take $\ell = 2\lceil \log N \rceil + \lambda + 2$. We construct an explainable shuffler $(\mathsf{Sample}, \mathsf{Explain})$ with randomness space $\mathcal{R}_{N,\ell}$, so $\rho = (N-1)\ell$ as follows.

$\mathsf{Sample}(1^N; r)$**.** Parse $r$ as $(H_N, \ldots, H_2) \in \mathcal{R}_{N,\ell}$. Initialize $a[k] := k$ for $k \in [N]$. For $i = N, N-1, \ldots, 2$, compute $j_i := 1 + (H_i \bmod i) \in \{1, \ldots, i\}$ and swap $a[i] \leftrightarrow a[j_i]$. Output $\pi := (a[1], \ldots, a[N])$.

$\mathsf{Explain}(1^N, \pi)$**.** Output $\perp$ if $\pi \notin \mathcal{S}_N$. Otherwise initialize $b[k] := k$ and $\mathsf{pos}[k] := k$ for $k \in [N]$ (maintaining $\mathsf{pos} = b^{-1}$). For $i = N, N-1, \ldots, 2$,

1. Set $j_i := \mathsf{pos}[\pi[i]]$ and $t_i := j_i - 1$.
2. Set $c_i := \lfloor (2^\ell - 1 - t_i)/i \rfloor + 1$, sample $u_i \leftarrow \{0, \ldots, c_i - 1\}$, and set $H_i := t_i + u_i \cdot i$.
3. Swap $b[i] \leftrightarrow b[j_i]$ and update $\mathsf{pos}[b[i]] := i$, $\mathsf{pos}[b[j_i]] := j_i$.

Output $r' := (H_N, \ldots, H_2)$.

Note that both algorithms run in time $\mathsf{poly}(\lambda, N)$. We write $\mathsf{Sample}^*(1^N; \mathbf{j})$ for the variant of $\mathsf{Sample}$ reading digits $\mathbf{j} \in \mathcal{J}_N$ directly, and $\mathsf{Trace}(1^N, \pi) := (j_N, \ldots, j_2)$ for the digit vector computed by $\mathsf{Explain}$ in step 1.

**Lemma A.3.** For any $N = N(\lambda)$,

(i) For $\mathbf{j} \in \mathcal{J}_N$ and $\pi = \mathsf{Sample}^*(1^N; \mathbf{j})$, the array $a$ satisfies $a[k] = \pi[k]$ for all $k > i$ at start of iteration $i$.

(ii) For every $\pi \in \mathcal{S}_N$, $\mathsf{Trace}(1^N, \pi)$ is well defined, i.e. $j_i \in \{1, \ldots, i\}$ for all $i$.

(iii) $\mathsf{Sample}^*(1^N; \cdot) \colon \mathcal{J}_N \to \mathcal{S}_N$ is a bijection with inverse $\mathsf{Trace}(1^N, \cdot)$. In particular each $\pi$ has a unique digit vector.

*Proof.* (i) Iteration $i$ modifies only $a[i]$ and $a[j_i]$ with $j_i \leq i$, so no position $k > i$ is touched after iteration $k$. The claim follows by induction, as $\pi$ is the final state of $a$.

(ii) Follows by induction. At $i = N$ the array $b$ is the identity, so $\mathsf{pos}[\pi[N]] = \pi[N] \leq N$. Assume the claim through iteration $i+1$, so that $b[k] = \pi[k]$ for all $k > i$ at the start of iteration $i$ (by the locality argument of (i)). As $\pi$ is injective, $\pi[i] \notin \{\pi[k] : k > i\} = \{b[k] : k > i\}$, hence $\mathsf{pos}[\pi[i]] = b^{-1}[\pi[i]] \leq i$. The swap places $\pi[i]$ at position $i$ and the update restores $\mathsf{pos} = b^{-1}$.

(iii) $\mathsf{Sample}^*$ and $\mathsf{Explain}$ perform the same swaps on $a$ and $b$ from the same initial state, so $a = b$ throughout. Now, by (ii) the final state of $b$ is $\pi$, giving $\mathsf{Sample}^*(1^N; \mathsf{Trace}(1^N, \pi)) = \pi$ and hence surjective. Since, $|\mathcal{J}_N| = N! = |\mathcal{S}_N|$, a surjection between finite sets of equal size is a bijection, and $\mathsf{Trace}$ is its inverse. □

**Lemma A.4.** Let $2^\ell \geq N$. For every $\pi \in \mathcal{S}_N$, the set $\mathcal{R}^\pi_{N,\ell} := \{r : \mathsf{Sample}(1^N; r) = \pi\}$ is nonempty, and $\mathsf{Explain}(1^N, \pi)$ is distributed uniformly over $\mathcal{R}^\pi_{N,\ell}$.

*Proof.* Let $\mathbf{j} = \mathsf{Trace}(1^N, \pi)$ and $t_i = j_i - 1$. By Lemma A.3 item (iii), $\mathsf{Sample}(1^N; r) = \pi$ holds iff $1 + (H_i \bmod i) = j_i$ for every $i$, i.e. iff $H_i \equiv t_i \pmod i$. Hence, $\mathcal{R}^\pi_{N,\ell} = A_2 \times \ldots \times A_N$ where $A_i = \{h \in \{0, \ldots, 2^\ell - 1\} : h \equiv t_i \bmod i\}$. Since $0 \leq t_i \leq i - 1 < N \leq 2^\ell$ we have $t_i \in A_i$, so $A_i \neq \emptyset$. Moreover, $A_i = \{t_i + u \cdot i : 0 \leq u < c_i\}$ with $|A_i| = \lfloor (2^\ell - 1 - t_i)/i \rfloor + 1 = c_i$. The algorithm samples $u_i$ uniformly and independently for each $i$, so its output is uniform on $A_2 \times \ldots \times A_N$. □

**Observation A.5.** Let $i, \ell \in \mathbb{N}$ with $1 \leq i \leq 2^\ell$. Then for $H \leftarrow \{0,1\}^\ell$, interpreted as an integer in $\{0, \ldots, 2^\ell - 1\}$, $\mathsf{SD}\big(H \bmod i, \mathcal{U}(\{0, \ldots, i-1\})\big) \leq i/2^{\ell+2}$.

**Theorem A.6.** Let $\ell \geq 2 \log N + \lambda + 2$. Then Construction A.2 is an explainable shuffler with $\rho = (N-1)\ell$.

*Proof.* **Sampling distribution.** The blocks $H_i$ are independent and uniform, so the digits $j_i = 1 + (H_i \bmod i)$ are independent, and by Observation A.5 the $i$-th lies within $i/2^{\ell+2}$ of $\mathcal{U}(\{1,\dots,i\})$. Over $N-1$ coordinates,

$$\mathsf{SD}\big(\mathcal{D}_{\mathbf{j}}, \mathcal{U}(\mathcal{J}_N)\big) \le \sum_{i=2}^{N} \frac{i}{2^{\ell+2}} = \frac{1}{2^{\ell+2}}\left(\tfrac{N(N+1)}{2} - 1\right) < \frac{N^2}{2^{\ell+2}}.$$

By Lemma A.3 item (iii), $\mathsf{Sample}^*$ maps $\mathcal{U}(\mathcal{J}_N)$ to $\mathcal{U}(\mathcal{S}_N)$, and statistical distance is preserved under a fixed function. Finally $\ell \ge 2\log N + \lambda + 2$ gives $2^{\ell+2} \ge N^2 2^{\lambda+4}$, so the bound is at most $2^{-\lambda}$.

**Explainability.** Let $P$ be the distribution of $\mathsf{Sample}(1^N; r)$ for uniform $r$, let $U = \mathcal{U}(\mathcal{S}_N)$, and let $Q(\cdot \mid \pi) = \mathcal{U}(\mathcal{R}^{\pi}_{N,\ell})$. By Lemma A.4 the conditional distribution of $r$ given $\pi$ is $Q(\cdot \mid \pi)$ in both experiments. Hence the joints differ only in the marginal on $\pi$, and statistical distance between these two distributions is $\frac{1}{2}\sum_{\pi} |P(\pi) - U(\pi)| \sum_r Q(r \mid \pi) = \mathsf{SD}(P, U) \le \mathsf{negl}(\lambda)$. □

# B Additional Proofs

This section is the same as [GY26a, Appendix A]. We provide this here for completeness.

**Lemma B.1** ([GPV08, MP12])**.** Let $n, m, q$ be lattice parameters such that $m = \Omega(n \log q)$, $q$ is an odd prime. Then, for $\sigma > \log m$, all but negligible fraction of matrices $\mathbf{B} \in \mathbb{Z}_q^{n \times m}$,

$$\{(\mathbf{B}, \mathbf{B}_\sigma^{-1}(\mathbf{y}), \mathbf{y}) : \mathbf{y} \leftarrow \mathbb{Z}_q^n\} \approx_s \{(\mathbf{B}, \mathbf{z}, \mathbf{Bz}) : \mathbf{z} \leftarrow \mathcal{D}_{\mathbb{Z},\sigma}^m\}$$

This can also be extended to matrices naturally by sampling column-wise.

**Lemma B.2** ([CHW25, WW25])**.** Let $n, m, q$ be lattice parameters such that $m = \Omega(n \log q)$, $q$ is an odd prime. Then, for any $\mathbf{y} \in \mathbb{Z}_q^n, \mathbf{z} \in \mathbb{Z}_q^m$, all but negligible fraction of matrices $\mathbf{B} \in \mathbb{Z}_q^{n \times m}$,

$$\{(\mathbf{B}, \mathbf{B}_\sigma^{-1}(\mathbf{y} + \mathbf{Bz}))\} \approx_s \{(\mathbf{B}, \mathbf{B}_\sigma^{-1}(\mathbf{y}) + \mathbf{z})\}$$

where $\sigma > \lambda^{\omega(1)}\sqrt{m}\,\|\mathbf{z}\|$. This can also be extended to matrices naturally by sampling column-wise.

Essentially, in this section, we prove that these lemmas hold with respect to the structured matrix $\mathbf{B}$ from decomposed LWE assumption.

**Lemma B.3.** For positive integers $n, m$, and odd prime $q$, such that $m = \Omega(n \log q)$, for any $\ell > 2, \ell = \mathsf{poly}(\lambda), L = \mathsf{poly}(\lambda)$, $\mathbf{W} \leftarrow \mathbb{Z}_q^{n \times \ell m}, \mathbf{D} \leftarrow \mathcal{D}_{\mathbb{Z},\sigma}^{m \times \ell m}, \mathbf{B} = \mathbf{W}(\mathbf{I}_\ell \otimes \mathbf{D}) + (\mathsf{vec}(\mathbf{I}_\ell))^\top \otimes \mathbf{G})$,

$$\{(\mathbf{W}, \mathbf{D}, \mathbf{B}, \mathbf{BR}, \mathbf{e}^\top \mathbf{R})\} \approx_s \{(\mathbf{W}, \mathbf{D}, \mathbf{B}, \mathbf{C}, \mathbf{e}^\top \mathbf{R})\}$$

where $\mathbf{R} \leftarrow \{\pm 1\}^{\ell^2 m \times L}, \sigma = \omega(\sqrt{\log m}), \mathbf{C} \leftarrow \mathbb{Z}_q^{n \times L}$, $\mathbf{e} \in \mathbb{Z}_q^{\ell^2 m}$ is any fixed vector.

*Proof.* Parse $\mathbf{W}$ as $[\mathbf{W}_1 \mid \ldots \mid \mathbf{W}_\ell] \in \mathbb{Z}_q^{n \times \ell m}$, $\mathbf{D}$ as $[\mathbf{D}_1 \mid \ldots \mid \mathbf{D}_\ell] \in \mathbb{Z}_q^{m \times \ell m}$.

$$
\left\{ (\mathbf{W}, \mathbf{D}, \mathbf{B}, \mathbf{BR}, \mathbf{e}^\top \mathbf{R}) : \begin{array}{l} \mathbf{W} \leftarrow \mathbb{Z}_q^{n \times \ell m}, \mathbf{D} \leftarrow \mathcal{D}_{\mathbb{Z},\sigma}^{m \times \ell m}, \\ \mathbf{B} = \mathbf{W}(\mathbf{I}_\ell \otimes \mathbf{D}) + \mathsf{vec}(\mathbf{I}_\ell)^\top \otimes \mathbf{G}, \\ \mathbf{R} \leftarrow \{\pm 1\}^{\ell^2 m \times L} \end{array} \right\}
$$

$$
\approx_s \left\{ (\mathbf{W}, \mathbf{D}, \mathbf{B}, \mathbf{BR}, \mathbf{e}^\top \mathbf{R}) : \begin{array}{l} (\mathbf{W}_1, T_{\mathbf{W}_1}) \leftarrow \mathsf{TrapGen}(1^n, 1^m, q), \\ [\mathbf{W}_2 \mid \ldots \mid \mathbf{W}_\ell] \leftarrow \mathbb{Z}_q^{n \times (\ell-1)m}, \\ \mathbf{D} \leftarrow \mathcal{D}_{\mathbb{Z},\sigma}^{m \times \ell m}, \\ \mathbf{B} = \mathbf{W}(\mathbf{I}_\ell \otimes \mathbf{D}) + \mathsf{vec}(\mathbf{I}_\ell)^\top \otimes \mathbf{G}, \\ \mathbf{R} \leftarrow \{\pm 1\}^{\ell^2 m \times L} \end{array} \right\} \left( \begin{array}{l} \text{By matrix distribution of} \\ \text{gadget trapdoors} \end{array} \right)
$$

$$
\approx_s \left\{ (\mathbf{W}, \mathbf{D}, \mathbf{B}, \mathbf{BR}, \mathbf{e}^\top \mathbf{R}) : \begin{array}{l} (\mathbf{W}_1, T_{\mathbf{W}_1}) \leftarrow \mathsf{TrapGen}(1^n, 1^m, q), \\ [\mathbf{W}_2 \mid \ldots \mid \mathbf{W}_\ell] \leftarrow \mathbb{Z}_q^{n \times (\ell-1)m}, \\ \mathbf{A} \leftarrow \mathbb{Z}_q^{n \times m}, \mathbf{D}_2 \leftarrow \mathsf{SamplePre}(\mathbf{W}_1, T_{\mathbf{W}_1}, \mathbf{A}, \sigma), \\ [\mathbf{D}_1 \mid \mathbf{D}_3 \mid \ldots \mid \mathbf{D}_\ell] \leftarrow \mathcal{D}_{\mathbb{Z},\sigma}^{m \times (\ell-1)m}, \\ \mathbf{B} = \mathbf{W}(\mathbf{I}_\ell \otimes \mathbf{D}) + \mathsf{vec}(\mathbf{I}_\ell)^\top \otimes \mathbf{G}, \\ \mathbf{R} = \left[\mathbf{R}_1^\top \mid \mathbf{R}_2^\top \mid \ldots \mathbf{R}_{\ell^2}^\top\right]^\top \leftarrow \{\pm 1\}^{\ell^2 m \times L} \end{array} \right\} \text{(By Lemma B.1)}
$$

$$
\equiv \left\{ (\mathbf{W}, \mathbf{D}, \mathbf{B}, \mathbf{AR}_2 + \mathbf{X}, \mathbf{e}^\top \mathbf{R}) : \begin{array}{l} (\mathbf{W}_1, T_{\mathbf{W}_1}) \leftarrow \mathsf{TrapGen}(1^n, 1^m, q), \\ [\mathbf{W}_2 \mid \ldots \mid \mathbf{W}_\ell] \leftarrow \mathbb{Z}_q^{n \times (\ell-1)m}, \\ \mathbf{A} \leftarrow \mathbb{Z}_q^{n \times m}, \mathbf{D}_2 \leftarrow \mathsf{SamplePre}(\mathbf{W}_1, T_{\mathbf{W}_1}, \mathbf{A}, \sigma), \\ [\mathbf{D}_1 \mid \mathbf{D}_3 \mid \ldots \mid \mathbf{D}_\ell] \leftarrow \mathcal{D}_{\mathbb{Z},\sigma}^{m \times (\ell-1)m}, \\ \mathbf{B} = \mathbf{W}(\mathbf{I}_\ell \otimes \mathbf{D}) + \mathsf{vec}(\mathbf{I}_\ell)^\top \otimes \mathbf{G}, \\ \mathbf{R} = \left[\mathbf{R}_1^\top \mid \mathbf{R}_2^\top \mid \ldots \mid \mathbf{R}_{\ell^2}^\top\right]^\top \leftarrow \{\pm 1\}^{\ell^2 m \times L} \end{array} \right\}
$$

where $\mathbf{X} = \sum_{(i,j) \in [\ell] \times [\ell] \setminus \{(1,2)\}} (\mathbf{W}_i \mathbf{D}_j + \delta_{ij} \mathbf{G}) \mathbf{R}_{(i-1)\ell + j}$ and $\delta_{ij} = 1$ if and only if $i = j$.

$$
\approx_s \left\{ (\mathbf{W}, \mathbf{D}, \mathbf{B}, \mathbf{C}, \mathbf{e}^\top \mathbf{R}) : \begin{array}{l} (\mathbf{W}_1, T_{\mathbf{W}_1}) \leftarrow \mathsf{TrapGen}(1^n, 1^m, q), \\ [\mathbf{W}_2 \mid \ldots \mid \mathbf{W}_\ell] \leftarrow \mathbb{Z}_q^{n \times (\ell-1)m}, \\ \mathbf{A} \leftarrow \mathbb{Z}_q^{n \times m}, \mathbf{D}_2 \leftarrow \mathsf{SamplePre}(\mathbf{W}_1, T_{\mathbf{W}_1}, \mathbf{A}, \sigma), \\ [\mathbf{D}_1 \mid \mathbf{D}_3 \mid \ldots \mid \mathbf{D}_\ell] \leftarrow \mathcal{D}_{\mathbb{Z},\sigma}^{m \times (\ell-1)m}, \\ \mathbf{B} = \mathbf{W}(\mathbf{I}_\ell \otimes \mathbf{D}) + \mathsf{vec}(\mathbf{I}_\ell)^\top \otimes \mathbf{G}, \\ \mathbf{C} \leftarrow \mathbb{Z}_q^{n \times L}, \mathbf{R} \leftarrow \{\pm 1\}^{\ell^2 m \times L} \end{array} \right\} \text{(By Lemma 3.2)}
$$

$$
\approx_s \left\{ (\mathbf{W}, \mathbf{D}, \mathbf{B}, \mathbf{C}, \mathbf{e}^\top \mathbf{R}) : \begin{array}{l} (\mathbf{W}_1, T_{\mathbf{W}_1}) \leftarrow \mathsf{TrapGen}(1^n, 1^m, q), \\ [\mathbf{W}_2 \mid \ldots \mid \mathbf{W}_\ell] \leftarrow \mathbb{Z}_q^{n \times (\ell-1)m}, \\ \mathbf{D} \leftarrow \mathcal{D}_{\mathbb{Z},\sigma}^{m \times \ell m}, \\ \mathbf{B} = \mathbf{W}(\mathbf{I}_\ell \otimes \mathbf{D}) + \mathsf{vec}(\mathbf{I}_\ell)^\top \otimes \mathbf{G}, \\ \mathbf{C} \leftarrow \mathbb{Z}_q^{n \times L}, \mathbf{R} \leftarrow \{\pm 1\}^{\ell^2 m \times L} \end{array} \right\} \text{(By Lemma B.1)}
$$

$$
\approx_s \left\{ (\mathbf{W}, \mathbf{D}, \mathbf{B}, \mathbf{C}, \mathbf{e}^\top \mathbf{R}) : \begin{array}{l} \mathbf{W} \leftarrow \mathbb{Z}_q^{n \times \ell m}, \mathbf{D} \leftarrow \mathcal{D}_{\mathbb{Z},\sigma}^{m \times \ell m}, \\ \mathbf{B} = \mathbf{W}(\mathbf{I}_\ell \otimes \mathbf{D}) + \mathsf{vec}(\mathbf{I}_\ell)^\top \otimes \mathbf{G}, \\ \mathbf{C} \leftarrow \mathbb{Z}_q^{n \times L}, \mathbf{R} \leftarrow \{\pm 1\}^{\ell^2 m \times L} \end{array} \right\} \left( \begin{array}{l} \text{By matrix distribution of} \\ \text{gadget trapdoors} \end{array} \right)
$$

□

**Lemma B.4.** Let $n, m, q$ be lattice parameters such that $m = \Omega(n \log q)$, $q$ is an odd prime. Then, for

$\sigma > \log m$,

$$\left\{(\mathbf{W},\mathbf{D},\mathbf{B}) : \mathbf{W} \leftarrow \mathbb{Z}_q^{n\times 2m^3}, \mathbf{D} \leftarrow \mathcal{D}_{\mathbb{Z},\sigma}^{m\times 2m^3}, \mathbf{B} = \mathbf{W}(\mathbf{I}_{2m^2}\otimes\mathbf{D}) + \mathsf{vec}(\mathbf{I}_{2m^2})^\top \otimes \mathbf{G}\right\} \approx_s$$

$$\left\{(\mathbf{W},\mathbf{D},\mathbf{B}) : \begin{array}{l} (\mathbf{W}_1, T_{\mathbf{W}_1}) \leftarrow \mathsf{TrapGen}(1^n, 1^m, q), \\ [\mathbf{W}_2 \mid \ldots \mid \mathbf{W}_{2m^2}] \leftarrow \mathbb{Z}_q^{n\times m}, \\ (\mathbf{A}, T_{\mathbf{A}}) \leftarrow \mathsf{TrapGen}(1^n, 1^m, q), \\ \mathbf{D}_2 \leftarrow \mathsf{SamplePre}(\mathbf{W}_1, T_{\mathbf{W}_1}, \mathbf{A}, \sigma), \\ [\mathbf{D}_1 \mid \mathbf{D}_3 \mid \ldots \mid \mathbf{D}_{2m^2}] \leftarrow \mathcal{D}_{\mathbb{Z},\sigma}^{m\times(2m^2-1)m}, \\ \mathbf{B} = \mathbf{W}(\mathbf{I}_{2m^2}\otimes\mathbf{D}) + \mathsf{vec}(\mathbf{I}_{2m^2})^\top \otimes \mathbf{G} \end{array}\right\}$$

*Proof.* Parse $\mathbf{W}$ as $[\mathbf{W}_1 \mid \ldots \mid \mathbf{W}_{2m^2}] \in \mathbb{Z}_q^{n\times 2m^3}$, $\mathbf{D}$ as $[\mathbf{D}_1 \mid \ldots \mid \mathbf{D}_{2m^2}] \in \mathbb{Z}_q^{m\times 2m^3}$.

$$\left\{(\mathbf{W},\mathbf{D},\mathbf{B}) : \mathbf{W} \leftarrow \mathbb{Z}_q^{n\times 2m^3}, \mathbf{D} \leftarrow \mathcal{D}_{\mathbb{Z},\sigma}^{m\times 2m^3}, \mathbf{B} = \mathbf{W}(\mathbf{I}_{2m^2}\otimes\mathbf{D}) + \mathsf{vec}(\mathbf{I}_{2m^2})^\top \otimes \mathbf{G}\right\}$$

$$\approx_s \left\{(\mathbf{W},\mathbf{D},\mathbf{B}) : \begin{array}{l} (\mathbf{W}_1, T_{\mathbf{W}_1}) \leftarrow \mathsf{TrapGen}(1^n, 1^m, q), \\ [\mathbf{W}_2 \mid \ldots \mid \mathbf{W}_{2m^2}] \leftarrow \mathbb{Z}_q^{n\times(2m^2-1)m}, \\ \mathbf{D} \leftarrow \mathcal{D}_{\mathbb{Z},\sigma}^{m\times 2m^3}, \\ \mathbf{B} = \mathbf{W}(\mathbf{I}_{2m^2}\otimes\mathbf{D}) + \mathsf{vec}(\mathbf{I}_{2m^2})^\top \otimes \mathbf{G} \end{array}\right\} \quad \left(\begin{array}{l}\text{By matrix distribution of} \\ \text{gadget trapdoors}\end{array}\right)$$

$$\approx_s \left\{(\mathbf{W},\mathbf{D},\mathbf{B}) : \begin{array}{l} (\mathbf{W}_1, T_{\mathbf{W}_1}) \leftarrow \mathsf{TrapGen}(1^n, 1^m, q), \\ [\mathbf{W}_2 \mid \ldots \mid \mathbf{W}_{2m^2}] \leftarrow \mathbb{Z}_q^{n\times m}, \\ \mathbf{A} \leftarrow \mathbb{Z}_q^{n\times m}, \mathbf{D}_2 \leftarrow \mathsf{SamplePre}(\mathbf{W}_1, T_{\mathbf{W}_1}, \mathbf{A}, \sigma), \\ [\mathbf{D}_1 \mid \mathbf{D}_3 \mid \ldots \mid \mathbf{D}_{2m^2}] \leftarrow \mathcal{D}_{\mathbb{Z},\sigma}^{m\times(2m^2-1)m}, \\ \mathbf{B} = \mathbf{W}(\mathbf{I}_{2m^2}\otimes\mathbf{D}) + \mathsf{vec}(\mathbf{I}_{2m^2})^\top \otimes \mathbf{G} \end{array}\right\} \quad \text{(By Lemma B.1)}$$

$$\approx_s \left\{(\mathbf{W},\mathbf{D},\mathbf{B}) : \begin{array}{l} (\mathbf{W}_1, T_{\mathbf{W}_1}) \leftarrow \mathsf{TrapGen}(1^n, 1^m, q), \\ [\mathbf{W}_2 \mid \ldots \mid \mathbf{W}_{2m^2}] \leftarrow \mathbb{Z}_q^{n\times m}, \\ (\mathbf{A}, T_{\mathbf{A}}) \leftarrow \mathsf{TrapGen}(1^n, 1^m, q), \\ \mathbf{D}_2 \leftarrow \mathsf{SamplePre}(\mathbf{W}_1, T_{\mathbf{W}_1}, \mathbf{A}, \sigma), \\ [\mathbf{D}_1 \mid \mathbf{D}_3 \mid \ldots \mid \mathbf{D}_{2m^2}] \leftarrow \mathcal{D}_{\mathbb{Z},\sigma}^{m\times(2m^2-1)m}, \\ \mathbf{B} = \mathbf{W}(\mathbf{I}_{2m^2}\otimes\mathbf{D}) + \mathsf{vec}(\mathbf{I}_{2m^2})^\top \otimes \mathbf{G} \end{array}\right\} \quad \left(\begin{array}{l}\text{By matrix distribution of} \\ \text{gadget trapdoors}\end{array}\right)$$

□

**Lemma B.5.** Let $n, m, q$ be lattice parameters such that $m = \Omega(n \log q)$, $q$ is an odd prime. Then, for $\sigma > \log m$, $\ell > 3, \ell = \mathsf{poly}(\lambda)$, any $L = \mathsf{poly}(\lambda)$,

$$\left\{(\mathbf{W},\mathbf{D},\mathbf{B},\mathbf{BR},\mathbf{e}^\top\mathbf{R}) : \begin{array}{l} (\mathbf{W}_1, T_{\mathbf{W}_1}) \leftarrow \mathsf{TrapGen}(1^n, 1^m, q), \\ [\mathbf{W}_2 \mid \ldots \mid \mathbf{W}_{2m^2}] \leftarrow \mathbb{Z}_q^{n\times m}, \\ (\mathbf{A}, T_{\mathbf{A}}) \leftarrow \mathsf{TrapGen}(1^n, 1^m, q), \\ \mathbf{D}_2 \leftarrow \mathsf{SamplePre}(\mathbf{W}_1, T_{\mathbf{W}_1}, \mathbf{A}, \sigma), \\ [\mathbf{D}_1 \mid \mathbf{D}_3 \mid \ldots \mid \mathbf{D}_{2m^2}] \leftarrow \mathcal{D}_{\mathbb{Z},\sigma}^{m\times(2m^2-1)m}, \\ \mathbf{B} = \mathbf{W}(\mathbf{I}_{2m^2}\otimes\mathbf{D}) + \mathsf{vec}(\mathbf{I}_{2m^2})^\top \otimes \mathbf{G}, \\ \mathbf{R} \leftarrow \{\pm 1\}^{4m^5\times L} \end{array}\right\} \approx_s$$

$$\left\{(\mathbf{W},\mathbf{D},\mathbf{B},\mathbf{C},\mathbf{e}^\top\mathbf{R}) : \begin{array}{l} (\mathbf{W}_1, T_{\mathbf{W}_1}) \leftarrow \mathsf{TrapGen}(1^n, 1^m, q), \\ [\mathbf{W}_2 \mid \ldots \mid \mathbf{W}_{2m^2}] \leftarrow \mathbb{Z}_q^{n\times m}, \\ (\mathbf{A}, T_{\mathbf{A}}) \leftarrow \mathsf{TrapGen}(1^n, 1^m, q), \\ \mathbf{D}_2 \leftarrow \mathsf{SamplePre}(\mathbf{W}_1, T_{\mathbf{W}_1}, \mathbf{A}, \sigma), \\ [\mathbf{D}_1 \mid \mathbf{D}_3 \mid \ldots \mid \mathbf{D}_{2m^2}] \leftarrow \mathcal{D}_{\mathbb{Z},\sigma}^{m\times(2m^2-1)m}, \\ \mathbf{B} = \mathbf{W}(\mathbf{I}_{2m^2}\otimes\mathbf{D}) + \mathsf{vec}(\mathbf{I}_{2m^2})^\top \otimes \mathbf{G}, \\ \mathbf{C} \leftarrow \mathbb{Z}_q^{n\times L}, \mathbf{R} \leftarrow \{\pm 1\}^{4m^5\times L} \end{array}\right\}$$

where $\mathbf{e} \in \mathbb{Z}_q^{4m^5}$ is any fixed vector.

*Proof.* Parse $\mathbf{W}$ as $[\mathbf{W}_1 \mid \ldots \mid \mathbf{W}_{2m^2}] \in \mathbb{Z}_q^{n \times 2m^3}$, $\mathbf{D}$ as $[\mathbf{D}_1 \mid \ldots \mid \mathbf{D}_{2m^2}] \in \mathbb{Z}_q^{m \times 2m^3}$.

$$\left\{ (\mathbf{W}, \mathbf{D}, \mathbf{B}, \mathbf{BR}, \mathbf{e}^\top \mathbf{R}) : \begin{array}{l} (\mathbf{W}_1, T_{\mathbf{W}_1}) \leftarrow \mathsf{TrapGen}(1^n, 1^m, q), \\ [\mathbf{W}_2 \mid \ldots \mid \mathbf{W}_{2m^2}] \leftarrow \mathbb{Z}_q^{n \times m}, \\ (\mathbf{A}, T_{\mathbf{A}}) \leftarrow \mathsf{TrapGen}(1^n, 1^m, q), \\ \mathbf{D}_2 \leftarrow \mathsf{SamplePre}(\mathbf{W}_1, T_{\mathbf{W}_1}, \mathbf{A}, \sigma), \\ [\mathbf{D}_1 \mid \mathbf{D}_3 \mid \ldots \mid \mathbf{D}_{2m^2}] \leftarrow \mathcal{D}_{\mathbb{Z},\sigma}^{m \times (2m^2-1)m}, \\ T_{\mathbf{B}} = \mathbb{1}_{4m^4,2} \otimes T_{\mathbf{A}}, \\ \mathbf{B} = \mathbf{W}(\mathbf{I}_{2m^2} \otimes \mathbf{D}) + \mathsf{vec}(\mathbf{I}_{2m^2})^\top \otimes \mathbf{G}, \\ \mathbf{R} \leftarrow \{\pm 1\}^{4m^5 \times L} \end{array} \right\}$$

$$\approx_s \left\{ (\mathbf{W}, \mathbf{D}, \mathbf{B}, \mathbf{BR}, \mathbf{e}^\top \mathbf{R}) : \begin{array}{l} (\mathbf{W}_1, T_{\mathbf{W}_1}) \leftarrow \mathsf{TrapGen}(1^n, 1^m, q), \\ [\mathbf{W}_2 \mid \ldots \mid \mathbf{W}_{2m^2}] \leftarrow \mathbb{Z}_q^{n \times (2m^2-1)m}, \\ (\mathbf{A}, T_{\mathbf{A}}) \leftarrow \mathsf{TrapGen}(1^n, 1^m, q), \\ \mathbf{D}_2 \leftarrow \mathsf{SamplePre}(\mathbf{W}_1, T_{\mathbf{W}_1}, \mathbf{A}, \sigma), \\ \mathbf{A}' \leftarrow \mathbb{Z}_q^{n \times m}, \mathbf{D}_3 \leftarrow \mathsf{SamplePre}(\mathbf{W}_1, T_{\mathbf{W}_1}, \mathbf{A}', \sigma), \\ [\mathbf{D}_1 \mid \mathbf{D}_4 \mid \ldots \mid \mathbf{D}_{2m^2}] \leftarrow \mathcal{D}_{\mathbb{Z},\sigma}^{m \times (2m^2-2)m}, \\ T_{\mathbf{B}} = \mathbb{1}_{4m^4,2} \otimes T_{\mathbf{A}}, \\ \mathbf{B} = \mathbf{W}(\mathbf{I}_{2m^2} \otimes \mathbf{D}) + \mathsf{vec}(\mathbf{I}_{2m^2})^\top \otimes \mathbf{G}, \\ \mathbf{R} = \left[\mathbf{R}_1^\top \mid \mathbf{R}_2^\top \mid \mathbf{R}_3^\top \mid \ldots \mid \mathbf{R}_{4m^4}^\top\right]^\top \leftarrow \{\pm 1\}^{4m^5 \times L} \end{array} \right\} \text{(By Lemma B.1)}$$

$$\equiv \left\{ (\mathbf{W}, \mathbf{D}, \mathbf{B}, \mathbf{A}'\mathbf{R}_3 + \mathbf{X}, \mathbf{e}^\top \mathbf{R}) : \begin{array}{l} (\mathbf{W}_1, T_{\mathbf{W}_1}) \leftarrow \mathsf{TrapGen}(1^n, 1^m, q), \\ [\mathbf{W}_2 \mid \ldots \mid \mathbf{W}_{2m^2}] \leftarrow \mathbb{Z}_q^{n \times (2m^2-1)m}, \\ (\mathbf{A}, T_{\mathbf{A}}) \leftarrow \mathsf{TrapGen}(1^n, 1^m, q), \\ \mathbf{D}_2 \leftarrow \mathsf{SamplePre}(\mathbf{W}_1, T_{\mathbf{W}_1}, \mathbf{A}, \sigma), \\ \mathbf{A}' \leftarrow \mathbb{Z}_q^{n \times m}, \mathbf{D}_3 \leftarrow \mathsf{SamplePre}(\mathbf{W}_1, T_{\mathbf{W}_1}, \mathbf{A}', \sigma), \\ [\mathbf{D}_1 \mid \mathbf{D}_4 \mid \ldots \mid \mathbf{D}_{2m^2}] \leftarrow \mathcal{D}_{\mathbb{Z},\sigma}^{m \times (2m^2-2)m}, \\ T_{\mathbf{B}} = \mathbb{1}_{4m^4,2} \otimes T_{\mathbf{A}}, \\ \mathbf{B} = \mathbf{W}(\mathbf{I}_{2m^2} \otimes \mathbf{D}) + \mathsf{vec}(\mathbf{I}_{2m^2})^\top \otimes \mathbf{G}, \\ \mathbf{R} = \left[\mathbf{R}_1^\top \mid \mathbf{R}_2^\top \mid \mathbf{R}_3^\top \mid \ldots \mid \mathbf{R}_{4m^4}^\top\right]^\top \leftarrow \{\pm 1\}^{4m^5 \times L} \end{array} \right\}$$

where $\mathbf{X} = \sum_{(i,j)\in[2m^2]\times[2m^2]\setminus\{(1,3)\}}(\mathbf{W}_i\mathbf{D}_j + \delta_{ij}\mathbf{G})\mathbf{R}_{(i-1)2m^2+j}$ and $\delta_{ij} = 1$ if and only if $i = j$.

$$\approx_s \left\{ (\mathbf{W},\mathbf{D},\mathbf{B},\mathbf{C},\mathbf{e}^\top\mathbf{R}) : \begin{array}{l} (\mathbf{W}_1, T_{\mathbf{W}_1}) \leftarrow \mathsf{TrapGen}(1^n, 1^m, q), \\ [\mathbf{W}_2 \mid \ldots \mid \mathbf{W}_{2m^2}] \leftarrow \mathbb{Z}_q^{n\times(2m^2-1)m}, \\ (\mathbf{A}, T_{\mathbf{A}}) \leftarrow \mathsf{TrapGen}(1^n, 1^m, q), \\ \mathbf{D}_2 \leftarrow \mathsf{SamplePre}(\mathbf{W}_1, T_{\mathbf{W}_1}, \mathbf{A}, \sigma), \\ \mathbf{A}' \leftarrow \mathbb{Z}_q^{n\times m}, \mathbf{D}_3 \leftarrow \mathsf{SamplePre}(\mathbf{W}_1, T_{\mathbf{W}_1}, \mathbf{A}', \sigma), \\ [\mathbf{D}_1 \mid \mathbf{D}_4 \mid \ldots \mid \mathbf{D}_{2m^2}] \leftarrow \mathcal{D}_{\mathbb{Z},\sigma}^{m\times(2m^2-2)m}, \\ T_{\mathbf{B}} = \mathbb{1}_{4m^4,2} \otimes T_{\mathbf{A}}, \\ \mathbf{B} = \mathbf{W}(\mathbf{I}_{2m^2} \otimes \mathbf{D}) + \mathsf{vec}(\mathbf{I}_{2m^2})^\top \otimes \mathbf{G}, \\ \mathbf{C} \leftarrow \mathbb{Z}_q^{n\times L}, \mathbf{R} \leftarrow \{\pm 1\}^{4m^5\times L} \end{array} \right\} \text{(By Lemma 3.2)}$$

$$\approx_s \left\{ (\mathbf{W},\mathbf{D},\mathbf{B},\mathbf{C},\mathbf{e}^\top\mathbf{R}) : \begin{array}{l} (\mathbf{W}_1, T_{\mathbf{W}_1}) \leftarrow \mathsf{TrapGen}(1^n, 1^m, q), \\ [\mathbf{W}_2 \mid \ldots \mid \mathbf{W}_{2m^2}] \leftarrow \mathbb{Z}_q^{n\times(2m^2-1)m}, \\ (\mathbf{A}, T_{\mathbf{A}}) \leftarrow \mathsf{TrapGen}(1^n, 1^m, q), \\ \mathbf{D}_2 \leftarrow \mathsf{SamplePre}(\mathbf{W}_1, T_{\mathbf{W}_1}, \mathbf{A}, \sigma), \\ [\mathbf{D}_1 \mid \mathbf{D}_3 \mid \mathbf{D}_4 \mid \ldots \mid \mathbf{D}_{2m^2}] \leftarrow \mathcal{D}_{\mathbb{Z},\sigma}^{m\times(2m^2-1)m}, \\ T_{\mathbf{B}} = \mathbb{1}_{4m^4,2} \otimes T_{\mathbf{A}}, \\ \mathbf{B} = \mathbf{W}(\mathbf{I}_{2m^2} \otimes \mathbf{D}) + \mathsf{vec}(\mathbf{I}_{2m^2})^\top \otimes \mathbf{G}, \\ \mathbf{C} \leftarrow \mathbb{Z}_q^{n\times L}, \mathbf{R} \leftarrow \{\pm 1\}^{4m^5\times L} \end{array} \right\} \text{(By Lemma B.1)}$$

Note that in the above argument, $T_{\mathbf{B}}$ is independent of $\mathbf{A}'$ and $T_{\mathbf{W}_1}$. So, this property holds even if we use $T_{\mathbf{B}}$ to generate pre-images for any random $\mathbf{y} \leftarrow \mathbb{Z}_q^n$ similar to [DKW21, Lemma 3.4]. □

**Lemma B.6.** Let $n, m, q$ be lattice parameters such that $m = \Omega(n \log q)$, $q$ is an odd prime. Then, for $\sigma > \log m$,

$$\left\{ (\mathbf{W},\mathbf{D},\mathbf{B},\mathbf{d},\mathbf{y}) : \begin{array}{l} (\mathbf{W}_1, T_{\mathbf{W}_1}) \leftarrow \mathsf{TrapGen}(1^n, 1^m, q), \\ [\mathbf{W}_2 \mid \ldots \mid \mathbf{W}_{2m^2}] \leftarrow \mathbb{Z}_q^{n\times m}, \\ (\mathbf{A}, T_{\mathbf{A}}) \leftarrow \mathsf{TrapGen}(1^n, 1^m, q), \\ \mathbf{D}_2 \leftarrow \mathsf{SamplePre}(\mathbf{W}_1, T_{\mathbf{W}_1}, \mathbf{A}, \sigma), \\ [\mathbf{D}_1 \mid \mathbf{D}_3 \mid \ldots \mid \mathbf{D}_{2m^2}] \leftarrow \mathcal{D}_{\mathbb{Z},\sigma}^{m\times(2m^2-1)m}, \\ \mathbf{B} = \mathbf{W}(\mathbf{I}_{2m^2} \otimes \mathbf{D}) + \mathsf{vec}(\mathbf{I}_{2m^2})^\top \otimes \mathbf{G}, , \\ \mathbf{y} \leftarrow \mathbb{Z}_q^n, \mathbf{d} \leftarrow \mathsf{SamplePre}(\mathbf{B}, T_{\mathbf{B}}, \mathbf{y}, \sigma) \end{array} \right\} \approx_s$$

$$\left\{ (\mathbf{W},\mathbf{D},\mathbf{B},\mathbf{d},\mathbf{B}\mathbf{d}) : \begin{array}{l} (\mathbf{W}_1, T_{\mathbf{W}_1}) \leftarrow \mathsf{TrapGen}(1^n, 1^m, q), \\ [\mathbf{W}_2 \mid \ldots \mid \mathbf{W}_{2m^2}] \leftarrow \mathbb{Z}_q^{n\times m}, \\ (\mathbf{A}, T_{\mathbf{A}}) \leftarrow \mathsf{TrapGen}(1^n, 1^m, q), \\ \mathbf{D}_2 \leftarrow \mathsf{SamplePre}(\mathbf{W}_1, T_{\mathbf{W}_1}, \mathbf{A}, \sigma), \\ [\mathbf{D}_1 \mid \mathbf{D}_3 \mid \ldots \mid \mathbf{D}_{2m^2}] \leftarrow \mathcal{D}_{\mathbb{Z},\sigma}^{m\times(2m^2-1)m}, \\ \mathbf{B} = \mathbf{W}(\mathbf{I}_{2m^2} \otimes \mathbf{D}) + \mathsf{vec}(\mathbf{I}_{2m^2})^\top \otimes \mathbf{G}, \\ \mathbf{d} \leftarrow \mathcal{D}_{\mathbb{Z},\sigma}^{4m^5} \end{array} \right\}$$

This can also be extended to matrices naturally by sampling column-wise

*Proof.* Parse $\mathbf{W}$ as $[\mathbf{W}_1 \mid \ldots \mid \mathbf{W}_{2m^2}] \in \mathbb{Z}_q^{n\times 2m^3}$, $\mathbf{D}$ as $[\mathbf{D}_1 \mid \ldots \mid \mathbf{D}_{2m^2}] \in \mathbb{Z}_q^{m\times 2m^3}$. Note that by construction, there exists a matrix $\mathbf{A}, T_{\mathbf{A}}$ inside $\mathbf{B}, T_{\mathbf{B}}$. Moreover, $\mathbf{A} = \mathbf{W}_1\mathbf{D}_2$. Parse $\mathbf{d} \in \mathbb{Z}_q^{4m^5}$ as $\mathbf{d} = [\mathbf{d}_1^\top \mid$

$\ldots \mid \mathbf{d}_{4m^4}^\top]^\top$. Let $\widetilde{\mathbf{y}} = \mathbf{B}(\mathbf{d} - \mathbb{1}_{4m^4,2} \otimes \mathbf{d}_2) \in \mathbb{Z}_q^n$ in each of these hybrids below.

$$\left\{ (\mathbf{W}, \mathbf{D}, \mathbf{B}, \mathbf{d}, \mathbf{Bd}) : \begin{array}{l} (\mathbf{W}_1, T_{\mathbf{W}_1}) \leftarrow \mathsf{TrapGen}(1^n, 1^m, q), \\ [\mathbf{W}_2 \mid \ldots \mid \mathbf{W}_{2m^2}] \leftarrow \mathbb{Z}_q^{n \times m}, \\ (\mathbf{A}, T_{\mathbf{A}}) \leftarrow \mathsf{TrapGen}(1^n, 1^m, q), \\ \mathbf{D}_2 \leftarrow \mathsf{SamplePre}(\mathbf{W}_1, T_{\mathbf{W}_1}, \mathbf{A}, \sigma), \\ [\mathbf{D}_1 \mid \mathbf{D}_3 \mid \ldots \mid \mathbf{D}_{2m^2}] \leftarrow \mathcal{D}_{\mathbb{Z},\sigma}^{m \times (2m^2-1)m}, \\ \mathbf{B} = \mathbf{W}(\mathbf{I}_{2m^2} \otimes \mathbf{D}) + \mathsf{vec}(\mathbf{I}_{2m^2})^\top \otimes \mathbf{G}, \\ \mathbf{d} \leftarrow \mathcal{D}_{\mathbb{Z},\sigma}^{4m^5} \end{array} \right\}$$

$$\equiv \left\{ (\mathbf{W}, \mathbf{D}, \mathbf{B}, \mathbf{d}, \mathbf{Ad}_2 + \widetilde{\mathbf{y}}) : \begin{array}{l} (\mathbf{W}_1, T_{\mathbf{W}_1}) \leftarrow \mathsf{TrapGen}(1^n, 1^m, q), \\ [\mathbf{W}_2 \mid \ldots \mid \mathbf{W}_{2m^2}] \leftarrow \mathbb{Z}_q^{n \times m}, \\ (\mathbf{A}, T_{\mathbf{A}}) \leftarrow \mathsf{TrapGen}(1^n, 1^m, q), \\ \mathbf{D}_2 \leftarrow \mathsf{SamplePre}(\mathbf{W}_1, T_{\mathbf{W}_1}, \mathbf{A}, \sigma), \\ [\mathbf{D}_1 \mid \mathbf{D}_3 \mid \ldots \mid \mathbf{D}_{2m^2}] \leftarrow \mathcal{D}_{\mathbb{Z},\sigma}^{m \times (2m^2-1)m}, \\ \mathbf{B} = \mathbf{W}(\mathbf{I}_{2m^2} \otimes \mathbf{D}) + \mathsf{vec}(\mathbf{I}_{2m^2})^\top \otimes \mathbf{G}, \\ \mathbf{d} \leftarrow \mathcal{D}_{\mathbb{Z},\sigma}^{4m^5} \end{array} \right\}$$

$$\approx_s \left\{ (\mathbf{W}, \mathbf{D}, \mathbf{B}, \mathbf{d}, \mathbf{y}' + \widetilde{\mathbf{y}}) : \begin{array}{l} (\mathbf{W}_1, T_{\mathbf{W}_1}) \leftarrow \mathsf{TrapGen}(1^n, 1^m, q), \\ [\mathbf{W}_2 \mid \ldots \mid \mathbf{W}_{2m^2}] \leftarrow \mathbb{Z}_q^{n \times m}, \\ (\mathbf{A}, T_{\mathbf{A}}) \leftarrow \mathsf{TrapGen}(1^n, 1^m, q), \\ \mathbf{D}_2 \leftarrow \mathsf{SamplePre}(\mathbf{W}_1, T_{\mathbf{W}_1}, \mathbf{A}, \sigma), \\ [\mathbf{D}_1 \mid \mathbf{D}_3 \mid \ldots \mid \mathbf{D}_{2m^2}] \leftarrow \mathcal{D}_{\mathbb{Z},\sigma}^{m \times (2m^2-1)m}, \\ \mathbf{B} = \mathbf{W}(\mathbf{I}_{2m^2} \otimes \mathbf{D}) + \mathsf{vec}(\mathbf{I}_{2m^2})^\top \otimes \mathbf{G}, \\ \mathbf{y}' \leftarrow \mathbb{Z}_q^n, \mathbf{d}_1 \leftarrow \mathcal{D}_{\mathbb{Z},\sigma}^m, \\ \left[\mathbf{d}_3^\top \mid \ldots \mid \mathbf{d}_{4m^4}^\top\right]^\top \leftarrow \mathcal{D}_{\mathbb{Z},\sigma}^{(4m^4-2)m}, \\ \mathbf{d}_2 \leftarrow \mathsf{SamplePre}(\mathbf{A}, T_{\mathbf{A}}, \mathbf{y}', \sigma) \end{array} \right\} \quad \text{(By Lemma B.1)}$$

$$\equiv \left\{ (\mathbf{W}, \mathbf{D}, \mathbf{B}, \mathbf{d}, \mathbf{y}) : \begin{array}{l} (\mathbf{W}_1, T_{\mathbf{W}_1}) \leftarrow \mathsf{TrapGen}(1^n, 1^m, q), \\ [\mathbf{W}_2 \mid \ldots \mid \mathbf{W}_{2m^2}] \leftarrow \mathbb{Z}_q^{n \times m}, \\ (\mathbf{A}, T_{\mathbf{A}}) \leftarrow \mathsf{TrapGen}(1^n, 1^m, q), \\ \mathbf{D}_2 \leftarrow \mathsf{SamplePre}(\mathbf{W}_1, T_{\mathbf{W}_1}, \mathbf{A}, \sigma), \\ [\mathbf{D}_1 \mid \mathbf{D}_3 \mid \ldots \mid \mathbf{D}_{2m^2}] \leftarrow \mathcal{D}_{\mathbb{Z},\sigma}^{m \times (2m^2-1)m}, \\ \mathbf{B} = \mathbf{W}(\mathbf{I}_{2m^2} \otimes \mathbf{D}) + \mathsf{vec}(\mathbf{I}_{2m^2})^\top \otimes \mathbf{G}, \\ \mathbf{y} \leftarrow \mathbb{Z}_q^n, \mathbf{d}_1 \leftarrow \mathcal{D}_{\mathbb{Z},\sigma}^m, [\mathbf{d}_3^\top \mid \ldots \mid \mathbf{d}_{4m^4}^\top]^\top \leftarrow \mathcal{D}_{\mathbb{Z},\sigma}^{(4m^4-2)m}, \\ \mathbf{d}_2 \leftarrow \mathsf{SamplePre}(\mathbf{A}, T_{\mathbf{A}}, \mathbf{y} - \widetilde{\mathbf{y}}, \sigma) \end{array} \right\}$$

$$\approx_s \left\{ (\mathbf{W}, \mathbf{D}, \mathbf{B}, \mathbf{d}, \mathbf{y}) : \begin{array}{l} (\mathbf{W}_1, T_{\mathbf{W}_1}) \leftarrow \mathsf{TrapGen}(1^n, 1^m, q), \\ [\mathbf{W}_2 \mid \ldots \mid \mathbf{W}_{2m^2}] \leftarrow \mathbb{Z}_q^{n \times m}, \\ (\mathbf{A}, T_{\mathbf{A}}) \leftarrow \mathsf{TrapGen}(1^n, 1^m, q), \\ \mathbf{D}_2 \leftarrow \mathsf{SamplePre}(\mathbf{W}_1, T_{\mathbf{W}_1}, \mathbf{A}, \sigma), \\ [\mathbf{D}_1 \mid \mathbf{D}_3 \mid \ldots \mid \mathbf{D}_{2m^2}] \leftarrow \mathcal{D}_{\mathbb{Z},\sigma}^{m \times (2m^2-1)m}, \\ \mathbf{B} = \mathbf{W}(\mathbf{I}_{2m^2} \otimes \mathbf{D}) + \mathsf{vec}(\mathbf{I}_{2m^2})^\top \otimes \mathbf{G}, \\ \mathbf{y} \leftarrow \mathbb{Z}_q^n, \mathbf{d} \leftarrow \mathsf{SamplePre}(\mathbf{B}, T_{\mathbf{B}}, \mathbf{y}, \sigma) \end{array} \right\} \quad \text{(By [ABB10])}$$

□

**Lemma B.7.** Let $n, m, q$ be lattice parameters such that $m = \Omega(n \log q)$, $q$ is an odd prime. Then, for any

$\mathbf{y} \in \mathbb{Z}_q^n, \mathbf{z} \in \mathbb{Z}_q^{4m^5}$,

$$\left\{ (\mathbf{W}, \mathbf{D}, \mathbf{B}, \mathbf{d}) : \begin{array}{l} (\mathbf{W}_1, T_{\mathbf{W}_1}) \leftarrow \mathsf{TrapGen}(1^n, 1^m, q), \\ [\mathbf{W}_2 \mid \ldots \mid \mathbf{W}_{2m^2}] \leftarrow \mathbb{Z}_q^{n \times m}, \\ (\mathbf{A}, T_{\mathbf{A}}) \leftarrow \mathsf{TrapGen}(1^n, 1^m, q), \\ \mathbf{D}_2 \leftarrow \mathsf{SamplePre}(\mathbf{W}_1, T_{\mathbf{W}_1}, \mathbf{A}, \sigma), \\ [\mathbf{D}_1 \mid \mathbf{D}_3 \mid \ldots \mid \mathbf{D}_{2m^2}] \leftarrow \mathcal{D}_{\mathbb{Z},\sigma}^{m \times (2m^2-1)m}, \\ \mathbf{B} = \mathbf{W}(\mathbf{I}_{2m^2} \otimes \mathbf{D}) + \mathsf{vec}(\mathbf{I}_{2m^2})^\top \otimes \mathbf{G}, \\ \mathbf{d} \leftarrow \mathsf{SamplePre}(\mathbf{B}, T_{\mathbf{B}}, \mathbf{y} + \mathbf{B}\mathbf{z}, \sigma) \end{array} \right\} \approx_s$$

$$\left\{ (\mathbf{W}, \mathbf{D}, \mathbf{B}, \mathbf{d}) : \begin{array}{l} (\mathbf{W}_1, T_{\mathbf{W}_1}) \leftarrow \mathsf{TrapGen}(1^n, 1^m, q), \\ [\mathbf{W}_2 \mid \ldots \mid \mathbf{W}_{2m^2}] \leftarrow \mathbb{Z}_q^{n \times m}, \\ (\mathbf{A}, T_{\mathbf{A}}) \leftarrow \mathsf{TrapGen}(1^n, 1^m, q), \\ \mathbf{D}_2 \leftarrow \mathsf{SamplePre}(\mathbf{W}_1, T_{\mathbf{W}_1}, \mathbf{A}, \sigma), \\ [\mathbf{D}_1 \mid \mathbf{D}_3 \mid \ldots \mid \mathbf{D}_{2m^2}] \leftarrow \mathcal{D}_{\mathbb{Z},\sigma}^{m \times (2m^2-1)m}, \\ \mathbf{B} = \mathbf{W}(\mathbf{I}_{2m^2} \otimes \mathbf{D}) + \mathsf{vec}(\mathbf{I}_{2m^2})^\top \otimes \mathbf{G}, \\ \mathbf{d} \leftarrow \mathsf{SamplePre}(\mathbf{B}, T_{\mathbf{B}}, \mathbf{y}, \sigma) + \mathbf{z} \end{array} \right\}$$

where $\sigma > \lambda^{\omega(1)} \sqrt{m} \, \|\mathbf{z}\|$. This can also be extended to matrices naturally by sampling column-wise.

*Proof.* This is similar to proof of Lemma B.6 except using Lemma B.2. □